DRAFT

CMPU Aged Order Analysis

Author: Mehmet Yunt

Date: November 3, 2026

1 Executive Summary

In CMPU (Customer Must Pickup and Upload) vet diet orders require that the customer pick up the their prescriptions and upload them to Chewy to get approval as a result of the approving vets preference.

This order group suffers from high cancellation rates; the top cancellation reason begin “aged order”; an automatic cancellation that happens if the order has not been shipped or otherwise been canceled by the customer after ten days from order creation. This type of cancellation is followed by explicit cancellations by the customer (“customer initiated cancellation” and “customer request”). The glaring issue is that cancellation reasons do not provide actionable root causes and require further investigative actions and analyses. Possible root causes are; the inconvenience of picking up the prescription, issues in uploading the vet prescription, wrong order, ability to fulfill the order at the vet.

This report focuses on aged order cancellations. Among CMPU vet diet orders shipped or canceled between May 1, 2026 and July 31, 2026, 38,314 orders were canceled due to aging. The number of aging orders was about 10% of the orders that were actually shipped with an estimated loss of 3,5 million, again about 10% of the net sales of the actually shipped orders (Table 1). We note that customer initiated cancellations and customer requested cancellations combined comprise a comparable-sized loss.

In the following seven days after a cancellation, 2,316 out of 30,382 customers return to shop the same part number for the same pet and successfully reorder for a total net sales of 221 thousand (Table 2) recovering 7.6% of the loss. After 28 days, Chewy recovers only about 20% of the estimated net sales. By 70 days, we have only 26% of the customers shopping the same product for the same pet. The attrition is worse for customers whose first CMPU order was the aged order; retention is 7.2% and 21.72% respectively for 7 and 70 days (11,500 customers, see Table 2). The worst segment is the new to vet diet customers; and their 70 day reorder rate is only 15% (5K customers, see Table 2). The rate of reorder in Chewy of these cohorts follows the same order 61%, 58% and 50%.

All of the customers seem to have contacted Chewy, most of them at the creation of the order (Table 7). Some of them have contacted Chewy on multiple days (Table 8). People who contacted closer to the aged order cancellation date have a higher reorder rate (Table 10).

The study analyzes first orders and order lines associated with CMPU prescriptions. Details can be found in Section 2.3. The main findings are that highly likely aged orders are caused by customers not uploading their prescription documents; and that after 3 days there is 50%+ chance that a CMPU order becomes an aged order.

That the attrition rate is higher for customers who encounter this first time hints that uploading the prescription is an onus on the customer. The customer might not be uploading the prescription because it might be hard to obtain from the vet in person, it might be hard to upload for some reason (the Chewy web site or Chewy app not working for the customer), or once the customer goes to the vet, they have the ability procure the product on site or nearby.

One suggestion to increase the upload rate (communication with Courtney Gleede) is to send reminder emails to customer. At this point, Chewy does not contact customers if there is a chance that their order might become aged. There is no Chewy relevant outbound contact data for a comparative/causal analysis. The only contact happens when the customer is notified that their order has been canceled. Assuming that people who receive an email before cancellation, react similarly to the cohort that reorders within 7 days after cancellation, we expect that out of 30K people considered in the opportunity study with aged cancellations, 2.3K would react before order cancellation corresponding to 221K sales. The best effort guess is that Chewy prevents people from skipping one reorder for this cohort for an additional gain of 221K net sales. The upper bound on the best effort estimate is 390K corresponding to the 14 day cohort. This represents about 1% increase in net sales for CMPU vet diet orders.

The effect of reminder emails on long term retention is an open question. Uploading a prescription is still an onus with or without a reminder.

1.1 Suggested Next Steps

- Chewy should contact customers who had aged orders in person or via AI (AI calls or emails) to understand the reasons of why customers have not uploaded their prescriptions and any other difficulty they may have encountered in the process. Chewy should focus on first time vet diet customers, as their retention is the worst.
- A controlled experiment where Chewy sends reminder emails to get a better understanding of the entitlement regarding short term gain is suggested. The suggestion is to pick a cohort of new vet diet customers who need to upload their prescriptions, randomly split it in half and send an email to one half to observe if Chewy can reduce the aged orders in the treatment group (customers to whom an email is sent) compared to the control (no email sent).

2 Appendix

Appendix contents

2.1 What is the financial impact of CMPU aged orders? . . . 3
2.1.1 What is the estimated net sales lost from CMPU aged orders? . . . 3
2.1.2 What percentage of customers come back and place a successful vet diet order for the same pet and part number after an aged order in the next 70 days? . . . 4
2.1.3 What percentage of customers come back and place a successful order for the same pet but different product after an aged order in the next 70 days? . . . 5
2.1.4 What percentage of customers place a successful vet diet order after an aged order in the next 70 days? . . . 6
2.1.5 What percentage of customers place a Chewy order successfully in the following 70 days after their aged order? . . . 7
2.1.6 SQL used in the financial analysis . . . 8
2.1.7 Section Reproduction Steps . . . 30
2.2 What is the contact pattern of CMPU aged order customers? . . . 31
2.2.1 What is the distribution of CMPU customers with APPROVAL CHANNEL = CUSTOMER_UPLOAD that contacted Chewy by cancellation reason? . . . 31
2.2.2 When did the CMPU aged order customers contact Chewy relative to the cancellation date? . . . 32
2.2.3 What was the frequency of contacting Chewy for CMPU aged order customers? . . . 33
2.2.4 Does more frequent contact imply higher propensity to re-order successfully for the same pet and part number in 14 days? . . . 33
2.2.5 Does contact closer to cancellation imply higher propensity to re-order successfully in 14 days for the same pet and part number? . . . 34
2.2.6 Section Reproduction Steps . . . 35
2.3 What are the delays in processing uploaded prescriptions? . . . 36
2.3.1 How long do customers wait to upload a prescription? . . . 39
2.3.2 How long does it take for an uploaded prescription to be ready for approval? . . . 39
2.3.3 How long does it take for an accepted prescription document to get approved? . . . 40
2.3.4 Section Reproduction Steps . . . 41
2.4 What is the probability that after X days the order will be an aged order? . . . 42
2.4.1 Section Reproduction Steps . . . 43
2.5 How do I reproduce the results and the report? . . . 43
2.6 What are the data sources? . . . 44
2.6.1 Where is the prescription upload data coming from? . . . 44
2.6.1.1 What's the distribution of all the vet diet prescriptions in the prescription upload tables by APPROVAL_CHANNEL ? . . . 47
2.6.1.2 What is the coverage of vet diet prescriptions with APPROVAL_CHANNEL = CUSTOMER_UPLOAD, by PRESCRIPTION_SOURCE and RX_STATUS? . . . 49
2.6.1.3 What is the distribution of order cancellation reasons in the prescription upload tables? . . . 52
2.6.1.4 What are the order cancellation reasons for null associated resource type entries in the rxp_notification_service.upload_entity table? . . . 54
2.6.2 How are data sources connected? . . . 56

DRAFT

2.1 What is the financial impact of CMPU aged orders?

Table 1 reports distinct customer and order counts, qualifying order-line counts and recorded net sales by cancellation reason for orders shipped or cancelled from May 1, 2026 through July 31, 2026, inclusive, UTC (`[2026-05-01, 2026-08-01)`, with an exclusive end date). The population requires vet-diet lines matched by prescription ID and numeric customer ID to vet-diet prescriptions with `PRESCRIPTION_SOURCE = CMPU` and `APPROVAL_CHANNEL = CUSTOMER_UPLOAD`.

2.1.1 What is the estimated net sales lost from CMPU aged orders?

Table 1: Financial opportunity baseline: cancellation reasons, customers, orders, qualifying order lines and net sales for CMPU vet-diet customer-upload orders shipped or cancelled May 1, 2026 through July 31, 2026, inclusive, UTC.

Cancellation reason	Customers	Orders	Order lines	Estimated net sales
[No recorded cancellation reason] (actual net sales)	172,878	387,707	433,051	35,821,384.85
Aged order	30,382	38,314	41,713	3,470,380.19
Customer initiated cancellation	29,455	32,939	37,959	2,945,818.13
Customer request	11,382	11,948	13,662	1,042,195.19
Duplicate order	161	165	188	14,763.64
Fraud	36	38	45	4,339.97
Inventory issue	5	5	5	583.24
No response	1,035	1,229	1,329	111,509.37
Other	784	809	940	71,158.59
Payment declined	4,242	5,533	6,067	523,931.07
Payment issue	46	47	57	4,207.82
Pharmacy rejection	278	310	351	29,055.56
Prescription issue	5,051	5,329	5,822	480,281.17
Restricted availability	1	1	1	25.64
TOTAL	225,434	484,374	541,190	44,519,634.43

Notes: Includes all qualifying orders, including subsequent orders; shipment or cancellation records must fall in the reporting window. There is no order-placement or prescription-creation restriction. This differs from the prescription-creation cohort and aged-cancellation cutoff used in Table 11. Counts are distinct customers, orders and physical order lines. Customers may appear in multiple reason rows; TOTAL counts each customer once. Sales include only qualifying lines, summed from `ORDER_LINE_NET_SALES` in `EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP`, once per shipment-and-line record with a shipment or cancellation timestamp in the window. Split-shipment sales outside the window are excluded; records meeting both dates count once. Zero and negative sales remain included; any missing sales make the group total incomplete. Display rounds to two decimals; exports retain five. Current cancellation reasons are grouped case-insensitively; a missing reason does not establish an uncancelled order. Orders can appear in multiple reason rows if their lines have different reasons; TOTAL counts each order once. Line counts and sales are additive. Gray reason-row net-sales amounts have a recorded cancellation reason and represent unrealized sales, not realized revenue. The gray TOTAL combines these amounts with the no-recorded-cancellation-reason amount. Recorded net sales are a baseline for opportunity sizing, not an estimate of recoverable revenue from cancelled orders.

DRAFT

2.1.2 What percentage of customers come back and place a successful vet diet order for the same pet and part number after an aged order in the next 70 days?

The three cohorts below compare non-cancelled reorders after CMPU vet-diet CUSTOMER_UPLOAD aged cancellations from May 1, 2026 through July 31, 2026, inclusive, UTC. The yellow columns show returning customers. Each cohort uses its own fixed denominator.

Table 2: Same-pet, same-product follow-up: cumulative customers, cohort percentages, distinct orders and matching-line net sales across three aged-customer cohorts.

	All aged customers $N = 30,382$				First CUSTOMER_UPLOAD vet-diet order aged; $N = 11,550$				First vet-diet order aged $N = 5,097$			
Days	Customers	Cohort (%)	Orders	Net sales	Customers	Cohort (%)	Orders	Net sales	Customers	Cohort (%)	Orders	Net sales
7	2,316	7.62%	2,407	221,122.35	833	7.21%	855	72,184.56	312	6.12%	322	23,939.00
14	3,832	12.61%	4,143	386,124.68	1,281	11.09%	1,365	116,938.34	468	9.18%	500	36,606.75
21	5,133	16.89%	5,951	558,914.70	1,633	14.14%	1,840	159,141.20	565	11.08%	642	47,225.57
28	5,850	19.25%	7,289	685,719.76	1,846	15.98%	2,256	193,780.75	631	12.38%	792	58,032.36
35	6,461	21.27%	8,905	841,298.69	2,026	17.54%	2,698	234,112.61	680	13.34%	914	67,607.06
42	6,818	22.44%	10,118	958,671.19	2,149	18.61%	3,113	272,143.01	720	14.13%	1,061	79,250.13
49	7,280	23.96%	11,898	1,131,240.46	2,297	19.89%	3,641	322,633.43	749	14.69%	1,191	89,793.43
56	7,510	24.72%	13,152	1,252,558.38	2,372	20.54%	3,995	354,230.43	766	15.03%	1,308	99,811.18
63	7,756	25.53%	14,621	1,393,281.96	2,450	21.21%	4,425	393,797.77	781	15.32%	1,425	109,761.26
70	7,944	26.15%	15,808	1,507,063.94	2,509	21.72%	4,801	427,522.19	800	15.70%	1,567	120,434.49

Cohorts and anchors: All aged customers includes every qualifying aged cancellation, without a prescription-creation or first-order restriction. For each first-order cohort, the customer's first qualifying aged order is the sole anchor; later aged orders do not reset follow-up. The first customer-upload cohort uses the earliest observed vet-diet CUSTOMER_UPLOAD order across historical prescription sources, using linked prescriptions and ECOM order history. The first vet-diet cohort uses earliest observed vet-diet orders across all historical approval channels and prescription sources, using RxP vet-diet lines and ECOM order history. Each percentage divides returning customers by its group's fixed full cohort, including customers without a qualifying reorder or with unusable matching keys. The cohorts overlap; do not add their counts.
Shared follow-up rules: Require the same numeric customer ID, exact PRODUCT_PART_NUMBER and non-null PET_ID on a different order placed strictly after the matched cancellation through the indicated day, inclusive. Later orders may use any approval channel or prescription source within the certified RxP line view. Exclude the entire later order if any line has status X, a recorded cancellation reason, a cancellation timestamp or unknown line status. Statuses and prescription channels reflect the extraction snapshot. All windows require complete follow-up through 2026-10-28 00:00 UTC. Each customer, order and matching physical line counts once per window, despite repeated anchor associations. Windows overlap and must not be added. Net sales sum ORDER_LINE_NET_SALES once per matching shipment/line contribution, preserving zero and negative amounts; missing sales make totals incomplete. Sales can be recorded after the placement window and do not prove recovered revenue. Amounts display two decimals; exports retain five. N/A denotes a zero cohort denominator.

DRAFT

2.1.3 What percentage of customers come back and place a successful order for the same pet but different product after an aged order in the next 70 days?

The three cohorts below compare non-cancelled reorders after CMPU vet-diet CUSTOMER_UPLOAD aged cancellations from May 1, 2026 through July 31, 2026, inclusive, UTC. The yellow columns show returning customers. Each cohort uses its own fixed denominator.

Table 3: Same-pet, different-product vet-diet follow-up: cumulative customers, cohort percentages, distinct orders and matching-line net sales across three aged-customer cohorts.

	All aged customers $N = 30,382$				First CUSTOMER_UPLOAD vet-diet order aged; $N = 11,550$				First vet-diet order aged $N = 5,097$			
Days	Customers	Cohort (%)	Orders	Net sales	Customers	Cohort (%)	Orders	Net sales	Customers	Cohort (%)	Orders	Net sales
7	1,332	4.38%	1,457	132,081.20	444	3.84%	466	40,916.08	181	3.55%	189	15,801.67
14	2,182	7.18%	2,553	230,877.67	727	6.29%	792	68,360.26	276	5.41%	307	24,589.86
21	2,890	9.51%	3,677	331,564.87	924	8.00%	1,089	94,359.87	332	6.51%	399	32,725.31
28	3,413	11.23%	4,690	423,982.81	1,080	9.35%	1,379	119,958.61	382	7.49%	505	40,896.95
35	3,814	12.55%	5,711	517,496.22	1,217	10.54%	1,695	147,193.63	423	8.30%	605	48,954.22
42	4,138	13.62%	6,656	602,901.60	1,314	11.38%	1,983	173,509.35	451	8.85%	704	57,525.03
49	4,421	14.55%	7,662	698,059.40	1,401	12.13%	2,289	201,286.86	479	9.40%	800	65,677.94
56	4,651	15.31%	8,624	784,739.53	1,479	12.81%	2,574	225,931.09	504	9.89%	896	73,577.63
63	4,856	15.98%	9,594	871,819.44	1,553	13.45%	2,893	253,413.39	530	10.40%	1,010	83,296.37
70	5,010	16.49%	10,552	959,049.10	1,617	14.00%	3,220	281,008.90	548	10.75%	1,122	91,928.98

Cohorts and anchors: All aged customers includes every qualifying aged cancellation, without a prescription-creation or first-order restriction. For each first-order cohort, the customer's first qualifying aged order is the sole anchor; later aged orders do not reset follow-up. The first customer-upload cohort uses the earliest observed vet-diet CUSTOMER_UPLOAD order across historical prescription sources, using linked prescriptions and ECOM order history. The first vet-diet cohort uses earliest observed vet-diet orders across all historical approval channels and prescription sources, using RxP vet-diet lines and ECOM order history. Each percentage divides returning customers by its group's fixed full cohort, including customers without a qualifying reorder or with unusable matching keys. The cohorts overlap; do not add their counts.
Shared follow-up rules: Require the same numeric customer ID and non-null PET_ID, a different nonblank PRODUCT_PART_NUMBER from the anchor line, and a TRUE vet-diet flag on the later line. Use a different order placed strictly after the matched cancellation through the indicated day, inclusive. Later orders may use any approval channel or prescription source within the certified RxP line view. Exclude the entire later order if any line has status X, a recorded cancellation reason, a cancellation timestamp or unknown line status. Statuses and prescription channels reflect the extraction snapshot. All windows require complete follow-up through 2026-10-28 00:00 UTC. Each customer, order and matching physical line counts once per window, despite repeated anchor associations. Windows overlap and must not be added. Net sales sum ORDER_LINE_NET_SALES once per matching shipment/line contribution, preserving zero and negative amounts; missing sales make totals incomplete. Sales can be recorded after the placement window and do not prove recovered revenue. Amounts display two decimals; exports retain five. N/A denotes a zero cohort denominator.

DRAFT

2.1.4 What percentage of customers place a successful vet diet order after an aged order in the next 70 days?

The three cohorts below compare non-cancelled reorders after CMPU vet-diet `CUSTOMER_UPLOAD` aged cancellations from May 1, 2026 through July 31, 2026, inclusive, UTC. The yellow columns show returning customers. Each cohort uses its own fixed denominator.

Table 4: Any vet-diet follow-up, for any pet or product: cumulative customers, cohort percentages, distinct orders and matching-line net sales across three aged-customer cohorts.

	All aged customers $N = 30,382$				First `CUSTOMER_UPLOAD` vet-diet order aged; $N = 11,550$				First vet-diet order aged $N = 5,097$			
Days	Customers	Cohort (%)	Orders	Net sales	Customers	Cohort (%)	Orders	Net sales	Customers	Cohort (%)	Orders	Net sales
7	3,594	11.83%	4,020	376,580.78	1,217	10.54%	1,305	115,132.09	438	8.59%	463	35,772.45
14	5,824	19.17%	7,052	666,449.99	1,914	16.57%	2,178	192,777.87	659	12.93%	737	55,857.40
21	7,637	25.14%	10,163	960,248.17	2,432	21.06%	3,005	267,367.22	800	15.70%	969	74,330.46
28	8,713	28.68%	12,680	1,202,863.81	2,766	23.95%	3,743	332,421.02	899	17.64%	1,208	92,469.85
35	9,555	31.45%	15,448	1,470,197.68	3,051	26.42%	4,540	405,245.66	980	19.23%	1,425	109,763.84
42	10,075	33.16%	17,772	1,694,190.10	3,240	28.05%	5,282	474,927.09	1,031	20.23%	1,653	128,552.70
49	10,631	34.99%	20,632	1,969,933.29	3,423	29.64%	6,127	555,536.18	1,072	21.03%	1,873	146,660.13
56	10,976	36.13%	23,017	2,198,639.92	3,547	30.71%	6,825	618,447.80	1,105	21.68%	2,077	163,846.89
63	11,314	37.24%	25,580	2,445,610.75	3,664	31.72%	7,624	693,176.04	1,137	22.31%	2,302	183,559.64
70	11,567	38.07%	27,878	2,665,884.12	3,753	32.49%	8,395	762,808.17	1,165	22.86%	2,541	202,269.09

Cohorts and anchors: All aged customers includes every qualifying aged cancellation, without a prescription-creation or first-order restriction. For each first-order cohort, the customer's first qualifying aged order is the sole anchor; later aged orders do not reset follow-up. The first customer-upload cohort uses the earliest observed vet-diet `CUSTOMER_UPLOAD` order across historical prescription sources, using linked prescriptions and ECOM order history. The first vet-diet cohort uses earliest observed vet-diet orders across all historical approval channels and prescription sources, using RxP vet-diet lines and ECOM order history. Each percentage divides returning customers by its group's fixed full cohort, including customers without a qualifying reorder or with unusable matching keys. The cohorts overlap; do not add their counts.
Shared follow-up rules: Require the same numeric customer ID and a TRUE vet-diet flag on the later line; purchases may be for any pet and any product part number. Use a different order placed strictly after the matched cancellation through the indicated day, inclusive. Later orders may use any approval channel or prescription source within the certified RxP line view. Exclude the entire later order if any line has status X, a recorded cancellation reason, a cancellation timestamp or unknown line status. Statuses and prescription channels reflect the extraction snapshot. All windows require complete follow-up through 2026-10-28 00:00 UTC. Each customer, order and matching physical line counts once per window, despite repeated anchor associations. Windows overlap and must not be added. Net sales sum `ORDER_LINE_NET_SALES` once per matching shipment/line contribution, preserving zero and negative amounts; missing sales make totals incomplete. Sales can be recorded after the placement window and do not prove recovered revenue. Amounts display two decimals; exports retain five. N/A denotes a zero cohort denominator.

DRAFT

2.1.5 What percentage of customers place a Chewy order successfully in the following 70 days after their aged order?

The three cohorts below compare non-cancelled reorders after CMPU vet-diet `CUSTOMER_UPLOAD` aged cancellations from May 1, 2026 through July 31, 2026, inclusive, UTC. The yellow columns show returning customers. Each cohort uses its own fixed denominator.

Table 5: All-Chewy follow-up, across all product categories: cumulative customers, cohort percentages, distinct orders and matching-line net sales across three aged-customer cohorts.

	All aged customers $N = 30,382$				First `CUSTOMER_UPLOAD` vet-diet order aged; $N = 11,550$				First vet-diet order aged $N = 5,097$			
Days	Customers	Cohort (%)	Orders	Net sales	Customers	Cohort (%)	Orders	Net sales	Customers	Cohort (%)	Orders	Net sales
7	6,477	21.32%	8,512	702,794.12	2,198	19.03%	2,653	213,634.05	826	16.21%	971	74,076.57
14	10,265	33.79%	15,918	1,307,265.44	3,492	30.23%	4,913	391,233.04	1,310	25.70%	1,784	130,104.51
21	13,193	43.42%	23,645	1,912,710.30	4,589	39.73%	7,278	560,461.37	1,780	34.92%	2,685	179,943.04
28	14,804	48.73%	30,064	2,440,684.77	5,197	45.00%	9,308	718,425.79	2,035	39.93%	3,448	232,948.35
35	15,917	52.39%	36,535	2,971,072.85	5,610	48.57%	11,423	885,226.79	2,183	42.83%	4,142	282,824.63
42	16,689	54.93%	42,616	3,471,768.55	5,938	51.41%	13,515	1,054,996.63	2,306	45.24%	4,921	340,115.01
49	17,370	57.17%	49,096	4,009,870.68	6,201	53.69%	15,674	1,231,716.11	2,402	47.13%	5,643	393,973.18
56	17,837	58.71%	55,126	4,500,734.83	6,411	55.51%	17,688	1,392,497.95	2,482	48.70%	6,370	449,207.78
63	18,249	60.07%	61,158	4,996,504.86	6,575	56.93%	19,779	1,560,537.97	2,539	49.81%	7,065	501,838.82
70	18,569	61.12%	66,946	5,473,570.84	6,712	58.11%	21,917	1,732,549.06	2,591	50.83%	7,865	561,647.94

Cohorts and anchors: All aged customers includes every qualifying aged cancellation, without a prescription-creation or first-order restriction. For each first-order cohort, the customer's first qualifying aged order is the sole anchor; later aged orders do not reset follow-up. The first customer-upload cohort uses the earliest observed vet-diet `CUSTOMER_UPLOAD` order across historical prescription sources, using linked prescriptions and ECOM order history. The first vet-diet cohort uses earliest observed vet-diet orders across all historical approval channels and prescription sources, using RxP vet-diet lines and ECOM order history. Each percentage divides returning customers by its group's fixed full cohort, including customers without a qualifying reorder or with unusable matching keys. The cohorts overlap; do not add their counts.
Shared follow-up rules: Match the same numeric customer ID only; include all Chewy product categories and any pet or product. Use a different order placed strictly after the matched cancellation through the indicated day, inclusive. Follow-up orders come from `EDLDB.ECOM.ORDER_LINE`; no later prescription, approval-channel, prescription-source, vet-diet, product or pet restriction applies. Exclude the entire later order if any line or order has status X, a recorded cancellation reason, a cancellation timestamp or unknown line status. Statuses and prescription channels reflect the extraction snapshot. All windows require complete follow-up through 2026-10-28 00:00 UTC. Each customer, order and matching physical line counts once per window, despite repeated anchor associations. Windows overlap and must not be added. Net sales sum `NET_SALES` on all qualifying ECOM order lines once per physical order line, preserving zero and negative amounts. Exact source duplicates collapse and conflicting line values fail validation; missing sales make totals incomplete. Sales can be recorded after the placement window and do not prove recovered revenue. Amounts display two decimals; ECOM supplies four-decimal amounts, retained exactly in five-decimal exports. N/A denotes a zero cohort denominator.

DRAFT

2.1.6 SQL used in the financial analysis

These are the retained, parameter-resolved queries behind Tables 1–5. The original window is May 1 inclusive to August 1 exclusive, 2026, UTC; follow-up extends through 70 days, observed through October 28, 2026, 00:00 UTC. Formatting preserves SQL tokens and the source files remain unchanged. Preparing these listings does not execute SQL or refresh data.

The baseline listing includes extraction and report aggregation. Follow-up listings are source extractions, not standalone queries for the displayed percentages. The three RxP extracts return candidates for all three matching scopes together; Python applies each endpoint, the ten cumulative 7–70-day windows and whole-order cancellation/unknown-status exclusions, then counts distinct returning customers and orders and deduplicates line sales across anchors. Percentages use each fixed full cohort denominator. RxP sales sum shipment/line contributions; Chewy sales use the full ECOM line amount once per physical line. The two history queries supply the frozen first-order cohorts.

The relevant post-extraction implementations are `src/python/create_opportunity_order_cancellation_summary.py`, `src/python/create_opportunity_aged_first_order_customers.py`, `src/python/create_opportunity_aged_reorders.py` and `src/python/create_opportunity_new_upload_customer_follow_up.py`. Local DuckDB verification SQL and audit exports remain beside each retained extract. The separate customer order-frequency analysis remains outside this report.

DRAFT

Baseline extraction and cancellation-reason aggregation

Table 1. This complete report SQL embeds the source extraction and applies prescription/customer eligibility, shipment deduplication, event-window sales selection and distinct totals. Source: `data/opportunity/order-cancellation-summary/2026-05-01_2026-08-01/shipped-or-cancelled/summary-report.sql`.

```sql
-- Financial opportunity table; UTC dates; includes TOTAL.
-- Source validation rejects conflicting line/shipment values before publication.
WITH SOURCE AS (
-- Opportunity baseline: shipped-or-cancelled reporting window [start_date, end_date), UTC.
-- Candidate vet-diet lines include every order, not only FIRST_ORDER_ID.
-- Fetch all shipment rows for candidate physical lines before validation.
WITH window_keys AS (
    SELECT DISTINCT order_id, order_line_id
    FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP
    WHERE is_vet_diet_flag = TRUE

      AND ((order_line_shipped_dttm >= '2026-05-01'::TIMESTAMP_NTZ
            AND order_line_shipped_dttm < '2026-08-01'::TIMESTAMP_NTZ)
        OR (order_cancel_dttm >= '2026-05-01'::TIMESTAMP_NTZ
            AND order_cancel_dttm < '2026-08-01'::TIMESTAMP_NTZ))

), source_lines AS (
    SELECT l.*
    FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
    JOIN window_keys k ON l.order_id IS NOT DISTINCT FROM k.order_id
                      AND l.order_line_id IS NOT DISTINCT FROM k.order_line_id
), registry AS (
    SELECT p.rx_id, MIN(p.customer_id) AS rx_customer_id,
           MIN(p.is_vet_diet_flag) AS rx_is_vet_diet_flag,
           MIN(p.approval_channel) AS rx_approval_channel,
           MIN(p.prescription_source) AS prescription_source,
           COUNT(*) AS rx_source_rows,
           COUNT(DISTINCT p.customer_id)
             + IFF(COUNT_IF(p.customer_id IS NULL)>0,1,0) AS rx_customer_variants,
           COUNT(DISTINCT p.is_vet_diet_flag)
             + IFF(COUNT_IF(p.is_vet_diet_flag IS NULL)>0,1,0) AS rx_vet_diet_variants,
           COUNT(DISTINCT p.approval_channel)
             + IFF(COUNT_IF(p.approval_channel IS NULL)>0,1,0) AS rx_channel_variants,
           COUNT(DISTINCT p.prescription_source)
             + IFF(COUNT_IF(p.prescription_source IS NULL)>0,1,0) AS rx_source_variants
    FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS p
    JOIN (SELECT DISTINCT NULLIF(TRIM(rx_id),'') AS rx_id FROM source_lines) k
      ON p.rx_id = k.rx_id
    GROUP BY p.rx_id
)
SELECT CURRENT_TIMESTAMP::VARCHAR AS extracted_at_session_time,
       CURRENT_WAREHOUSE() AS extraction_warehouse,
       l.order_id::VARCHAR AS order_id, l.order_line_id::VARCHAR AS order_line_id,
       l.shipment_key::VARCHAR AS shipment_key, l.customer_id::VARCHAR AS customer_id,
       l.rx_id AS raw_rx_id, NULLIF(TRIM(l.rx_id),'') AS rx_id,
       TO_CHAR(l.order_placed_dttm,'YYYY-MM-DD HH24:MI:SS.FF9') AS order_placed_utc,

       TO_CHAR(l.order_line_shipped_dttm,'YYYY-MM-DD HH24:MI:SS.FF9') AS shipped_at_utc,
       TO_CHAR(l.order_cancel_dttm,'YYYY-MM-DD HH24:MI:SS.FF9') AS cancelled_at_utc,

       l.is_vet_diet_flag, l.approval_channel AS line_approval_channel,
       l.order_order_line_status AS line_status, l.order_cancel_reason,
       -- Strings preserve NUMBER(28,5) exactly through either extraction engine.
       l.order_line_net_sales::VARCHAR AS order_line_net_sales,
       l.site_key::VARCHAR AS site_key, l.order_line_site_key::VARCHAR AS order_line_site_key,
       l.legal_company_description, l.order_line_transaction_type,
       p.rx_customer_id::VARCHAR AS rx_customer_id, p.rx_is_vet_diet_flag,
       p.rx_approval_channel, p.prescription_source,
       COALESCE(p.rx_source_rows,0) AS rx_source_rows,
       p.rx_customer_variants, p.rx_vet_diet_variants, p.rx_channel_variants, p.rx_source_variants
FROM source_lines l
LEFT JOIN registry p ON NULLIF(TRIM(l.rx_id),'') = p.rx_id
),
eligible_keys AS (
        SELECT DISTINCT TRY_CAST(order_id AS DECIMAL(38,0)) AS order_id,
                        TRY_CAST(order_line_id AS DECIMAL(38,0)) AS order_line_id
```

```
        FROM SOURCE
        WHERE rx_source_rows>0 AND is_vet_diet_flag=TRUE AND rx_is_vet_diet_flag=TRUE
          AND rx_approval_channel='CUSTOMER_UPLOAD' AND prescription_source='CMPU'
          AND TRY_CAST(customer_id AS DECIMAL(38,0))=TRY_CAST(rx_customer_id AS DECIMAL(38,0))

    ), financial AS (
        SELECT DISTINCT TRY_CAST(s.order_id AS DECIMAL(38,0)) AS order_id,
               TRY_CAST(s.order_line_id AS DECIMAL(38,0)) AS order_line_id,
               TRY_CAST(s.shipment_key AS DECIMAL(38,0)) AS shipment_key,
               TRY_CAST(s.customer_id AS DECIMAL(38,0)) AS customer_id,
               COALESCE(NULLIF(LOWER(TRIM(s.order_cancel_reason)),''),'[No recorded cancellation reason]') AS reason,
               CAST(s.order_line_net_sales AS DECIMAL(38,5)) AS sales
        FROM SOURCE s JOIN eligible_keys k
          ON TRY_CAST(s.order_id AS DECIMAL(38,0))=k.order_id
         AND TRY_CAST(s.order_line_id AS DECIMAL(38,0))=k.order_line_id
        WHERE ((CAST(s.shipped_at_utc AS TIMESTAMP_NTZ)>=CAST('2026-05-01' AS TIMESTAMP_NTZ)
              AND CAST(s.shipped_at_utc AS TIMESTAMP_NTZ)<CAST('2026-08-01' AS TIMESTAMP_NTZ))
           OR (CAST(s.cancelled_at_utc AS TIMESTAMP_NTZ)>=CAST('2026-05-01' AS TIMESTAMP_NTZ)
              AND CAST(s.cancelled_at_utc AS TIMESTAMP_NTZ)<CAST('2026-08-01' AS TIMESTAMP_NTZ)))
    ) SELECT 'reason' AS row_type,reason,COUNT(DISTINCT customer_id),COUNT(DISTINCT order_id),COUNT(DISTINCT order_id,
  order_line_id),
        CASE WHEN COUNT_IF(sales IS NULL)>0 THEN NULL ELSE COALESCE(SUM(sales),0) END
        FROM financial GROUP BY reason
        UNION ALL SELECT 'total','TOTAL',COUNT(DISTINCT customer_id),COUNT(DISTINCT order_id),COUNT(DISTINCT order_id,
  order_line_id),
        CASE WHEN COUNT_IF(sales IS NULL)>0 THEN NULL ELSE COALESCE(SUM(sales),0) END FROM financial
ORDER BY row_type, reason;
```

DRAFT

First customer-upload vet-diet order history

Defines the first customer-upload cohort used in the middle column group of each follow-up comparison. Python selects unambiguous first observed aged-order anchors from this history. Source: `data/opportunity/aged-first-customer-upload-vet-diet-order-customers/2026-05-01_2026-08-01/query.sql`.

```sql
-- CMPU vet-diet CUSTOMER_UPLOAD aged cancellations; first observed CUSTOMER_UPLOAD vet-diet order IDs.
-- Vet-diet CUSTOMER_UPLOAD history from RxP prescriptions; all sources/statuses/dates; ECOM chronology.
WITH registry AS (
 SELECT rx_id,MIN(customer_id) AS customer_id,
 MIN(is_vet_diet_flag) AS is_vet_diet_flag,MIN(approval_channel) AS approval_channel,
 MIN(prescription_source) AS prescription_source,
 GREATEST(COUNT(DISTINCT customer_id)+IFF(COUNT_IF(customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT is_vet_diet_flag)+IFF(COUNT_IF(is_vet_diet_flag IS NULL)>0,1,0),
 COUNT(DISTINCT approval_channel)+IFF(COUNT_IF(approval_channel IS NULL)>0,1,0),
 COUNT(DISTINCT prescription_source)+IFF(COUNT_IF(prescription_source IS NULL)>0,1,0)) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS GROUP BY rx_id
), aged_keys AS (
 SELECT l.order_id,l.order_line_id,MAX(p.registry_variants) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN registry p
 ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE
 AND p.approval_channel='CUSTOMER_UPLOAD' AND p.prescription_source='CMPU'
 AND LOWER(TRIM(l.order_cancel_reason))='aged order'
 AND l.order_cancel_dttm>='2026-05-01'::TIMESTAMP_NTZ
 AND l.order_cancel_dttm<'2026-08-01'::TIMESTAMP_NTZ
 GROUP BY l.order_id,l.order_line_id
), aged_lines AS (
 SELECT l.order_id,l.order_line_id,l.customer_id,l.order_placed_dttm AS placed_at_utc,
 l.order_cancel_dttm AS cancelled_at_utc,k.registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
), cohort_customers AS (
 SELECT DISTINCT customer_id FROM aged_lines
), vet_diet_order_keys AS (
 -- CUSTOMER_UPLOAD vet-diet history from every prescription source and date.
 SELECT l.customer_id,l.order_id,MAX(p.registry_variants) AS history_prescription_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN cohort_customers c ON l.customer_id=c.customer_id
 JOIN registry p ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE AND p.approval_channel='CUSTOMER_UPLOAD'
 GROUP BY l.customer_id,l.order_id
), history_keys AS (
 SELECT DISTINCT order_id FROM vet_diet_order_keys
), history AS (
 SELECT o.order_id,MIN(o.customer_id) AS customer_id,
 MIN(o.order_placed_dttm) AS placed_at_utc,MIN(o.order_first_order_placed_flag) AS first_placed_flag,
 COUNT(*) AS source_rows,
 GREATEST(COUNT(DISTINCT o.customer_id)+IFF(COUNT_IF(o.customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_placed_dttm)+IFF(COUNT_IF(o.order_placed_dttm IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_first_order_placed_flag)+IFF(COUNT_IF(o.order_first_order_placed_flag IS NULL)>0,1,0)) AS
  source_variants
 FROM EDLDB.ECOM.ORDERS o JOIN history_keys k ON o.order_id=k.order_id GROUP BY o.order_id
), vet_history_audit AS (
 SELECT v.customer_id,COUNT(DISTINCT IFF(h.order_id IS NULL,v.order_id,NULL)) AS vet_orders_missing_ecom_header,MAX(v.
  history_prescription_variants) AS history_prescription_variants
 FROM vet_diet_order_keys v LEFT JOIN history h ON v.order_id=h.order_id AND v.customer_id=h.customer_id
 GROUP BY v.customer_id
), ranked AS (
 SELECT *,MIN(placed_at_utc) OVER (PARTITION BY customer_id) AS earliest_placed_at_utc,
 ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY placed_at_utc NULLS LAST,order_id) AS first_rank
 FROM history
), first_orders AS (
 SELECT customer_id,COUNT(*) AS history_orders,
 COUNT_IF(placed_at_utc IS NULL) AS history_orders_missing_placement,
 MAX(source_variants) AS history_source_variants,
 SUM(source_rows-1) AS history_identical_duplicate_rows,
 MIN(earliest_placed_at_utc) AS first_placed_at_utc,
 COUNT_IF(placed_at_utc=earliest_placed_at_utc) AS earliest_timestamp_ties,
 MAX(IFF(first_rank=1,order_id,NULL)) AS first_order_id,
 COUNT_IF(first_placed_flag=TRUE) AS flagged_first_orders,
 MIN(IFF(first_placed_flag=TRUE,order_id,NULL)) AS flagged_first_order_id
```

```sql
 FROM ranked GROUP BY customer_id
)
SELECT a.*,e.customer_id AS ecom_aged_customer_id,e.placed_at_utc AS ecom_aged_placed_at_utc,
 e.first_placed_flag AS aged_order_first_placed_flag,e.source_variants AS aged_order_source_variants,
 f.history_orders,f.history_orders_missing_placement,f.history_source_variants,f.history_identical_duplicate_rows,
 v.vet_orders_missing_ecom_header,v.history_prescription_variants,f.first_order_id,f.first_placed_at_utc,f.
  earliest_timestamp_ties,f.flagged_first_orders,f.flagged_first_order_id
FROM aged_lines a LEFT JOIN history e ON a.order_id=e.order_id
LEFT JOIN first_orders f ON a.customer_id=f.customer_id
LEFT JOIN vet_history_audit v ON a.customer_id=v.customer_id
```

DRAFT

First vet-diet order history

Defines the first vet-diet cohort used in the right column group of each follow-up comparison. Python excludes unknown or ambiguous first-order histories. Source: `data/opportunity/aged-first-vet-diet-order-customers/2026-05-01_2026-08-01/query.sql`.

```sql
-- CMPU vet-diet CUSTOMER_UPLOAD aged cancellations; exact first observed vet-diet order IDs.
-- Vet-diet membership from RxP lines; chronology from ECOM.ORDERS, all channels/statuses/dates.
WITH registry AS (
 SELECT rx_id,MIN(customer_id) AS customer_id,
 MIN(is_vet_diet_flag) AS is_vet_diet_flag,MIN(approval_channel) AS approval_channel,
 MIN(prescription_source) AS prescription_source,
 GREATEST(COUNT(DISTINCT customer_id)+IFF(COUNT_IF(customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT is_vet_diet_flag)+IFF(COUNT_IF(is_vet_diet_flag IS NULL)>0,1,0),
 COUNT(DISTINCT approval_channel)+IFF(COUNT_IF(approval_channel IS NULL)>0,1,0),
 COUNT(DISTINCT prescription_source)+IFF(COUNT_IF(prescription_source IS NULL)>0,1,0)) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS GROUP BY rx_id
), aged_keys AS (
 SELECT l.order_id,l.order_line_id,MAX(p.registry_variants) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN registry p
 ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE
 AND p.approval_channel='CUSTOMER_UPLOAD' AND p.prescription_source='CMPU'
 AND LOWER(TRIM(l.order_cancel_reason))='aged order'
 AND l.order_cancel_dttm>='2026-05-01'::TIMESTAMP_NTZ
 AND l.order_cancel_dttm<'2026-08-01'::TIMESTAMP_NTZ
 GROUP BY l.order_id,l.order_line_id
), aged_lines AS (
 SELECT l.order_id,l.order_line_id,l.customer_id,l.order_placed_dttm AS placed_at_utc,
 l.order_cancel_dttm AS cancelled_at_utc,k.registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
), cohort_customers AS (
 SELECT DISTINCT customer_id FROM aged_lines
), vet_diet_order_keys AS (
 -- Full vet-diet history; no CMPU, channel, RX-ID or date restriction here.
 SELECT DISTINCT l.customer_id,l.order_id
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN cohort_customers c ON l.customer_id=c.customer_id
 WHERE l.is_vet_diet_flag=TRUE
), history_keys AS (
 SELECT DISTINCT order_id FROM vet_diet_order_keys
), history AS (
 SELECT o.order_id,MIN(o.customer_id) AS customer_id,
 MIN(o.order_placed_dttm) AS placed_at_utc,MIN(o.order_first_order_placed_flag) AS first_placed_flag,
 COUNT(*) AS source_rows,
 GREATEST(COUNT(DISTINCT o.customer_id)+IFF(COUNT_IF(o.customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_placed_dttm)+IFF(COUNT_IF(o.order_placed_dttm IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_first_order_placed_flag)+IFF(COUNT_IF(o.order_first_order_placed_flag IS NULL)>0,1,0)) AS
  source_variants
 FROM EDLDB.ECOM.ORDERS o JOIN history_keys k ON o.order_id=k.order_id GROUP BY o.order_id
), vet_history_audit AS (
 SELECT v.customer_id,COUNT(DISTINCT IFF(h.order_id IS NULL,v.order_id,NULL)) AS vet_orders_missing_ecom_header
 FROM vet_diet_order_keys v LEFT JOIN history h ON v.order_id=h.order_id AND v.customer_id=h.customer_id
 GROUP BY v.customer_id
), ranked AS (
 SELECT *,MIN(placed_at_utc) OVER (PARTITION BY customer_id) AS earliest_placed_at_utc,
 ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY placed_at_utc NULLS LAST,order_id) AS first_rank
 FROM history
), first_orders AS (
 SELECT customer_id,COUNT(*) AS history_orders,
 COUNT_IF(placed_at_utc IS NULL) AS history_orders_missing_placement,
 MAX(source_variants) AS history_source_variants,
 SUM(source_rows-1) AS history_identical_duplicate_rows,
 MIN(earliest_placed_at_utc) AS first_placed_at_utc,
 COUNT_IF(placed_at_utc=earliest_placed_at_utc) AS earliest_timestamp_ties,
 MAX(IFF(first_rank=1,order_id,NULL)) AS first_order_id,
 COUNT_IF(first_placed_flag=TRUE) AS flagged_first_orders,
 MIN(IFF(first_placed_flag=TRUE,order_id,NULL)) AS flagged_first_order_id
 FROM ranked GROUP BY customer_id
)
SELECT a.*,e.customer_id AS ecom_aged_customer_id,e.placed_at_utc AS ecom_aged_placed_at_utc,
```

```
 e.first_placed_flag AS aged_order_first_placed_flag,e.source_variants AS aged_order_source_variants,
 f.history_orders,f.history_orders_missing_placement,f.history_source_variants,f.history_identical_duplicate_rows,
 v.vet_orders_missing_ecom_header,f.first_order_id,f.first_placed_at_utc,f.earliest_timestamp_ties,f.
  flagged_first_orders,f.flagged_first_order_id
FROM aged_lines a LEFT JOIN history e ON a.order_id=e.order_id
LEFT JOIN first_orders f ON a.customer_id=f.customer_id
LEFT JOIN vet_history_audit v ON a.customer_id=v.customer_id
```

DRAFT

RxP follow-up: all aged customers

Tables 2, 3 and 4, left column groups. Source: `data/opportunity/aged-reorder-studies/2026-05-01_2026-08-01_7d-14d-21d-28d-35d-42d-49d-56d-63d-70d_as-of-2026-10-28/query.sql`.

```sql
-- Date-driven CMPU vet-diet CUSTOMER_UPLOAD aged cancellations; UTC follow-up.
-- Study-specific matching uses CUSTOMER_ID, PRODUCT_PART_NUMBER, PET_ID and vet-diet flags.
WITH registry AS (
 SELECT rx_id,MIN(customer_id) AS customer_id,
 MIN(is_vet_diet_flag) AS is_vet_diet_flag,MIN(approval_channel) AS approval_channel,
 MIN(prescription_source) AS prescription_source,
 GREATEST(COUNT(DISTINCT customer_id)+IFF(COUNT_IF(customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT is_vet_diet_flag)+IFF(COUNT_IF(is_vet_diet_flag IS NULL)>0,1,0),
 COUNT(DISTINCT approval_channel)+IFF(COUNT_IF(approval_channel IS NULL)>0,1,0),
 COUNT(DISTINCT prescription_source)+IFF(COUNT_IF(prescription_source IS NULL)>0,1,0)) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS GROUP BY rx_id
), aged_keys AS (
 SELECT l.order_id,l.order_line_id,MAX(p.registry_variants) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN registry p
 ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE
 AND p.approval_channel='CUSTOMER_UPLOAD' AND p.prescription_source='CMPU'
 AND LOWER(TRIM(l.order_cancel_reason))='aged order'
 AND l.order_cancel_dttm>='2026-05-01'::TIMESTAMP_NTZ
 AND l.order_cancel_dttm<'2026-08-01'::TIMESTAMP_NTZ
 GROUP BY l.order_id,l.order_line_id
), originals AS (
 SELECT l.order_id AS original_order_id,l.order_line_id AS original_line_id,
 MAX(k.registry_variants) AS registry_variants,
 MIN(l.customer_id) AS original_customer_id,MIN(l.pet_id) AS original_pet_id,
 MIN(l.product_part_number) AS original_product_part_number,
 MIN(l.order_cancel_dttm) AS original_cancel_utc,
 MIN(LOWER(TRIM(l.order_cancel_reason))) AS original_reason,
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0) AS customer_variants,
 COUNT(DISTINCT l.pet_id)+IFF(COUNT_IF(l.pet_id IS NULL)>0,1,0) AS pet_variants,
 COUNT(DISTINCT l.product_part_number)+IFF(COUNT_IF(l.product_part_number IS NULL)>0,1,0) AS part_variants,
 COUNT(DISTINCT l.order_cancel_dttm)+IFF(COUNT_IF(l.order_cancel_dttm IS NULL)>0,1,0) AS cancel_variants,
 COUNT(DISTINCT LOWER(TRIM(l.order_cancel_reason)))+IFF(COUNT_IF(l.order_cancel_reason IS NULL)>0,1,0) AS
  reason_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 GROUP BY l.order_id,l.order_line_id
), candidate_keys AS (
 SELECT DISTINCT l.order_id,l.order_line_id
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN originals a
 ON l.customer_id=a.original_customer_id

 AND (l.is_vet_diet_flag=TRUE OR (l.pet_id=a.original_pet_id
      AND l.product_part_number=a.original_product_part_number))

 AND l.order_id<>a.original_order_id
 AND l.order_placed_dttm>a.original_cancel_utc
 AND l.order_placed_dttm<=DATEADD(DAY,70,a.original_cancel_utc)
 AND l.order_placed_dttm<='2026-10-28'::TIMESTAMP_NTZ
), order_cancellations AS (
 -- Inspect every line of candidate orders, including other products.
 SELECT l.order_id,MAX(IFF(l.order_order_line_status='X'
 OR NULLIF(TRIM(l.order_cancel_reason),'') IS NOT NULL
 OR l.order_cancel_dttm IS NOT NULL,1,0)) AS order_has_cancellation,
 MAX(IFF(l.order_order_line_status IS NULL,1,0)) AS order_has_unknown_status
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN (SELECT DISTINCT order_id FROM candidate_keys) k ON l.order_id=k.order_id
 GROUP BY l.order_id
), financial AS (
 SELECT l.order_id,l.order_line_id,l.shipment_key,
 MIN(l.customer_id) AS customer_id,MIN(l.pet_id) AS pet_id,
 MIN(l.product_part_number) AS product_part_number,
 MIN(l.is_vet_diet_flag) AS is_vet_diet_flag,
 MIN(l.order_placed_dttm) AS placed_at_utc,
 MIN(l.order_line_shipped_dttm) AS shipped_at_utc,
 MIN(l.order_line_net_sales) AS net_sales,
 COUNT(DISTINCT l.is_vet_diet_flag)+IFF(COUNT_IF(l.is_vet_diet_flag IS NULL)>0,1,0) AS vet_diet_variants,
```

```
 COUNT(DISTINCT l.order_line_net_sales)+IFF(COUNT_IF(l.order_line_net_sales IS NULL)>0,1,0) AS sales_variants,
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0) AS customer_variants,
 COUNT(DISTINCT l.pet_id)+IFF(COUNT_IF(l.pet_id IS NULL)>0,1,0) AS pet_variants,
 COUNT(DISTINCT l.product_part_number)+IFF(COUNT_IF(l.product_part_number IS NULL)>0,1,0) AS part_variants,
 COUNT(DISTINCT l.order_placed_dttm)+IFF(COUNT_IF(l.order_placed_dttm IS NULL)>0,1,0) AS placement_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN candidate_keys k
 ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 GROUP BY l.order_id,l.order_line_id,l.shipment_key
), reorders AS (
 SELECT f.order_id AS reorder_order_id,f.order_line_id AS reorder_line_id,
 MIN(f.customer_id) AS reorder_customer_id,MIN(f.pet_id) AS reorder_pet_id,
 MIN(f.product_part_number) AS reorder_product_part_number,
 MIN(f.is_vet_diet_flag) AS reorder_is_vet_diet,
 MIN(f.placed_at_utc) AS reorder_placed_utc,MIN(f.shipped_at_utc) AS reorder_shipped_utc,
 SUM(f.net_sales)::VARCHAR AS reorder_known_net_sales,
 COUNT_IF(f.net_sales IS NULL) AS missing_sales_records,
 GREATEST(MAX(f.vet_diet_variants),COUNT(DISTINCT f.is_vet_diet_flag)+IFF(COUNT_IF(f.is_vet_diet_flag IS NULL)>0,1,0),
  MAX(f.sales_variants),MAX(f.customer_variants),MAX(f.pet_variants),MAX(f.part_variants),
 MAX(f.placement_variants),COUNT(DISTINCT f.customer_id)+IFF(COUNT_IF(f.customer_id IS NULL)>0,1,0),COUNT(DISTINCT f.
  pet_id)+IFF(COUNT_IF(f.pet_id IS NULL)>0,1,0),
 COUNT(DISTINCT f.product_part_number)+IFF(COUNT_IF(f.product_part_number IS NULL)>0,1,0),COUNT(DISTINCT f.
  placed_at_utc)+IFF(COUNT_IF(f.placed_at_utc IS NULL)>0,1,0)) AS source_variants,
 MAX(o.order_has_cancellation) AS reorder_order_has_cancellation,
 MAX(o.order_has_unknown_status) AS reorder_order_has_unknown_status
 FROM financial f JOIN order_cancellations o ON f.order_id=o.order_id GROUP BY f.order_id,f.order_line_id
)
SELECT a.* REPLACE(TO_CHAR(original_cancel_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS original_cancel_utc),
 r.* REPLACE(TO_CHAR(reorder_placed_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_placed_utc,
 TO_CHAR(reorder_shipped_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_shipped_utc)
FROM originals a LEFT JOIN reorders r ON r.reorder_customer_id=a.original_customer_id

 AND (r.reorder_is_vet_diet=TRUE OR (r.reorder_pet_id=a.original_pet_id
      AND r.reorder_product_part_number=a.original_product_part_number))

 AND r.reorder_order_id<>a.original_order_id AND r.reorder_placed_utc>a.original_cancel_utc
 AND r.reorder_placed_utc<=DATEADD(DAY,70,a.original_cancel_utc)
```

DRAFT

RxP follow-up: first customer-upload cohort

The same three RxP endpoint tables, middle column groups; only each customer's first qualifying aged order is an anchor. Source: `data/opportunity/new-upload-customer-follow-up-studies/2026-05-01_2026-08-01_7d-70d_as-of-2026-10-28/query.sql`.

```sql
-- Same-pet/product follow-up after cancellation of a first CUSTOMER_UPLOAD vet-diet aged order.
-- CMPU vet-diet CUSTOMER_UPLOAD aged cancellations; first observed CUSTOMER_UPLOAD vet-diet order IDs.
-- Vet-diet CUSTOMER_UPLOAD history from RxP prescriptions; all sources/statuses/dates; ECOM chronology.
WITH registry AS (
 SELECT rx_id,MIN(customer_id) AS customer_id,
 MIN(is_vet_diet_flag) AS is_vet_diet_flag,MIN(approval_channel) AS approval_channel,
 MIN(prescription_source) AS prescription_source,
 GREATEST(COUNT(DISTINCT customer_id)+IFF(COUNT_IF(customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT is_vet_diet_flag)+IFF(COUNT_IF(is_vet_diet_flag IS NULL)>0,1,0),
 COUNT(DISTINCT approval_channel)+IFF(COUNT_IF(approval_channel IS NULL)>0,1,0),
 COUNT(DISTINCT prescription_source)+IFF(COUNT_IF(prescription_source IS NULL)>0,1,0)) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS GROUP BY rx_id
), aged_keys AS (
 SELECT l.order_id,l.order_line_id,MAX(p.registry_variants) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN registry p
 ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE
 AND p.approval_channel='CUSTOMER_UPLOAD' AND p.prescription_source='CMPU'
 AND LOWER(TRIM(l.order_cancel_reason))='aged order'
 AND l.order_cancel_dttm>='2026-05-01'::TIMESTAMP_NTZ
 AND l.order_cancel_dttm<'2026-08-01'::TIMESTAMP_NTZ
 GROUP BY l.order_id,l.order_line_id
), aged_lines AS (
 SELECT l.order_id,l.order_line_id,l.customer_id,l.order_placed_dttm AS placed_at_utc,
 l.order_cancel_dttm AS cancelled_at_utc,k.registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
), cohort_customers AS (
 SELECT DISTINCT customer_id FROM aged_lines
), vet_diet_order_keys AS (
 -- CUSTOMER_UPLOAD vet-diet history from every prescription source and date.
 SELECT l.customer_id,l.order_id,MAX(p.registry_variants) AS history_prescription_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN cohort_customers c ON l.customer_id=c.customer_id
 JOIN registry p ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE AND p.approval_channel='CUSTOMER_UPLOAD'
 GROUP BY l.customer_id,l.order_id
), history_keys AS (
 SELECT DISTINCT order_id FROM vet_diet_order_keys
), history AS (
 SELECT o.order_id,MIN(o.customer_id) AS customer_id,
 MIN(o.order_placed_dttm) AS placed_at_utc,MIN(o.order_first_order_placed_flag) AS first_placed_flag,
 COUNT(*) AS source_rows,
 GREATEST(COUNT(DISTINCT o.customer_id)+IFF(COUNT_IF(o.customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_placed_dttm)+IFF(COUNT_IF(o.order_placed_dttm IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_first_order_placed_flag)+IFF(COUNT_IF(o.order_first_order_placed_flag IS NULL)>0,1,0)) AS
  source_variants
 FROM EDLDB.ECOM.ORDERS o JOIN history_keys k ON o.order_id=k.order_id GROUP BY o.order_id
), vet_history_audit AS (
 SELECT v.customer_id,COUNT(DISTINCT IFF(h.order_id IS NULL,v.order_id,NULL)) AS vet_orders_missing_ecom_header,MAX(v.
  history_prescription_variants) AS history_prescription_variants
 FROM vet_diet_order_keys v LEFT JOIN history h ON v.order_id=h.order_id AND v.customer_id=h.customer_id
 GROUP BY v.customer_id
), ranked AS (
 SELECT *,MIN(placed_at_utc) OVER (PARTITION BY customer_id) AS earliest_placed_at_utc,
 ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY placed_at_utc NULLS LAST,order_id) AS first_rank
 FROM history
), first_orders AS (
 SELECT customer_id,COUNT(*) AS history_orders,
 COUNT_IF(placed_at_utc IS NULL) AS history_orders_missing_placement,
 MAX(source_variants) AS history_source_variants,
 SUM(source_rows-1) AS history_identical_duplicate_rows,
 MIN(earliest_placed_at_utc) AS first_placed_at_utc,
 COUNT_IF(placed_at_utc=earliest_placed_at_utc) AS earliest_timestamp_ties,
 MAX(IFF(first_rank=1,order_id,NULL)) AS first_order_id,
 COUNT_IF(first_placed_flag=TRUE) AS flagged_first_orders,
```

```
 MIN(IFF(first_placed_flag=TRUE,order_id,NULL)) AS flagged_first_order_id
 FROM ranked GROUP BY customer_id
)
, selected_aged_keys AS (
 -- Only the first CUSTOMER_UPLOAD vet-diet aged order per customer is an anchor.
 SELECT DISTINCT a.order_id,a.order_line_id,
 GREATEST(a.registry_variants,f.history_source_variants,v.history_prescription_variants) AS registry_variants
 FROM aged_lines a JOIN first_orders f ON a.customer_id=f.customer_id AND a.order_id=f.first_order_id
 JOIN vet_history_audit v ON a.customer_id=v.customer_id
 WHERE f.first_placed_at_utc IS NOT NULL AND f.earliest_timestamp_ties=1
 AND f.history_orders_missing_placement=0 AND v.vet_orders_missing_ecom_header=0
), originals AS (
 SELECT l.order_id AS original_order_id,l.order_line_id AS original_line_id,
 MAX(k.registry_variants) AS registry_variants,
 MIN(l.customer_id) AS original_customer_id,MIN(l.pet_id) AS original_pet_id,
 MIN(l.product_part_number) AS original_product_part_number,
 MIN(l.order_cancel_dttm) AS original_cancel_utc,
 MIN(LOWER(TRIM(l.order_cancel_reason))) AS original_reason,
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0) AS customer_variants,
 COUNT(DISTINCT l.pet_id)+IFF(COUNT_IF(l.pet_id IS NULL)>0,1,0) AS pet_variants,
 COUNT(DISTINCT l.product_part_number)+IFF(COUNT_IF(l.product_part_number IS NULL)>0,1,0) AS part_variants,
 COUNT(DISTINCT l.order_cancel_dttm)+IFF(COUNT_IF(l.order_cancel_dttm IS NULL)>0,1,0) AS cancel_variants,
 COUNT(DISTINCT LOWER(TRIM(l.order_cancel_reason)))+IFF(COUNT_IF(l.order_cancel_reason IS NULL)>0,1,0) AS
  reason_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN selected_aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 GROUP BY l.order_id,l.order_line_id
), candidate_keys AS (
 SELECT DISTINCT l.order_id,l.order_line_id
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN originals a
 ON l.customer_id=a.original_customer_id

 AND (l.is_vet_diet_flag=TRUE OR (l.pet_id=a.original_pet_id
      AND l.product_part_number=a.original_product_part_number))

 AND l.order_id<>a.original_order_id
 AND l.order_placed_dttm>a.original_cancel_utc
 AND l.order_placed_dttm<=DATEADD(DAY,70,a.original_cancel_utc)
 AND l.order_placed_dttm<='2026-10-28'::TIMESTAMP_NTZ
), order_cancellations AS (
 -- Inspect every line of candidate orders, including other products.
 SELECT l.order_id,MAX(IFF(l.order_order_line_status='X'
 OR NULLIF(TRIM(l.order_cancel_reason),'') IS NOT NULL
 OR l.order_cancel_dttm IS NOT NULL,1,0)) AS order_has_cancellation,
 MAX(IFF(l.order_order_line_status IS NULL,1,0)) AS order_has_unknown_status
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN (SELECT DISTINCT order_id FROM candidate_keys) k ON l.order_id=k.order_id
 GROUP BY l.order_id
), financial AS (
 SELECT l.order_id,l.order_line_id,l.shipment_key,
 MIN(l.customer_id) AS customer_id,MIN(l.pet_id) AS pet_id,
 MIN(l.product_part_number) AS product_part_number,
 MIN(l.is_vet_diet_flag) AS is_vet_diet_flag,
 MIN(l.order_placed_dttm) AS placed_at_utc,
 MIN(l.order_line_shipped_dttm) AS shipped_at_utc,
 MIN(l.order_line_net_sales) AS net_sales,
 COUNT(DISTINCT l.is_vet_diet_flag)+IFF(COUNT_IF(l.is_vet_diet_flag IS NULL)>0,1,0) AS vet_diet_variants,
 COUNT(DISTINCT l.order_line_net_sales)+IFF(COUNT_IF(l.order_line_net_sales IS NULL)>0,1,0) AS sales_variants,
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0) AS customer_variants,
 COUNT(DISTINCT l.pet_id)+IFF(COUNT_IF(l.pet_id IS NULL)>0,1,0) AS pet_variants,
 COUNT(DISTINCT l.product_part_number)+IFF(COUNT_IF(l.product_part_number IS NULL)>0,1,0) AS part_variants,
 COUNT(DISTINCT l.order_placed_dttm)+IFF(COUNT_IF(l.order_placed_dttm IS NULL)>0,1,0) AS placement_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN candidate_keys k
 ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 GROUP BY l.order_id,l.order_line_id,l.shipment_key
), reorders AS (
 SELECT f.order_id AS reorder_order_id,f.order_line_id AS reorder_line_id,
 MIN(f.customer_id) AS reorder_customer_id,MIN(f.pet_id) AS reorder_pet_id,
 MIN(f.product_part_number) AS reorder_product_part_number,
 MIN(f.is_vet_diet_flag) AS reorder_is_vet_diet,
 MIN(f.placed_at_utc) AS reorder_placed_utc,MIN(f.shipped_at_utc) AS reorder_shipped_utc,
```

```
  SUM(f.net_sales)::VARCHAR AS reorder_known_net_sales,
  COUNT_IF(f.net_sales IS NULL) AS missing_sales_records,
  GREATEST(MAX(f.vet_diet_variants),COUNT(DISTINCT f.is_vet_diet_flag)+IFF(COUNT_IF(f.is_vet_diet_flag IS NULL)>0,1,0),
   MAX(f.sales_variants),MAX(f.customer_variants),MAX(f.pet_variants),MAX(f.part_variants),
  MAX(f.placement_variants),COUNT(DISTINCT f.customer_id)+IFF(COUNT_IF(f.customer_id IS NULL)>0,1,0),COUNT(DISTINCT f.
   pet_id)+IFF(COUNT_IF(f.pet_id IS NULL)>0,1,0),
  COUNT(DISTINCT f.product_part_number)+IFF(COUNT_IF(f.product_part_number IS NULL)>0,1,0),COUNT(DISTINCT f.
   placed_at_utc)+IFF(COUNT_IF(f.placed_at_utc IS NULL)>0,1,0)) AS source_variants,
  MAX(o.order_has_cancellation) AS reorder_order_has_cancellation,
  MAX(o.order_has_unknown_status) AS reorder_order_has_unknown_status
  FROM financial f JOIN order_cancellations o ON f.order_id=o.order_id GROUP BY f.order_id,f.order_line_id
)
SELECT a.* REPLACE(TO_CHAR(original_cancel_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS original_cancel_utc),
 r.* REPLACE(TO_CHAR(reorder_placed_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_placed_utc,
 TO_CHAR(reorder_shipped_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_shipped_utc)
FROM originals a LEFT JOIN reorders r ON r.reorder_customer_id=a.original_customer_id

 AND (r.reorder_is_vet_diet=TRUE OR (r.reorder_pet_id=a.original_pet_id
      AND r.reorder_product_part_number=a.original_product_part_number))

 AND r.reorder_order_id<>a.original_order_id AND r.reorder_placed_utc>a.original_cancel_utc
 AND r.reorder_placed_utc<=DATEADD(DAY,70,a.original_cancel_utc)
```

DRAFT

RxP follow-up: first vet-diet cohort

The same three RxP endpoint tables, right column groups; only each customer's first qualifying aged order is an anchor. Source: `data/opportunity/new-vet-diet-customer-follow-up-studies/2026-05-01_2026-08-01_7d-70d_as-of-2026-10-28/query.sql`.

```sql
-- Follow-up for customers whose aged order was their first observed vet-diet order, any historical approval channel/
  source.
-- CMPU vet-diet CUSTOMER_UPLOAD aged cancellations; exact first observed vet-diet order IDs.
-- Vet-diet membership from RxP lines; chronology from ECOM.ORDERS, all channels/statuses/dates.
WITH registry AS (
 SELECT rx_id,MIN(customer_id) AS customer_id,
 MIN(is_vet_diet_flag) AS is_vet_diet_flag,MIN(approval_channel) AS approval_channel,
 MIN(prescription_source) AS prescription_source,
 GREATEST(COUNT(DISTINCT customer_id)+IFF(COUNT_IF(customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT is_vet_diet_flag)+IFF(COUNT_IF(is_vet_diet_flag IS NULL)>0,1,0),
 COUNT(DISTINCT approval_channel)+IFF(COUNT_IF(approval_channel IS NULL)>0,1,0),
 COUNT(DISTINCT prescription_source)+IFF(COUNT_IF(prescription_source IS NULL)>0,1,0)) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS GROUP BY rx_id
), aged_keys AS (
 SELECT l.order_id,l.order_line_id,MAX(p.registry_variants) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN registry p
 ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE
 AND p.approval_channel='CUSTOMER_UPLOAD' AND p.prescription_source='CMPU'
 AND LOWER(TRIM(l.order_cancel_reason))='aged order'
 AND l.order_cancel_dttm>='2026-05-01'::TIMESTAMP_NTZ
 AND l.order_cancel_dttm<'2026-08-01'::TIMESTAMP_NTZ
 GROUP BY l.order_id,l.order_line_id
), aged_lines AS (
 SELECT l.order_id,l.order_line_id,l.customer_id,l.order_placed_dttm AS placed_at_utc,
 l.order_cancel_dttm AS cancelled_at_utc,k.registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
), cohort_customers AS (
 SELECT DISTINCT customer_id FROM aged_lines
), vet_diet_order_keys AS (
 -- Full vet-diet history; no CMPU, channel, RX-ID or date restriction here.
 SELECT DISTINCT l.customer_id,l.order_id
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN cohort_customers c ON l.customer_id=c.customer_id
 WHERE l.is_vet_diet_flag=TRUE
), history_keys AS (
 SELECT DISTINCT order_id FROM vet_diet_order_keys
), history AS (
 SELECT o.order_id,MIN(o.customer_id) AS customer_id,
 MIN(o.order_placed_dttm) AS placed_at_utc,MIN(o.order_first_order_placed_flag) AS first_placed_flag,
 COUNT(*) AS source_rows,
 GREATEST(COUNT(DISTINCT o.customer_id)+IFF(COUNT_IF(o.customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_placed_dttm)+IFF(COUNT_IF(o.order_placed_dttm IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_first_order_placed_flag)+IFF(COUNT_IF(o.order_first_order_placed_flag IS NULL)>0,1,0)) AS
  source_variants
 FROM EDLDB.ECOM.ORDERS o JOIN history_keys k ON o.order_id=k.order_id GROUP BY o.order_id
), vet_history_audit AS (
 SELECT v.customer_id,COUNT(DISTINCT IFF(h.order_id IS NULL,v.order_id,NULL)) AS vet_orders_missing_ecom_header
 FROM vet_diet_order_keys v LEFT JOIN history h ON v.order_id=h.order_id AND v.customer_id=h.customer_id
 GROUP BY v.customer_id
), ranked AS (
 SELECT *,MIN(placed_at_utc) OVER (PARTITION BY customer_id) AS earliest_placed_at_utc,
 ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY placed_at_utc NULLS LAST,order_id) AS first_rank
 FROM history
), first_orders AS (
 SELECT customer_id,COUNT(*) AS history_orders,
 COUNT_IF(placed_at_utc IS NULL) AS history_orders_missing_placement,
 MAX(source_variants) AS history_source_variants,
 SUM(source_rows-1) AS history_identical_duplicate_rows,
 MIN(earliest_placed_at_utc) AS first_placed_at_utc,
 COUNT_IF(placed_at_utc=earliest_placed_at_utc) AS earliest_timestamp_ties,
 MAX(IFF(first_rank=1,order_id,NULL)) AS first_order_id,
 COUNT_IF(first_placed_flag=TRUE) AS flagged_first_orders,
 MIN(IFF(first_placed_flag=TRUE,order_id,NULL)) AS flagged_first_order_id
 FROM ranked GROUP BY customer_id
```

```
)
, selected_aged_keys AS (
 -- Only the first vet-diet aged order per customer is an anchor.
 SELECT DISTINCT a.order_id,a.order_line_id,
 GREATEST(a.registry_variants,f.history_source_variants) AS registry_variants
 FROM aged_lines a JOIN first_orders f ON a.customer_id=f.customer_id AND a.order_id=f.first_order_id
 JOIN vet_history_audit v ON a.customer_id=v.customer_id
 WHERE f.first_placed_at_utc IS NOT NULL AND f.earliest_timestamp_ties=1
 AND f.history_orders_missing_placement=0 AND v.vet_orders_missing_ecom_header=0
), originals AS (
 SELECT l.order_id AS original_order_id,l.order_line_id AS original_line_id,
 MAX(k.registry_variants) AS registry_variants,
 MIN(l.customer_id) AS original_customer_id,MIN(l.pet_id) AS original_pet_id,
 MIN(l.product_part_number) AS original_product_part_number,
 MIN(l.order_cancel_dttm) AS original_cancel_utc,
 MIN(LOWER(TRIM(l.order_cancel_reason))) AS original_reason,
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0) AS customer_variants,
 COUNT(DISTINCT l.pet_id)+IFF(COUNT_IF(l.pet_id IS NULL)>0,1,0) AS pet_variants,
 COUNT(DISTINCT l.product_part_number)+IFF(COUNT_IF(l.product_part_number IS NULL)>0,1,0) AS part_variants,
 COUNT(DISTINCT l.order_cancel_dttm)+IFF(COUNT_IF(l.order_cancel_dttm IS NULL)>0,1,0) AS cancel_variants,
 COUNT(DISTINCT LOWER(TRIM(l.order_cancel_reason)))+IFF(COUNT_IF(l.order_cancel_reason IS NULL)>0,1,0) AS
  reason_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN selected_aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 GROUP BY l.order_id,l.order_line_id
), candidate_keys AS (
 SELECT DISTINCT l.order_id,l.order_line_id
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN originals a
 ON l.customer_id=a.original_customer_id

 AND (l.is_vet_diet_flag=TRUE OR (l.pet_id=a.original_pet_id
      AND l.product_part_number=a.original_product_part_number))

 AND l.order_id<>a.original_order_id
 AND l.order_placed_dttm>a.original_cancel_utc
 AND l.order_placed_dttm<=DATEADD(DAY,70,a.original_cancel_utc)
 AND l.order_placed_dttm<='2026-10-28'::TIMESTAMP_NTZ
), order_cancellations AS (
 -- Inspect every line of candidate orders, including other products.
 SELECT l.order_id,MAX(IFF(l.order_order_line_status='X'
 OR NULLIF(TRIM(l.order_cancel_reason),'') IS NOT NULL
 OR l.order_cancel_dttm IS NOT NULL,1,0)) AS order_has_cancellation,
 MAX(IFF(l.order_order_line_status IS NULL,1,0)) AS order_has_unknown_status
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN (SELECT DISTINCT order_id FROM candidate_keys) k ON l.order_id=k.order_id
 GROUP BY l.order_id
), financial AS (
 SELECT l.order_id,l.order_line_id,l.shipment_key,
 MIN(l.customer_id) AS customer_id,MIN(l.pet_id) AS pet_id,
 MIN(l.product_part_number) AS product_part_number,
 MIN(l.is_vet_diet_flag) AS is_vet_diet_flag,
 MIN(l.order_placed_dttm) AS placed_at_utc,
 MIN(l.order_line_shipped_dttm) AS shipped_at_utc,
 MIN(l.order_line_net_sales) AS net_sales,
 COUNT(DISTINCT l.is_vet_diet_flag)+IFF(COUNT_IF(l.is_vet_diet_flag IS NULL)>0,1,0) AS vet_diet_variants,
 COUNT(DISTINCT l.order_line_net_sales)+IFF(COUNT_IF(l.order_line_net_sales IS NULL)>0,1,0) AS sales_variants,
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0) AS customer_variants,
 COUNT(DISTINCT l.pet_id)+IFF(COUNT_IF(l.pet_id IS NULL)>0,1,0) AS pet_variants,
 COUNT(DISTINCT l.product_part_number)+IFF(COUNT_IF(l.product_part_number IS NULL)>0,1,0) AS part_variants,
 COUNT(DISTINCT l.order_placed_dttm)+IFF(COUNT_IF(l.order_placed_dttm IS NULL)>0,1,0) AS placement_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN candidate_keys k
 ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 GROUP BY l.order_id,l.order_line_id,l.shipment_key
), reorders AS (
 SELECT f.order_id AS reorder_order_id,f.order_line_id AS reorder_line_id,
 MIN(f.customer_id) AS reorder_customer_id,MIN(f.pet_id) AS reorder_pet_id,
 MIN(f.product_part_number) AS reorder_product_part_number,
 MIN(f.is_vet_diet_flag) AS reorder_is_vet_diet,
 MIN(f.placed_at_utc) AS reorder_placed_utc,MIN(f.shipped_at_utc) AS reorder_shipped_utc,
 SUM(f.net_sales)::VARCHAR AS reorder_known_net_sales,
 COUNT_IF(f.net_sales IS NULL) AS missing_sales_records,
```

```
 GREATEST(MAX(f.vet_diet_variants),COUNT(DISTINCT f.is_vet_diet_flag)+IFF(COUNT_IF(f.is_vet_diet_flag IS NULL)>0,1,0),
  MAX(f.sales_variants),MAX(f.customer_variants),MAX(f.pet_variants),MAX(f.part_variants),
 MAX(f.placement_variants),COUNT(DISTINCT f.customer_id)+IFF(COUNT_IF(f.customer_id IS NULL)>0,1,0),COUNT(DISTINCT f.
  pet_id)+IFF(COUNT_IF(f.pet_id IS NULL)>0,1,0),
 COUNT(DISTINCT f.product_part_number)+IFF(COUNT_IF(f.product_part_number IS NULL)>0,1,0),COUNT(DISTINCT f.
  placed_at_utc)+IFF(COUNT_IF(f.placed_at_utc IS NULL)>0,1,0)) AS source_variants,
 MAX(o.order_has_cancellation) AS reorder_order_has_cancellation,
 MAX(o.order_has_unknown_status) AS reorder_order_has_unknown_status
 FROM financial f JOIN order_cancellations o ON f.order_id=o.order_id GROUP BY f.order_id,f.order_line_id
)
SELECT a.* REPLACE(TO_CHAR(original_cancel_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS original_cancel_utc),
 r.* REPLACE(TO_CHAR(reorder_placed_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_placed_utc,
 TO_CHAR(reorder_shipped_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_shipped_utc)
FROM originals a LEFT JOIN reorders r ON r.reorder_customer_id=a.original_customer_id

 AND (r.reorder_is_vet_diet=TRUE OR (r.reorder_pet_id=a.original_pet_id
      AND r.reorder_product_part_number=a.original_product_part_number))

 AND r.reorder_order_id<>a.original_order_id AND r.reorder_placed_utc>a.original_cancel_utc
 AND r.reorder_placed_utc<=DATEADD(DAY,70,a.original_cancel_utc)
```

DRAFT

All-Chewy follow-up: all aged customers

Table 5, left column group. Customer-only matching against `EDLDB.ECOM.ORDER_LINE` across all categories. Source: `data/opportunity/aged-chewy-follow-up/2026-05-01_2026-08-01_7d-14d-21d-28d-35d-42d-49d-56d-63d-70d_as-of-2026-10-28/query.sql`.

```sql
-- Date-driven CMPU vet-diet CUSTOMER_UPLOAD aged cancellations; UTC follow-up.
-- All-Chewy follow-up uses EDLDB.ECOM.ORDER_LINE and NET_SALES; customer matching only.
WITH registry AS (
 SELECT rx_id,MIN(customer_id) AS customer_id,
 MIN(is_vet_diet_flag) AS is_vet_diet_flag,MIN(approval_channel) AS approval_channel,
 MIN(prescription_source) AS prescription_source,
 GREATEST(COUNT(DISTINCT customer_id)+IFF(COUNT_IF(customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT is_vet_diet_flag)+IFF(COUNT_IF(is_vet_diet_flag IS NULL)>0,1,0),
 COUNT(DISTINCT approval_channel)+IFF(COUNT_IF(approval_channel IS NULL)>0,1,0),
 COUNT(DISTINCT prescription_source)+IFF(COUNT_IF(prescription_source IS NULL)>0,1,0)) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS GROUP BY rx_id
), aged_keys AS (
 SELECT l.order_id,l.order_line_id,MAX(p.registry_variants) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN registry p
 ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE
 AND p.approval_channel='CUSTOMER_UPLOAD' AND p.prescription_source='CMPU'
 AND LOWER(TRIM(l.order_cancel_reason))='aged order'
 AND l.order_cancel_dttm>='2026-05-01'::TIMESTAMP_NTZ
 AND l.order_cancel_dttm<'2026-08-01'::TIMESTAMP_NTZ
 GROUP BY l.order_id,l.order_line_id
), originals AS (
 SELECT l.order_id AS original_order_id,l.order_line_id AS original_line_id,
 MAX(k.registry_variants) AS registry_variants,
 MIN(l.customer_id) AS original_customer_id,MIN(l.pet_id) AS original_pet_id,
 MIN(l.product_part_number) AS original_product_part_number,
 MIN(l.order_cancel_dttm) AS original_cancel_utc,
 MIN(LOWER(TRIM(l.order_cancel_reason))) AS original_reason,
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0) AS customer_variants,
 COUNT(DISTINCT l.pet_id)+IFF(COUNT_IF(l.pet_id IS NULL)>0,1,0) AS pet_variants,
 COUNT(DISTINCT l.product_part_number)+IFF(COUNT_IF(l.product_part_number IS NULL)>0,1,0) AS part_variants,
 COUNT(DISTINCT l.order_cancel_dttm)+IFF(COUNT_IF(l.order_cancel_dttm IS NULL)>0,1,0) AS cancel_variants,
 COUNT(DISTINCT LOWER(TRIM(l.order_cancel_reason)))+IFF(COUNT_IF(l.order_cancel_reason IS NULL)>0,1,0) AS
  reason_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 GROUP BY l.order_id,l.order_line_id
), candidate_keys AS (
 SELECT DISTINCT l.order_id,l.order_line_id
 FROM EDLDB.ECOM.ORDER_LINE l JOIN originals a ON l.customer_id=a.original_customer_id
 AND l.order_id<>a.original_order_id
 AND l.order_placed_dttm>a.original_cancel_utc
 AND l.order_placed_dttm<=DATEADD(DAY,70,a.original_cancel_utc)
 AND l.order_placed_dttm<='2026-10-28'::TIMESTAMP_NTZ
), candidate_lines AS (
 -- All lines on each candidate order are needed for whole-order exclusion.
 SELECT l.* FROM EDLDB.ECOM.ORDER_LINE l
 JOIN (SELECT DISTINCT order_id FROM candidate_keys) k ON l.order_id=k.order_id
), order_cancellations AS (
 SELECT order_id,MAX(IFF(order_order_line_status='X' OR order_status='X'
 OR NULLIF(TRIM(order_cancel_reason),'') IS NOT NULL OR order_cancel_dttm IS NOT NULL,1,0)) AS order_has_cancellation,
 MAX(IFF(order_order_line_status IS NULL,1,0)) AS order_has_unknown_status
 FROM candidate_lines GROUP BY order_id
), reorders AS (
 -- ECOM.ORDER_LINE is a physical-line view; NET_SALES is the full line amount.
 -- Exact source duplicates collapse. Conflicting line values fail validation.
 SELECT l.order_id AS reorder_order_id,l.order_line_id AS reorder_line_id,
 MIN(l.customer_id) AS reorder_customer_id,
 NULL::NUMBER(38,0) AS reorder_pet_id,NULL::VARCHAR AS reorder_product_part_number,
 MIN(l.order_placed_dttm) AS reorder_placed_utc,
 MIN(l.order_line_shipped_dttm) AS reorder_shipped_utc,
 MIN(l.net_sales)::VARCHAR AS reorder_known_net_sales,
 IFF(COUNT_IF(l.net_sales IS NULL)>0,1,0) AS missing_sales_records,
 COUNT(*) AS ecom_source_rows,
 MIN(l.order_line_currency) AS ecom_currency,
 GREATEST(
```

```
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT l.order_placed_dttm)+IFF(COUNT_IF(l.order_placed_dttm IS NULL)>0,1,0),
 COUNT(DISTINCT l.net_sales)+IFF(COUNT_IF(l.net_sales IS NULL)>0,1,0),
 COUNT(DISTINCT l.order_line_currency)+IFF(COUNT_IF(l.order_line_currency IS NULL)>0,1,0)) AS source_variants,
 MAX(o.order_has_cancellation) AS reorder_order_has_cancellation,
 MAX(o.order_has_unknown_status) AS reorder_order_has_unknown_status
 FROM candidate_lines l JOIN candidate_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 JOIN order_cancellations o ON l.order_id=o.order_id
 GROUP BY l.order_id,l.order_line_id
)
SELECT a.* REPLACE(TO_CHAR(original_cancel_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS original_cancel_utc),
 r.* REPLACE(TO_CHAR(reorder_placed_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_placed_utc,
 TO_CHAR(reorder_shipped_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_shipped_utc),
 (SELECT COUNT_IF(order_line_id IS NULL) FROM candidate_keys) AS candidate_missing_line_keys
FROM originals a LEFT JOIN reorders r ON r.reorder_customer_id=a.original_customer_id
 AND r.reorder_order_id<>a.original_order_id AND r.reorder_placed_utc>a.original_cancel_utc
 AND r.reorder_placed_utc<=DATEADD(DAY,70,a.original_cancel_utc)
```

DRAFT

All-Chewy follow-up: first customer-upload cohort

Table 5, middle column group. Source: `data/opportunity/new-upload-customer-follow-up-chewy/2026-05-01_2026-08-01_7d-70d_as-of-2026-10-28/query.sql`.

```sql
-- All-Chewy purchases for frozen first CUSTOMER_UPLOAD vet-diet aged customers.
-- Same-pet/product follow-up after cancellation of a first CUSTOMER_UPLOAD vet-diet aged order.
-- CMPU vet-diet CUSTOMER_UPLOAD aged cancellations; first observed CUSTOMER_UPLOAD vet-diet order IDs.
-- Vet-diet CUSTOMER_UPLOAD history from RxP prescriptions; all sources/statuses/dates; ECOM chronology.
WITH registry AS (
 SELECT rx_id,MIN(customer_id) AS customer_id,
 MIN(is_vet_diet_flag) AS is_vet_diet_flag,MIN(approval_channel) AS approval_channel,
 MIN(prescription_source) AS prescription_source,
 GREATEST(COUNT(DISTINCT customer_id)+IFF(COUNT_IF(customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT is_vet_diet_flag)+IFF(COUNT_IF(is_vet_diet_flag IS NULL)>0,1,0),
 COUNT(DISTINCT approval_channel)+IFF(COUNT_IF(approval_channel IS NULL)>0,1,0),
 COUNT(DISTINCT prescription_source)+IFF(COUNT_IF(prescription_source IS NULL)>0,1,0)) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS GROUP BY rx_id
), aged_keys AS (
 SELECT l.order_id,l.order_line_id,MAX(p.registry_variants) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN registry p
 ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE
 AND p.approval_channel='CUSTOMER_UPLOAD' AND p.prescription_source='CMPU'
 AND LOWER(TRIM(l.order_cancel_reason))='aged order'
 AND l.order_cancel_dttm>='2026-05-01'::TIMESTAMP_NTZ
 AND l.order_cancel_dttm<'2026-08-01'::TIMESTAMP_NTZ
 GROUP BY l.order_id,l.order_line_id
), aged_lines AS (
 SELECT l.order_id,l.order_line_id,l.customer_id,l.order_placed_dttm AS placed_at_utc,
 l.order_cancel_dttm AS cancelled_at_utc,k.registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
), cohort_customers AS (
 SELECT DISTINCT customer_id FROM aged_lines
), vet_diet_order_keys AS (
 -- CUSTOMER_UPLOAD vet-diet history from every prescription source and date.
 SELECT l.customer_id,l.order_id,MAX(p.registry_variants) AS history_prescription_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN cohort_customers c ON l.customer_id=c.customer_id
 JOIN registry p ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE AND p.approval_channel='CUSTOMER_UPLOAD'
 GROUP BY l.customer_id,l.order_id
), history_keys AS (
 SELECT DISTINCT order_id FROM vet_diet_order_keys
), history AS (
 SELECT o.order_id,MIN(o.customer_id) AS customer_id,
 MIN(o.order_placed_dttm) AS placed_at_utc,MIN(o.order_first_order_placed_flag) AS first_placed_flag,
 COUNT(*) AS source_rows,
 GREATEST(COUNT(DISTINCT o.customer_id)+IFF(COUNT_IF(o.customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_placed_dttm)+IFF(COUNT_IF(o.order_placed_dttm IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_first_order_placed_flag)+IFF(COUNT_IF(o.order_first_order_placed_flag IS NULL)>0,1,0)) AS
  source_variants
 FROM EDLDB.ECOM.ORDERS o JOIN history_keys k ON o.order_id=k.order_id GROUP BY o.order_id
), vet_history_audit AS (
 SELECT v.customer_id,COUNT(DISTINCT IFF(h.order_id IS NULL,v.order_id,NULL)) AS vet_orders_missing_ecom_header,MAX(v.
  history_prescription_variants) AS history_prescription_variants
 FROM vet_diet_order_keys v LEFT JOIN history h ON v.order_id=h.order_id AND v.customer_id=h.customer_id
 GROUP BY v.customer_id
), ranked AS (
 SELECT *,MIN(placed_at_utc) OVER (PARTITION BY customer_id) AS earliest_placed_at_utc,
 ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY placed_at_utc NULLS LAST,order_id) AS first_rank
 FROM history
), first_orders AS (
 SELECT customer_id,COUNT(*) AS history_orders,
 COUNT_IF(placed_at_utc IS NULL) AS history_orders_missing_placement,
 MAX(source_variants) AS history_source_variants,
 SUM(source_rows-1) AS history_identical_duplicate_rows,
 MIN(earliest_placed_at_utc) AS first_placed_at_utc,
 COUNT_IF(placed_at_utc=earliest_placed_at_utc) AS earliest_timestamp_ties,
 MAX(IFF(first_rank=1,order_id,NULL)) AS first_order_id,
 COUNT_IF(first_placed_flag=TRUE) AS flagged_first_orders,
```

```
 MIN(IFF(first_placed_flag=TRUE,order_id,NULL)) AS flagged_first_order_id
 FROM ranked GROUP BY customer_id
)
, selected_aged_keys AS (
 -- Only the first CUSTOMER_UPLOAD vet-diet aged order per customer is an anchor.
 SELECT DISTINCT a.order_id,a.order_line_id,
 GREATEST(a.registry_variants,f.history_source_variants,v.history_prescription_variants) AS registry_variants
 FROM aged_lines a JOIN first_orders f ON a.customer_id=f.customer_id AND a.order_id=f.first_order_id
 JOIN vet_history_audit v ON a.customer_id=v.customer_id
 WHERE f.first_placed_at_utc IS NOT NULL AND f.earliest_timestamp_ties=1
 AND f.history_orders_missing_placement=0 AND v.vet_orders_missing_ecom_header=0
), originals AS (
 SELECT l.order_id AS original_order_id,l.order_line_id AS original_line_id,
 MAX(k.registry_variants) AS registry_variants,
 MIN(l.customer_id) AS original_customer_id,MIN(l.pet_id) AS original_pet_id,
 MIN(l.product_part_number) AS original_product_part_number,
 MIN(l.order_cancel_dttm) AS original_cancel_utc,
 MIN(LOWER(TRIM(l.order_cancel_reason))) AS original_reason,
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0) AS customer_variants,
 COUNT(DISTINCT l.pet_id)+IFF(COUNT_IF(l.pet_id IS NULL)>0,1,0) AS pet_variants,
 COUNT(DISTINCT l.product_part_number)+IFF(COUNT_IF(l.product_part_number IS NULL)>0,1,0) AS part_variants,
 COUNT(DISTINCT l.order_cancel_dttm)+IFF(COUNT_IF(l.order_cancel_dttm IS NULL)>0,1,0) AS cancel_variants,
 COUNT(DISTINCT LOWER(TRIM(l.order_cancel_reason)))+IFF(COUNT_IF(l.order_cancel_reason IS NULL)>0,1,0) AS
  reason_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN selected_aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 GROUP BY l.order_id,l.order_line_id
), candidate_keys AS (
 SELECT DISTINCT l.order_id,l.order_line_id
 FROM EDLDB.ECOM.ORDER_LINE l JOIN originals a ON l.customer_id=a.original_customer_id
 AND l.order_id<>a.original_order_id
 AND l.order_placed_dttm>a.original_cancel_utc
 AND l.order_placed_dttm<=DATEADD(DAY,70,a.original_cancel_utc)
 AND l.order_placed_dttm<='2026-10-28'::TIMESTAMP_NTZ
), candidate_lines AS (
 -- All lines on each candidate order are needed for whole-order exclusion.
 SELECT l.* FROM EDLDB.ECOM.ORDER_LINE l
 JOIN (SELECT DISTINCT order_id FROM candidate_keys) k ON l.order_id=k.order_id
), order_cancellations AS (
 SELECT order_id,MAX(IFF(order_order_line_status='X' OR order_status='X'
 OR NULLIF(TRIM(order_cancel_reason),'') IS NOT NULL OR order_cancel_dttm IS NOT NULL,1,0)) AS order_has_cancellation,
 MAX(IFF(order_order_line_status IS NULL,1,0)) AS order_has_unknown_status
 FROM candidate_lines GROUP BY order_id
), reorders AS (
 -- ECOM.ORDER_LINE is a physical-line view; NET_SALES is the full line amount.
 -- Exact source duplicates collapse. Conflicting line values fail validation.
 SELECT l.order_id AS reorder_order_id,l.order_line_id AS reorder_line_id,
 MIN(l.customer_id) AS reorder_customer_id,
 NULL::NUMBER(38,0) AS reorder_pet_id,NULL::VARCHAR AS reorder_product_part_number,
 MIN(l.order_placed_dttm) AS reorder_placed_utc,
 MIN(l.order_line_shipped_dttm) AS reorder_shipped_utc,
 MIN(l.net_sales)::VARCHAR AS reorder_known_net_sales,
 IFF(COUNT_IF(l.net_sales IS NULL)>0,1,0) AS missing_sales_records,
 COUNT(*) AS ecom_source_rows,
 MIN(l.order_line_currency) AS ecom_currency,
 GREATEST(
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT l.order_placed_dttm)+IFF(COUNT_IF(l.order_placed_dttm IS NULL)>0,1,0),
 COUNT(DISTINCT l.net_sales)+IFF(COUNT_IF(l.net_sales IS NULL)>0,1,0),
 COUNT(DISTINCT l.order_line_currency)+IFF(COUNT_IF(l.order_line_currency IS NULL)>0,1,0)) AS source_variants,
 MAX(o.order_has_cancellation) AS reorder_order_has_cancellation,
 MAX(o.order_has_unknown_status) AS reorder_order_has_unknown_status
 FROM candidate_lines l JOIN candidate_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 JOIN order_cancellations o ON l.order_id=o.order_id
 GROUP BY l.order_id,l.order_line_id
)
SELECT a.* REPLACE(TO_CHAR(original_cancel_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS original_cancel_utc),
 r.* REPLACE(TO_CHAR(reorder_placed_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_placed_utc,
 TO_CHAR(reorder_shipped_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_shipped_utc),
 (SELECT COUNT_IF(order_line_id IS NULL) FROM candidate_keys) AS candidate_missing_line_keys
FROM originals a LEFT JOIN reorders r ON r.reorder_customer_id=a.original_customer_id
```

```
136  AND r.reorder_order_id<>a.original_order_id AND r.reorder_placed_utc>a.original_cancel_utc
137  AND r.reorder_placed_utc<=DATEADD(DAY,70,a.original_cancel_utc)
```

DRAFT

All-Chewy follow-up: first vet-diet cohort

Table 5, right column group. Source: `data/opportunity/new-vet-diet-customer-follow-up-chewy/2026-05-01_2026-08-01_7d-70d_as-of-2026-10-28/query.sql`.

```sql
-- Follow-up for customers whose aged order was their first observed vet-diet order, any historical approval channel/
  source.
-- CMPU vet-diet CUSTOMER_UPLOAD aged cancellations; exact first observed vet-diet order IDs.
-- Vet-diet membership from RxP lines; chronology from ECOM.ORDERS, all channels/statuses/dates.
WITH registry AS (
 SELECT rx_id,MIN(customer_id) AS customer_id,
 MIN(is_vet_diet_flag) AS is_vet_diet_flag,MIN(approval_channel) AS approval_channel,
 MIN(prescription_source) AS prescription_source,
 GREATEST(COUNT(DISTINCT customer_id)+IFF(COUNT_IF(customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT is_vet_diet_flag)+IFF(COUNT_IF(is_vet_diet_flag IS NULL)>0,1,0),
 COUNT(DISTINCT approval_channel)+IFF(COUNT_IF(approval_channel IS NULL)>0,1,0),
 COUNT(DISTINCT prescription_source)+IFF(COUNT_IF(prescription_source IS NULL)>0,1,0)) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS GROUP BY rx_id
), aged_keys AS (
 SELECT l.order_id,l.order_line_id,MAX(p.registry_variants) AS registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l JOIN registry p
 ON NULLIF(TRIM(l.rx_id),'')=p.rx_id AND l.customer_id=p.customer_id
 WHERE l.is_vet_diet_flag=TRUE AND p.is_vet_diet_flag=TRUE
 AND p.approval_channel='CUSTOMER_UPLOAD' AND p.prescription_source='CMPU'
 AND LOWER(TRIM(l.order_cancel_reason))='aged order'
 AND l.order_cancel_dttm>='2026-05-01'::TIMESTAMP_NTZ
 AND l.order_cancel_dttm<'2026-08-01'::TIMESTAMP_NTZ
 GROUP BY l.order_id,l.order_line_id
), aged_lines AS (
 SELECT l.order_id,l.order_line_id,l.customer_id,l.order_placed_dttm AS placed_at_utc,
 l.order_cancel_dttm AS cancelled_at_utc,k.registry_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
), cohort_customers AS (
 SELECT DISTINCT customer_id FROM aged_lines
), vet_diet_order_keys AS (
 -- Full vet-diet history; no CMPU, channel, RX-ID or date restriction here.
 SELECT DISTINCT l.customer_id,l.order_id
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN cohort_customers c ON l.customer_id=c.customer_id
 WHERE l.is_vet_diet_flag=TRUE
), history_keys AS (
 SELECT DISTINCT order_id FROM vet_diet_order_keys
), history AS (
 SELECT o.order_id,MIN(o.customer_id) AS customer_id,
 MIN(o.order_placed_dttm) AS placed_at_utc,MIN(o.order_first_order_placed_flag) AS first_placed_flag,
 COUNT(*) AS source_rows,
 GREATEST(COUNT(DISTINCT o.customer_id)+IFF(COUNT_IF(o.customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_placed_dttm)+IFF(COUNT_IF(o.order_placed_dttm IS NULL)>0,1,0),
 COUNT(DISTINCT o.order_first_order_placed_flag)+IFF(COUNT_IF(o.order_first_order_placed_flag IS NULL)>0,1,0)) AS
  source_variants
 FROM EDLDB.ECOM.ORDERS o JOIN history_keys k ON o.order_id=k.order_id GROUP BY o.order_id
), vet_history_audit AS (
 SELECT v.customer_id,COUNT(DISTINCT IFF(h.order_id IS NULL,v.order_id,NULL)) AS vet_orders_missing_ecom_header
 FROM vet_diet_order_keys v LEFT JOIN history h ON v.order_id=h.order_id AND v.customer_id=h.customer_id
 GROUP BY v.customer_id
), ranked AS (
 SELECT *,MIN(placed_at_utc) OVER (PARTITION BY customer_id) AS earliest_placed_at_utc,
 ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY placed_at_utc NULLS LAST,order_id) AS first_rank
 FROM history
), first_orders AS (
 SELECT customer_id,COUNT(*) AS history_orders,
 COUNT_IF(placed_at_utc IS NULL) AS history_orders_missing_placement,
 MAX(source_variants) AS history_source_variants,
 SUM(source_rows-1) AS history_identical_duplicate_rows,
 MIN(earliest_placed_at_utc) AS first_placed_at_utc,
 COUNT_IF(placed_at_utc=earliest_placed_at_utc) AS earliest_timestamp_ties,
 MAX(IFF(first_rank=1,order_id,NULL)) AS first_order_id,
 COUNT_IF(first_placed_flag=TRUE) AS flagged_first_orders,
 MIN(IFF(first_placed_flag=TRUE,order_id,NULL)) AS flagged_first_order_id
 FROM ranked GROUP BY customer_id
)
```

```
, selected_aged_keys AS (
 -- Only the first vet-diet aged order per customer is an anchor.
 SELECT DISTINCT a.order_id,a.order_line_id,
 GREATEST(a.registry_variants,f.history_source_variants) AS registry_variants
 FROM aged_lines a JOIN first_orders f ON a.customer_id=f.customer_id AND a.order_id=f.first_order_id
 JOIN vet_history_audit v ON a.customer_id=v.customer_id
 WHERE f.first_placed_at_utc IS NOT NULL AND f.earliest_timestamp_ties=1
 AND f.history_orders_missing_placement=0 AND v.vet_orders_missing_ecom_header=0
), originals AS (
 SELECT l.order_id AS original_order_id,l.order_line_id AS original_line_id,
 MAX(k.registry_variants) AS registry_variants,
 MIN(l.customer_id) AS original_customer_id,MIN(l.pet_id) AS original_pet_id,
 MIN(l.product_part_number) AS original_product_part_number,
 MIN(l.order_cancel_dttm) AS original_cancel_utc,
 MIN(LOWER(TRIM(l.order_cancel_reason))) AS original_reason,
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0) AS customer_variants,
 COUNT(DISTINCT l.pet_id)+IFF(COUNT_IF(l.pet_id IS NULL)>0,1,0) AS pet_variants,
 COUNT(DISTINCT l.product_part_number)+IFF(COUNT_IF(l.product_part_number IS NULL)>0,1,0) AS part_variants,
 COUNT(DISTINCT l.order_cancel_dttm)+IFF(COUNT_IF(l.order_cancel_dttm IS NULL)>0,1,0) AS cancel_variants,
 COUNT(DISTINCT LOWER(TRIM(l.order_cancel_reason)))+IFF(COUNT_IF(l.order_cancel_reason IS NULL)>0,1,0) AS
  reason_variants
 FROM EDLDB.BT_HCA_HCDM.ORDER_LINE_COST_MEASURES_RXP l
 JOIN selected_aged_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 GROUP BY l.order_id,l.order_line_id
), candidate_keys AS (
 SELECT DISTINCT l.order_id,l.order_line_id
 FROM EDLDB.ECOM.ORDER_LINE l JOIN originals a ON l.customer_id=a.original_customer_id
 AND l.order_id<>a.original_order_id
 AND l.order_placed_dttm>a.original_cancel_utc
 AND l.order_placed_dttm<=DATEADD(DAY,70,a.original_cancel_utc)
 AND l.order_placed_dttm<='2026-10-28'::TIMESTAMP_NTZ
), candidate_lines AS (
 -- All lines on each candidate order are needed for whole-order exclusion.
 SELECT l.* FROM EDLDB.ECOM.ORDER_LINE l
 JOIN (SELECT DISTINCT order_id FROM candidate_keys) k ON l.order_id=k.order_id
), order_cancellations AS (
 SELECT order_id,MAX(IFF(order_order_line_status='X' OR order_status='X'
 OR NULLIF(TRIM(order_cancel_reason),'') IS NOT NULL OR order_cancel_dttm IS NOT NULL,1,0)) AS order_has_cancellation,
 MAX(IFF(order_order_line_status IS NULL,1,0)) AS order_has_unknown_status
 FROM candidate_lines GROUP BY order_id
), reorders AS (
 -- ECOM.ORDER_LINE is a physical-line view; NET_SALES is the full line amount.
 -- Exact source duplicates collapse. Conflicting line values fail validation.
 SELECT l.order_id AS reorder_order_id,l.order_line_id AS reorder_line_id,
 MIN(l.customer_id) AS reorder_customer_id,
 NULL::NUMBER(38,0) AS reorder_pet_id,NULL::VARCHAR AS reorder_product_part_number,
 MIN(l.order_placed_dttm) AS reorder_placed_utc,
 MIN(l.order_line_shipped_dttm) AS reorder_shipped_utc,
 MIN(l.net_sales)::VARCHAR AS reorder_known_net_sales,
 IFF(COUNT_IF(l.net_sales IS NULL)>0,1,0) AS missing_sales_records,
 COUNT(*) AS ecom_source_rows,
 MIN(l.order_line_currency) AS ecom_currency,
 GREATEST(
 COUNT(DISTINCT l.customer_id)+IFF(COUNT_IF(l.customer_id IS NULL)>0,1,0),
 COUNT(DISTINCT l.order_placed_dttm)+IFF(COUNT_IF(l.order_placed_dttm IS NULL)>0,1,0),
 COUNT(DISTINCT l.net_sales)+IFF(COUNT_IF(l.net_sales IS NULL)>0,1,0),
 COUNT(DISTINCT l.order_line_currency)+IFF(COUNT_IF(l.order_line_currency IS NULL)>0,1,0)) AS source_variants,
 MAX(o.order_has_cancellation) AS reorder_order_has_cancellation,
 MAX(o.order_has_unknown_status) AS reorder_order_has_unknown_status
 FROM candidate_lines l JOIN candidate_keys k ON l.order_id=k.order_id AND l.order_line_id=k.order_line_id
 JOIN order_cancellations o ON l.order_id=o.order_id
 GROUP BY l.order_id,l.order_line_id
)
SELECT a.* REPLACE(TO_CHAR(original_cancel_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS original_cancel_utc),
 r.* REPLACE(TO_CHAR(reorder_placed_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_placed_utc,
 TO_CHAR(reorder_shipped_utc,'YYYY-MM-DD HH24:MI:SS.FF9') AS reorder_shipped_utc),
 (SELECT COUNT_IF(order_line_id IS NULL) FROM candidate_keys) AS candidate_missing_line_keys
FROM originals a LEFT JOIN reorders r ON r.reorder_customer_id=a.original_customer_id
 AND r.reorder_order_id<>a.original_order_id AND r.reorder_placed_utc>a.original_cancel_utc
 AND r.reorder_placed_utc<=DATEADD(DAY,70,a.original_cancel_utc)
```

2.1.7 Section Reproduction Steps

From the repository root, after `make setup`, replay the retained May–July 2026 extracts and rebuild the PDF:

```
make reproduce-financial-section
make report
```

The section target uses cached inputs only. It recreates Table 1 with `opportunity-order-cancellation-summary` using `DATE_BASIS=shipped-or-cancelled` and `WRITE_LATEX=1`. The four follow-up comparisons use `opportunity-follow-up-sales`, `opportunity-new-upload-customer-follow-up` and `opportunity-new-vet-diet-customer-follow-up`, including their Chewy-wide companions, then `combined-follow-up-table`. `financial-sql-listings` recreates the executable SQL listings. All use the displayed May 1 inclusive–August 1 exclusive cohort and October 28, 2026 observation cutoff; follow-up windows are 7 through 70 days.

Retained Parquet extracts, manifests and first-order cohort inputs under `data/opportunity/` are required and are not stored in Git. Fingerprints are checked before reuse. To obtain a new warehouse snapshot, run the underlying targets without `CACHED=1`, following HOWTO.md for the first-order history prerequisites. New snapshots may change counts.

DRAFT

2.2 What is the contact pattern of CMPU aged order customers?

2.2.1 What is the distribution of CMPU customers with APPROVAL CHANNEL = CUSTOMER_UPLOAD that contacted Chewy by cancellation reason?

Table 6 compares customers with inbound contacts across CMPU vet-diet `CUSTOMER_UPLOAD` orders with qualifying shipment or cancellation activity from May 1, 2026 through July 31, 2026, inclusive, UTC.

Table 6: Distinct customers contacting by current order cancellation reason, through the latest observed shipment or cancellation event.

Cancellation reason	Cohort customers	Customers contacting	Contacted (%)
[SQL NULL]	172,878	51,983	30.07%
Aged order	30,382	30,304	99.74%
Customer initiated cancellation	29,455	10,567	35.88%
Customer request	11,382	9,377	82.38%
Duplicate order	161	111	68.94%
Fraud	36	13	36.11%
Inventory issue	5	2	40.00%
No response	1,035	898	86.76%
Other	784	661	84.31%
Payment declined	4,242	2,393	56.41%
Payment issue	46	32	69.57%
Pharmacy rejection	278	264	94.96%
Prescription issue	5,051	5,012	99.23%
Restricted availability	1	1	100.00%
TOTAL	225,434	101,227	44.90%

Notes: No lower or upper day limit relative to order creation applies. Include contacts at any available earlier timestamp, capped inclusively at the latest observed shipment/cancellation timestamp among qualifying rows for that order and reason. For split or mixed outcomes, use the later of the observed events. Match inbound contacts on the original order ID and numeric customer ID, across all categories. Include handled, unhandled and unknown-handled inbound attempts. Count each customer once per reason across all their qualifying orders and contact days. Percentages divide contacting customers by all eligible customers in that reason, including those without a qualifying contact. Customers can appear in multiple reason rows; TOTAL independently counts each customer once overall and is not the sum of the reason rows. Reasons reflect the current snapshot, grouped case-insensitively after trimming. SQL NULL is explicit; blank strings, if present, form a separate row. NULL does not imply no cancellation. This cohort includes 434 NULL-reason order/reason records with observed cancellation but no qualifying shipment. This summary has no creation-relative day bounds. Observation ends at each order's observed event, so exposure varies; these are descriptive contact shares, not causal effects. Exact duplicate contacts collapse; missing or mismatching contact customers, missing contact times and non-inbound directions are excluded and audited. Inbound direction is a source proxy, not proof of human initiation. No prescription-creation or first-order restriction applies. Extraction time (UTC): 2026-11-02T20:50:53.130130+00:00. Executable SQL and complete order/contact records are under `data/opportunity/customer-contacts-by-cancel-reason/`.

DRAFT

2.2.2 When did the CMPU aged order customers contact Chewy relative to the cancellation date?

Table 7 summarizes inbound contacts linked to CMPU vet-diet `CUSTOMER_UPLOAD` aged orders cancelled from May 1, 2026 through July 31, 2026, inclusive, UTC. The full cohort contains 30,382 customers and 38,314 aged orders.

Table 7: Customer-initiated inbound contacts by day relative to aged-order cancellation: distinct customers contacting and distinct aged order IDs, from 13 days before through day 3 after cancellation. A total of 30,299 distinct customers contacted across this window.

Days relative to cancellation	Customers contacting	Aged order IDs
-13	38	38
-12	858	861
-11	19,267	23,158
-10	11,364	13,530
-9	571	575
-8	215	215
-7	143	143
-6	123	123
-5	106	106
-4	99	99
-3	128	128
-2	251	256
-1	248	248
0	71	71
1	53	53
2	28	28
3	22	23

Notes: Customer-initiated is defined by `SESSION_INITIAL_DIRECTION=inbound` in `EDLDB.BT_CUSTOMER_CARE.FCT_CONTACTS_BRD`, across all contact categories. Include handled, unhandled and unknown-handled inbound attempts. Match the original aged `ORDER_ID` and require the contact's numeric `CUSTOMER_ID` to match the order's customer. Use UTC `SESSION_START_DTTM`. Day n is the elapsed 24-hour interval $[n, n+1)$ days relative to cancellation: day -1 is the preceding 24 hours, and day 0 starts exactly at cancellation. These are elapsed bins rather than calendar-date differences. All seventeen rows from -13 through $+3$ are retained, with zero for missing days. Count each customer and aged order once per day, regardless of repeat contact legs, sessions or qualifying order lines. Counts may overlap across days and should not be added as global unique totals. Missing timestamps or customer IDs, mismatching customer IDs, unknown directions and outbound contacts remain audited and are excluded from this table. Email contacts with Other ingress are included; inbound direction is a source proxy and does not establish human initiation. Full channel/ingress audits are exported with the data. Original eligibility uses cancellation dates and all qualifying orders, without a first-order or prescription-creation restriction. Observation cutoff: 2026-10-28 00:00 UTC, allowing the full positive-day window.

DRAFT

2.2.3 What was the frequency of contacting Chewy for CMPU aged order customers?

Table 8 groups customers by the number of distinct relative days on which they contacted during the same -13 through $+3$ window.

Table 8: Frequency of customer contact days around aged-order cancellation: number of distinct relative days contacted versus number of customers. Of 30,382 cohort customers, 30,299 contacted on at least one day.

Number of days contacted	Number of customers
0	83
1	27,265
2	2,808
3	201
4	24
5	1
6	0
7	0
8	0
9	0
10	0
11	0
12	0
13	0
14	0
15	0
16	0
17	0

Notes: Same CMPU vet-diet `CUSTOMER_UPLOAD` aged cohort, cancellation dates, observation cutoff and inbound contact rules as Table 7. Count each distinct relative 24-hour day bin from -13 through $+3$ once per customer across all qualifying aged orders. Multiple contacts, sessions, legs or orders on the same relative day do not increase the day count. These are relative cancellation-day bins, rather than distinct calendar dates. Each customer appears in exactly one frequency row; zero includes cohort customers with no qualifying contact in this window. All frequencies from 0 through 17 are retained, including zero-count rows. Customer counts sum to the full cohort; positive-day rows sum to the distinct contacting customers.

2.2.4 Does more frequent contact imply higher propensity to re-order successfully for the same pet and part number in 14 days?

Table 9 applies the contact-day frequency segments in Table 8 to same-pet, same-product reorders.

Table 9: Same-pet, same-product-part-number reorders within 14 days after aged cancellation, by customer contact-day frequency. Original CMPU vet-diet `CUSTOMER_UPLOAD` orders were cancelled from May 1, 2026 through July 31, 2026, inclusive, UTC.

Days contacted	Segment customers	Customers reordered	Segment reordered (%)
1	27,265	3,376	12.38%
2	2,808	396	14.10%

Notes: Segments count distinct relative 24-hour contact-day bins from -13 through $+3$, pooled across each customer's qualifying aged orders. Multiple contacts on the same relative day count as one day. Match inbound contacts to the original aged order and its customer, across all categories. Include handled, unhandled and unknown-handled inbound attempts. Reorders must match the customer, non-null `PET_ID` and nonblank `PRODUCT_PART_NUMBER`, on a different order placed strictly after its matched aged cancellation through 14 days inclusive. Apply the established non-cancelled-order rules: exclude an entire reorder order if any line has cancellation evidence or unknown status. Reorders may use any approval channel or prescription source. Count each customer once in their segment if any qualifying aged order has a matching reorder in its own follow-up window. Percentages use all customers in the contact segment, including those missing matching keys or without reorders; N/A denotes an empty segment. Contact-day segments include post-cancellation contacts through day $+3$. Observation cutoff: 2026-10-28 00:00 UTC.

DRAFT

2.2.5 Does contact closer to cancellation imply higher propensity to re-order successfully in 14 days for the same pet and part number?

Table 10 compares customers with a qualifying contact on days -7 through $+3$ relative to aged cancellation against the rest of the aged-order cohort. The contacting group's 14-day reorder rate is 7.70 percentage points higher.

Table 10: Same-pet, same-product-part-number reorders within 14 days after aged cancellation: customers contacting on days -7 through $+3$ versus the rest of the cohort. Original CMPU vet-diet `CUSTOMER_UPLOAD` orders were cancelled May 1–July 31, 2026, inclusive, UTC.

Customer group	Customers	Customers reordered	Reorder rate (%)
Contacted on days -7 through $+3$	1,184	237	20.02%
Rest of aged cohort	29,198	3,595	12.31%

Notes: A customer belongs to the contacting group if any qualifying inbound contact is linked to one of their original aged orders and the contact customer ID matches the order's customer. Include all contact categories and handled states. Days are elapsed 24-hour bins: the window starts exactly seven days before cancellation and ends just before four days after cancellation, covering all of day $+3$. The complementary group includes customers contacting only outside this window and customers with no qualifying contacts. Each customer belongs to exactly one group; the groups sum to 30,382 customers and 3,832 returning customers. Use the same non-cancelled same-customer, `PET_ID` and `PRODUCT_PART_NUMBER` reorder endpoint as Table 9: a different order placed strictly after its matched aged cancellation through 14 days inclusive. Count a customer once if any qualifying aged order has a matching reorder in its own window; the contact need not concern that same aged order. Percentages use all customers in each group. Observation cutoff: October 28, 2026 at 00:00 UTC. Contacts after cancellation contribute to group membership; this is a descriptive comparison and does not establish a causal effect of contact.

DRAFT

2.2.6 Section Reproduction Steps

From the repository root, after `make setup`, run:

```
make reproduce-contact-section
make report
```

The target reuses fingerprint-checked May–July 2026 inputs and the October 28, 2026 observation cutoff; no warehouse refresh occurs. It runs:

- `opportunity-customer-contacts-by-cancel-reason` for Table 6; matching inbound contacts have no creation-relative day bounds and end at the observed order endpoint.
- `opportunity-aged-customer-contacts` for Tables 7 and 8, using elapsed cancellation days -13 through $+3$ and all handled states.
- `opportunity-contact-frequency-reorders` for Table 9 and `opportunity-contact-window-reorders` for Table 10. Both use successful same-customer, same-pet and same-part-number reorders within 14 days.

The last target also regenerates the frequency study from its verified cache before calculating the contact-window comparison. Retained inputs under `data/opportunity/` are required; they are not in Git. For new Snowflake snapshots, run the underlying extraction targets without `CACHED=1` using the same dates, then regenerate the comparisons. See HOWTO.md.

DRAFT

2.3 What are the delays in processing uploaded prescriptions?

An order becomes an aged order when canceled automatically after the time between order creation and time of shipment or any other cancellation exceeds 11 days. A detailed breakdown of cancellation reasons can be found in Figure 1 for the study population.

The population of orders for this study consists of first orders and first order lines for prescriptions as we would like to analyze issues at prescription upload and approval.

For this analysis; 95,073 vet diet prescriptions created between August 15, 2026 and October 15, 2026 were used with `CUSTOMER_UPLOAD` as the approval channel. This corresponds to 87,724 distinct orders and 95,073 order lines. The 24,408 orders described in `OMS` with the cancellation reason `AGED_ORDER` constitute the aged orders leading to 26,443 line items, 26,443 corresponding prescriptions and 20,587 customers.

The difference in counts between first orders and order lines and all orders and order lines is shown in Table tab:first-all-order-cancellations.

Factors that might be contributing to an order aging are: delay in prescription upload by the customer, delay in processing the upload, and delay in approving the prescription after the uploaded prescription document is accepted.

The data for uploaded documents/entities does not cover all the prescriptions, the coverage is about 31% in this study. The number of prescription documents uploaded for 26,443 aged order prescriptions is 782 (3%) compared to 33,336 uploads for 95,073 prescriptions overall (35%) pointing to a severe non-response by the customers. See Table 11.

Only 6 aged order related prescription were approved out of 52 accepted/782 uploaded (See Table 12 for comparison with the rest of the order line population.) The median time for these prescriptions to become accepted was 10 days after upload, as a result missing the aged order cancellation deadline (Figure 4). For comparison, the distributions of the time difference between upload of the documents and their final states are in Figure 3 for all entities. Customers seem to upload at similar delays after order creation for overall and aged order prescriptions (Figure 2).

From the available entity upload data, it takes less than a minute to approve an accepted prescription document see Figure 5.

Table 11: Study population and uploaded-document counts. Counts are distinct within each population.

					Uploaded documents	
Population	Prescriptions	Orders	Order lines	Customers	Total	Accepted
All approval channels, matched to first-order lines	103,516	95,533	103,516	80,649	30,735	26,254
`CUSTOMER_UPLOAD`, matched to first-order lines	95,073	87,724	95,073	73,905	29,715	25,606
`CUSTOMER_UPLOAD` aged-cancellation subset	26,443	24,408	26,443	20,587	782	52

Notes: CMPU vet-diet prescriptions created from August 15, 2026 inclusive through October 15, 2026 exclusive, UTC. The aged-cancellation subset includes qualifying cancellations observed through October 27, 2026, 00:00 UTC. Line associations and document counts were refreshed on October 28, 2026. Uploaded documents count distinct linked `DIET_PRESCRIPTION` upload entities created August 1–October 25, 2026, UTC, with a matched first-order line; entities shared by prescriptions count once per row. Accepted documents have current `STATE_NAME = ACCEPTED`.

Table 12: Prescription decisions for the study populations.

Population	Prescriptions	Approved	Unknown	Rejected
All approval channels, matched to first-order lines	103,516	36,874	66,595	47
`CUSTOMER_UPLOAD`, matched to first-order lines	95,073	30,504	64,569	0
`CUSTOMER_UPLOAD` aged-cancellation subset	26,443	6	26,437	0

Notes: Distinct prescriptions in the same populations as Table 11. Approved means `PRESCRIPTION_APPROVED = TRUE`, consistent with the prescription registry's `APPROVED_NUMERATOR`. Rejected means current `RX_STATUS = DENIED` without an approval flag. Unknown includes cancelled and pending prescriptions without a recorded approval or rejection outcome. The three categories are mutually exclusive and sum to the prescription count in each row. Decisions use the current source snapshot refreshed October 28, 2026; approval and denial times are not restricted to the aged-cancellation cutoff.

DRAFT

Table 13: Cancellation reasons, order counts and net sales for first orders and all linked orders.

	First orders			All orders		
Cancellation reason	Orders	Lines	Net sales	Orders	Lines	Net sales
[Aged order outside cutoff]	372	398	33,489.70	372	398	33,489.70
[No recorded cancellation reason]	38,413	40,864	3,163,597.96	60,879	69,510	5,372,693.08
Aged order	24,408	26,443	2,236,727.20	24,409	26,445	2,236,909.55
Customer initiated cancellation	11,601	13,127	995,271.62	12,255	13,933	1,064,806.64
Customer request	4,890	5,486	428,933.38	5,037	5,683	446,285.60
Duplicate order	61	64	5,237.98	74	80	6,499.90
Fraud	13	15	1,809.92	13	15	1,809.92
Inventory issue	3	3	264.42	3	3	264.42
No response	541	575	48,509.38	559	593	50,252.01
Other	302	336	25,662.96	317	354	26,850.17
Payment declined	2,444	2,679	221,343.75	2,541	2,786	229,682.76
Payment issue	15	17	1,503.07	16	18	1,638.91
Pharmacy rejection	137	157	13,237.61	137	157	13,237.61
Prescription issue	4,524	4,909	412,858.46	4,524	4,910	412,972.44
Reseller	1	1	132.99	1	1	132.99
TOTAL	87,725	95,074	7,588,580.40	111,137	124,886	9,897,525.70

Notes: CMPU vet-diet `CUSTOMER_UPLOAD` prescriptions created August 15–October 14, 2026, UTC. All orders includes first and subsequent linked orders, without an order-placement window. Qualifying aged cancellations are observed on or before October 27, 2026, 00:00 UTC; current aged reasons outside that cutoff are shown separately for reconciliation. Other reasons reflect the comparison snapshot. Counts are distinct orders and physical lines within each group. Net sales sum `ORDER_LINE_NET_SALES` once per shipment/line record, for qualifying lines only. Amounts are displayed to two decimals; five-decimal values remain in the data exports. Gray net-sales amounts have a recorded cancellation reason and represent unrealized sales, not realized revenue. TOTAL includes these gray amounts. The validated comparison snapshot contains one additional matched first-order line/order relative to the earlier frozen population in Table 11.

DRAFT

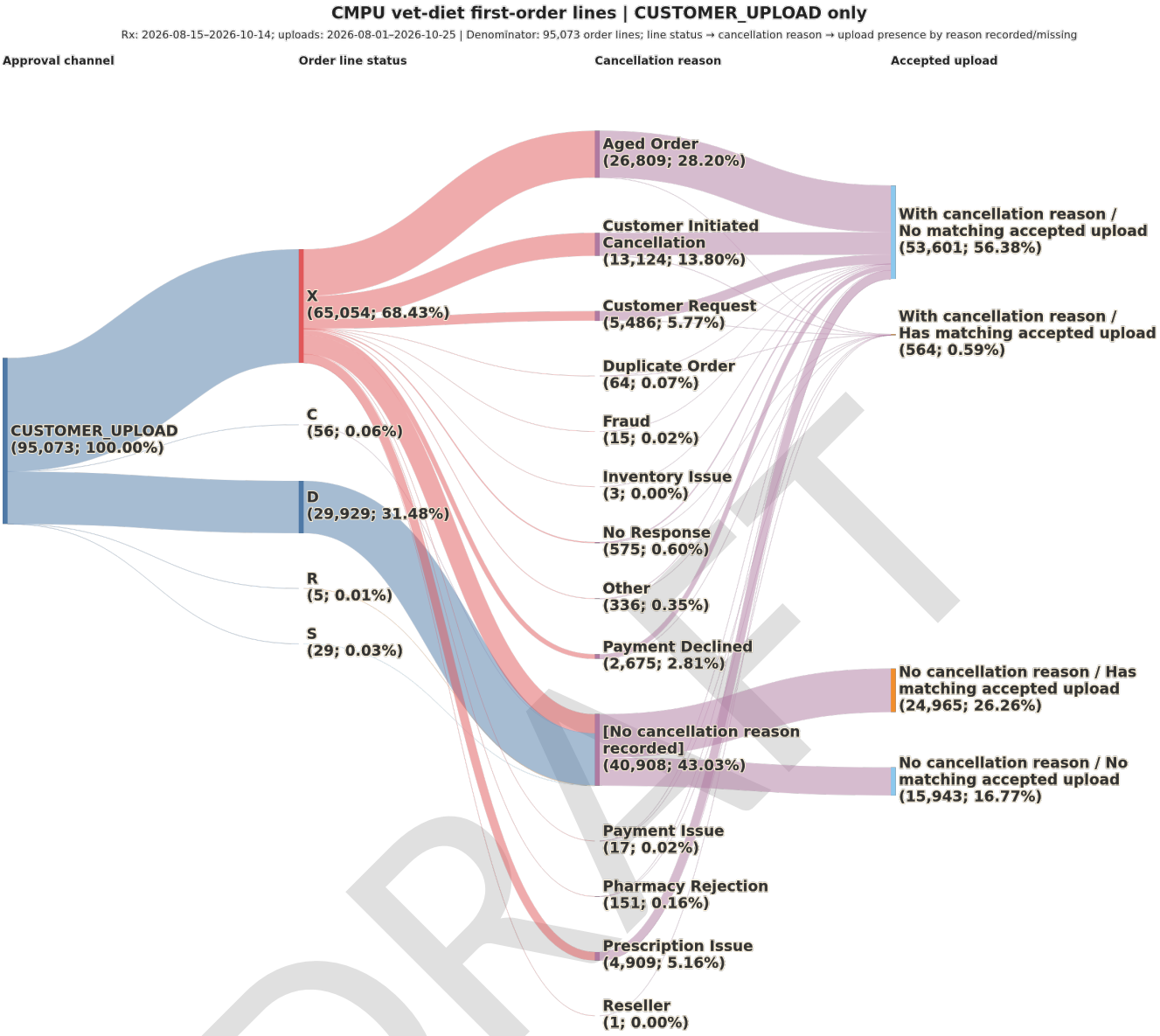


Figure 1: CMPU vet-diet first-order lines: `CUSTOMER_UPLOAD` approval channel.

DRAFT

2.3.1 How long do customers wait to upload a prescription?

Figure 2 compares upload entity creation relative to order placement for the overall and aged-order populations.

The distribution of time between document upload for the whole analysis population and the aged orders do not differ significantly.

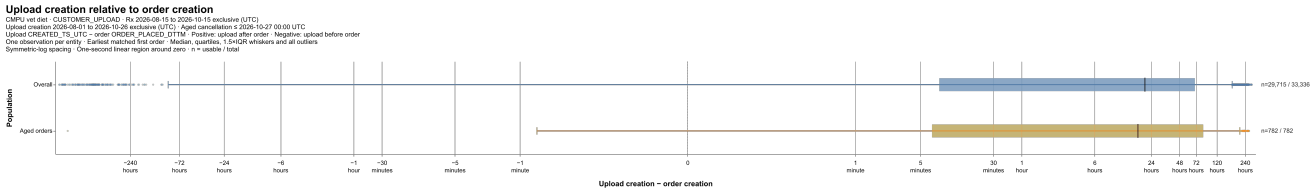


Figure 2: Upload creation relative to order placement for the overall and aged-order entity populations. Elapsed time is upload creation minus order placement: positive values indicate upload after ordering and negative values indicate upload before ordering. Each entity uses its earliest matched first-order placement. Plotted durations include 29,715 overall entities and 782 aged-order entities; entities missing an order-placement timestamp are excluded from the boxplots. Medians, quartiles, 1.5-times-IQR whiskers and all outliers are shown.

2.3.2 How long does it take for an uploaded prescription to be ready for approval?

Figures 3 and 4 show the elapsed time from upload entity creation to its current recorded state change, grouped by final state, for the overall study-linked population and its aged-order subset. Both figures use the same state order and time scale.

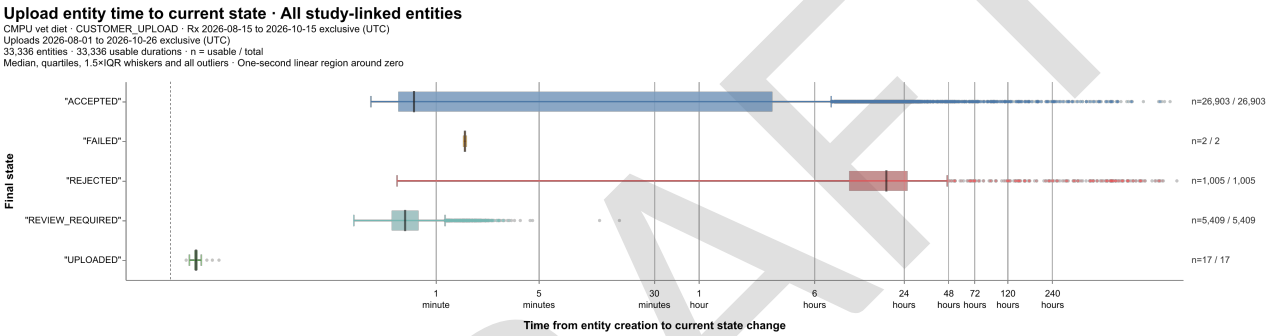


Figure 3: Upload creation to current recorded state change for all study-linked CMPU vet-diet `CUSTOMER_UPLOAD` entities ($N = 33{,}336$). Each upload entity counts once. Boxplots show medians, quartiles, 1.5-times-IQR whiskers and all outliers, grouped by final state.

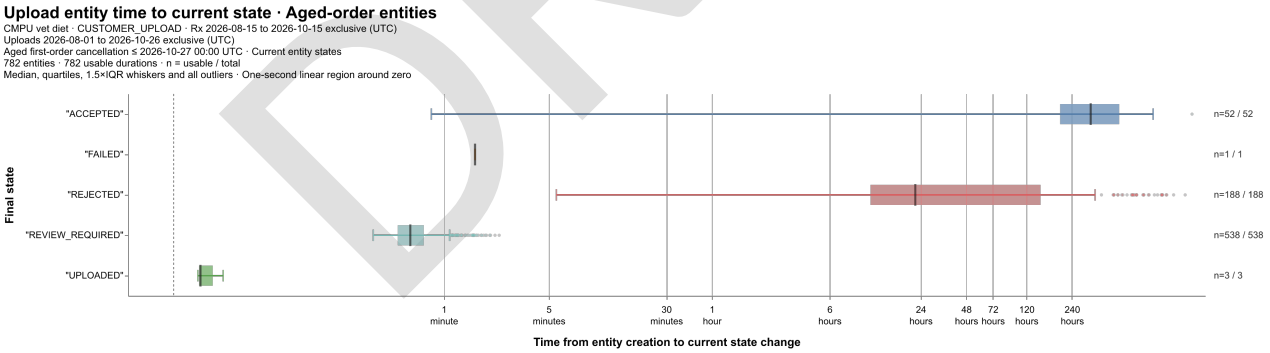


Figure 4: Upload creation to current recorded state change for the aged-order entity subset ($N = 782$). A qualifying matched first-order line was cancelled for aging on or before October 27, 2026, 00:00 UTC. Current entity states and state-change times are not capped at cancellation.

DRAFT

2.3.3 How long does it take for an accepted prescription document to get approved?

Figure 5 shows the cumulative distribution of the interval between the selected upload entity's accepted-state change and the latest recorded approval for qualifying first-order lines.

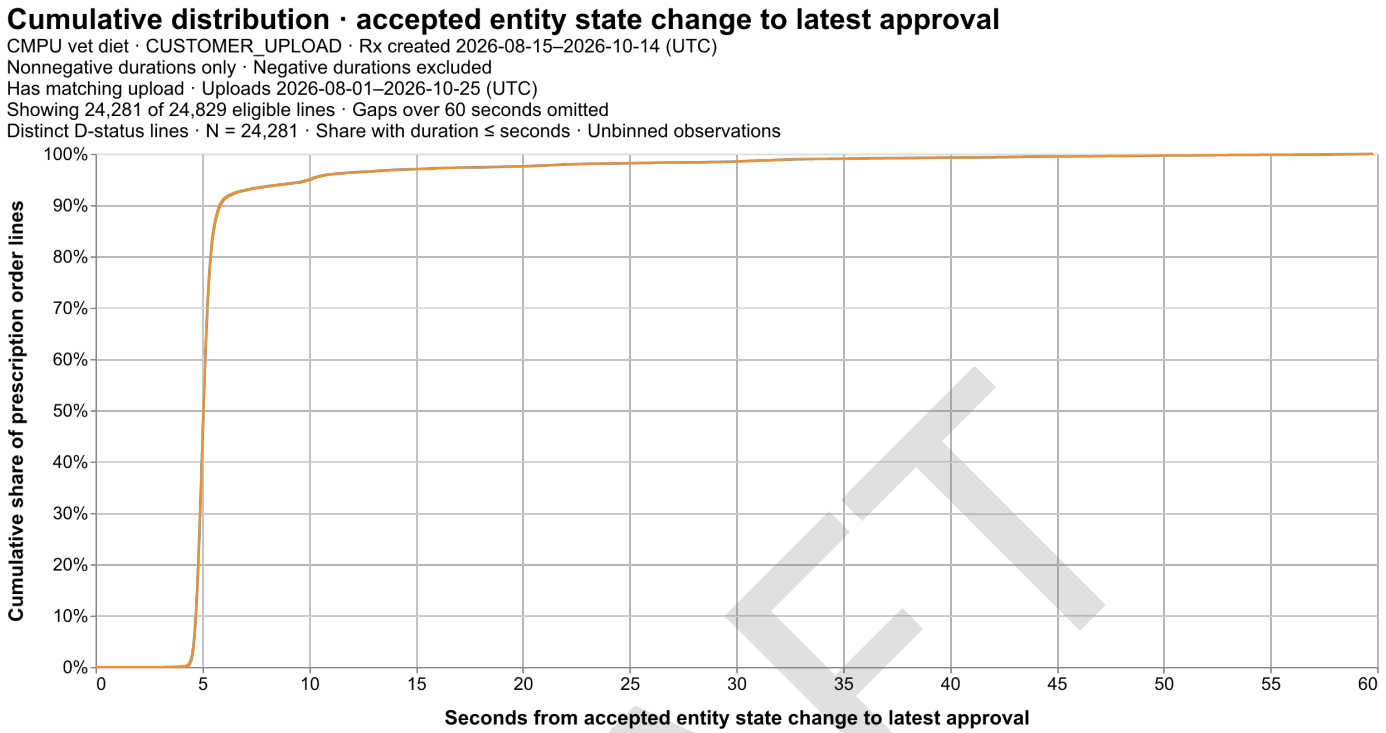


Figure 5: Cumulative distribution of accepted-upload-to-approval time for CMPU vet-diet `CUSTOMER_UPLOAD` first-order lines with current status D. Elapsed time is the latest recorded approval timestamp minus the selected accepted entity's state-change timestamp. The curve shows only nonnegative gaps from 0 through 60 seconds inclusive and is normalized to that subset; gaps above 60 seconds are omitted. Each distinct matched first-order line counts once.

DRAFT

2.3.4 Section Reproduction Steps

After `make setup`, ensure Docker is running and accessible for the Mermaid Sankey renderer. From the repository root, run:

```
make reproduce-upload-delay-section
make report
```

This replays the retained August 15 inclusive–October 15 exclusive prescription cohort, August 1–October 25 upload discovery window, and October 27, 2026 aged cancellation cutoff. The section target runs:

- `study-population-tables`: validates and renders Tables 11 and 12, then invokes `prescription-cohort-order-comparison` for Table 13.
- `cmpu-first-order-line-status`: regenerates the `CUSTOMER_UPLOAD` Sankey in Figure 1.
- `cmpu-upload-entity-order-creation-distributions`: regenerates Figure 2.
- `cmpu-upload-entity-state-change-distributions`: regenerates Figures 3 and 4.
- `cmpu-upload-to-approval-distribution`: regenerates Figure 5.

All inputs are fingerprint-checked retained snapshots under `data/`; no warehouse refresh occurs. The two population tables retain their frozen first-order membership, while the financial comparison uses its documented later snapshot. Exact replay requires these ignored extracts and manifests. See HOWTO.md for source SQL and extraction steps; new snapshots can change membership and must be reconciled before replacing the frozen populations.

DRAFT

2.4 What is the probability that after X days the order will be an aged order?

We look at the chance of an order becoming an aged order in this section. Here, we build a survival curve. Survival means that an order has yet to encounter a document upload or cancellation. The cancellations after day 10 are system created aged order cancellations. From figure 6, that after day 3, the change of an order becoming an aged order is about 50% (28.90%/55.80%).

Figure 6 shows the probability that a first-order line is still awaiting an accepted upload or cancellation after order placement.

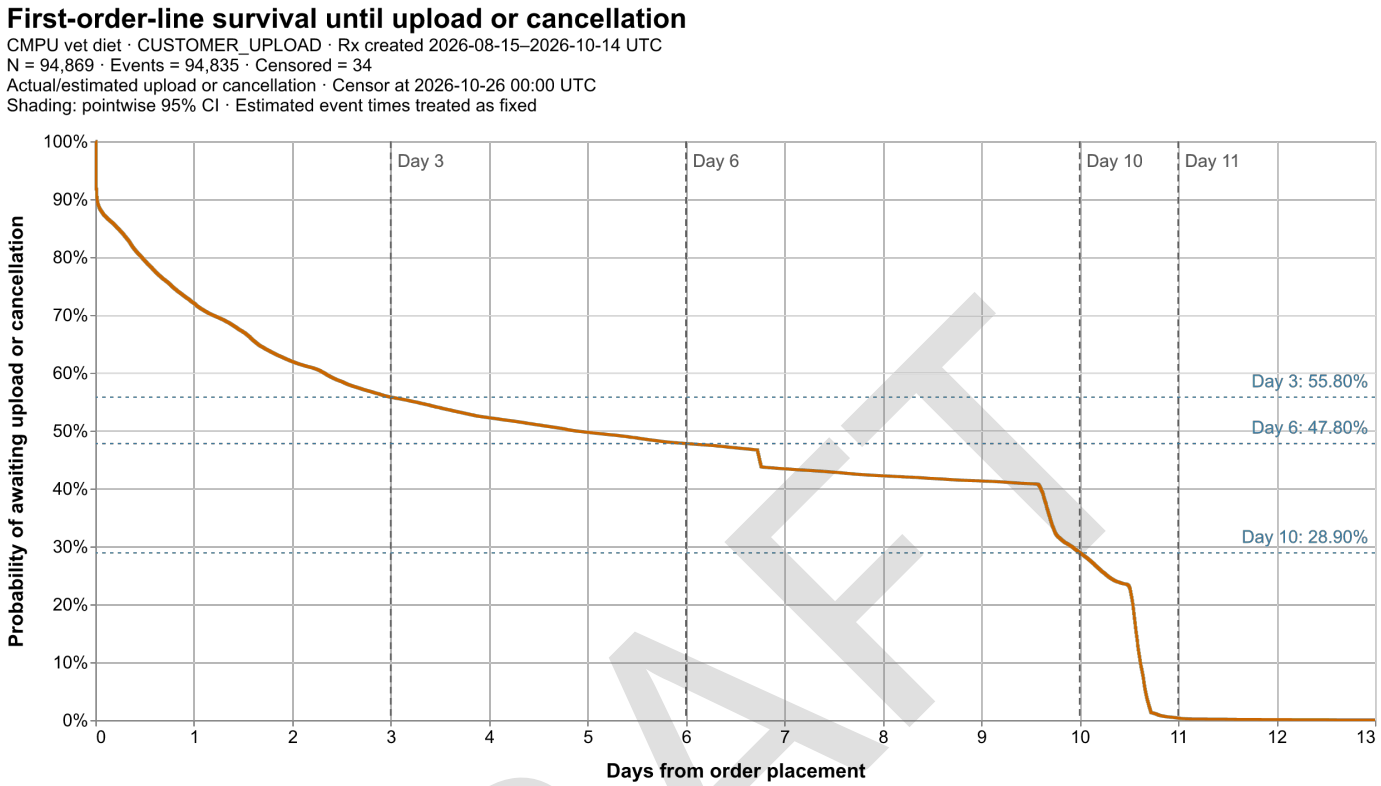


Figure 6: Kaplan–Meier survival curve for 94,869 eligible distinct first-order lines linked to CMPU vet-diet `CUSTOMER_UPLOAD` prescriptions created August 15–October 14, 2026, inclusive, UTC. Time zero is order placement; the event is the earlier of the actual or estimated accepted-entity state-change time and order cancellation. The upload-entity discovery window is August 1–October 25, 2026, inclusive. Missing events and events after October 26, 2026 at 00:00 UTC are right-censored at that cutoff; events exactly at the cutoff are observed. Lines with missing placement, placement after the cutoff or negative event durations are excluded. Shading shows pointwise 95% log-log Greenwood confidence intervals conditional on the supplied event times; it does not include uncertainty from the fixed 0.96 estimation rule. The plot displays days 0–13, with vertical markers at days 3, 6, 10 and 11 and horizontal guides at days 3, 6 and 10. Survival means neither accepted-state change nor cancellation has occurred; it does not measure upload completion alone.

DRAFT

2.4.1 Section Reproduction Steps

From the repository root, after `make setup`, run:

```
make reproduce-survival-section
make report
```

The target runs `cmpu-first-order-line-survival`, which validates the retained event-time source and regenerates Figure 6. It uses prescriptions created August 15 inclusive–October 15 exclusive, 2026, and upload discovery starting August 1. The retained source defines an exclusive October 26 upload endpoint and the same October 26, 2026, 00:00 UTC right-censoring cutoff.

Retained event-time Parquet extracts and manifests under `data/cmpu-first-order-line-event-times/` are required and are not in Git. The plotting target checks the source query, dates and fingerprint, and performs no extraction. To obtain a new snapshot, run `make cmpu-first-order-line-event-times` first with the same dates; then rerun the section target. New snapshots may change the curve.

2.5 How do I reproduce the results and the report?

The code to reproduce all the results is in [1]. Follow the instructions in the HOWTO.md file to generate the figures, models and analyses. You will need to clone the repository to run the code. The code uses make and python.

Section-specific commands and table/figure mappings are in Sections 2.1.7, 2.2.6, 2.3.4 and 2.4.1. Those commands replay retained, fingerprint-checked extracts; the ignored `data/` files are required in addition to the repository checkout. Run `make setup` first and `make report` after regenerating results. The latter also prepares the data-source Mermaid diagrams and SQL listings.

To recreate all six upload-source verification tables and their SQL listings, run `make upload-data-verification-all`, then `make report` to rebuild this report. Fresh extraction requires Snowflake access. To regenerate the tables from retained extracts, use `make upload-data-verification-all CACHED=1` instead; see HOWTO.md for cache requirements and validation details.

DRAFT

2.6 What are the data sources?

2.6.1 Where is the prescription upload data coming from?

The report's upload analyses use `EDLDB.BT_HCA_HCDM.rxp_upload_entity`. Table 14 compares the entries in that source with `rxp_notification_service.upload_entity`, resolved in the `EDLDB` database, by `associated_resource_type`.

This verification covers all rows currently available in each source, without date, CMPU, vet-diet, upload-type, or study-population filters. In Table 14, each physical row counts once; no deduplication or joins are applied. These source counts therefore describe the source tables rather than the report's study cohort. They do not establish record-level equivalence between the two sources or document their upstream lineage.

Table 14: Upload-source entries and entries with a non-zero prescription ID, by associated resource type.

	`EDLDB.BT_HCA_HCDM.rxp_upload_entity`		`rxp_notification_service.upload_entity`	
`associated_resource_type`	All entries	With Rx ID	All entries	With Rx ID
DIET_PRESCRIPTION	176,401	176,401	179,262	179,260
DRUG_PRESCRIPTION	39,941	39,941	40,516	40,516
[SQL NULL]	1	0	40,452	0
Total	216,343	216,342	260,230	219,776

Extraction: 2026-11-02 16:02:32 UTC.

Notes: Rows include every resource type present in either source. A zero means that the source has no entries for that type. `[SQL NULL]` denotes a missing resource type, separately from any literal string value. Totals are queried independently from each source and reconciled against the sum of its grouped counts before publication. The extraction time above records the snapshot used for this comparison; rerunning the query may change the counts as sources are updated. *With Rx ID* counts entries with at least one populated, non-zero prescription identifier in HCDM `RX_ID` or notification-service `ASSOCIATED_RESOURCE_IDS`. Braces, array delimiters and quotes are removed before checking individual comma-separated IDs; SQL NULL, JSON null, blank, empty-list and numeric-zero values are excluded. An entry counts once regardless of how many IDs it contains. IDs are checked for presence, without joining to the prescription registry. Both count columns reconcile to their separately queried source totals. This table is refreshed independently of the subsequent order analyses; each analysis reports its own extraction time.

SQL for Table 14

The primary query returns the displayed counts. The independent check expands the ID lists and counts distinct upload IDs; these counts reconcile to the physical-row totals in this extract.

Source-entry and prescription-ID counts

```sql
-- All available physical rows; no date, upload-type or study-cohort filters.
-- Total counts are separate scans, not sums of the grouped results.
-- Both ID fields can contain brace-delimited/comma-separated lists; the
-- notification-service field is a JSON array. Count an entry once if any
-- normalized token is populated and is not a numeric zero or JSON null.
WITH hcdm AS (
    SELECT associated_resource_type,
           ARRAY_SIZE(FILTER(
               SPLIT(TRANSLATE(COALESCE(rx_id, ''), '{}[]"', ''), ','),
               token -> LOWER(TRIM(token::VARCHAR)) NOT IN ('', 'null')
                    AND COALESCE(TRY_TO_DOUBLE(TRIM(token::VARCHAR)), 1) <> 0
           )) > 0 AS has_nonzero_rx_id
    FROM EDLDB.BT_HCA_HCDM.rxp_upload_entity
), notification AS (
    SELECT associated_resource_type,
           ARRAY_SIZE(FILTER(
               SPLIT(TRANSLATE(COALESCE(TO_JSON(associated_resource_ids), ''), '{}[]"', ''), ','),
               token -> LOWER(TRIM(token::VARCHAR)) NOT IN ('', 'null')
                    AND COALESCE(TRY_TO_DOUBLE(TRIM(token::VARCHAR)), 1) <> 0
           )) > 0 AS has_nonzero_rx_id
    FROM EDLDB.rxp_notification_service.upload_entity
), counts AS (
    SELECT 'EDLDB.BT_HCA_HCDM.rxp_upload_entity' AS source_table,
           associated_resource_type::VARCHAR AS associated_resource_type,
           COUNT(*) AS entry_count,
           COALESCE(COUNT_IF(has_nonzero_rx_id), 0) AS entries_with_nonzero_rx_id,
           FALSE AS is_total
    FROM hcdm
    GROUP BY associated_resource_type
```

DRAFT

```
    UNION ALL
    SELECT 'EDLDB.BT_HCA_HCDM.rxp_upload_entity', NULL, COUNT(*),
           COALESCE(COUNT_IF(has_nonzero_rx_id), 0), TRUE
    FROM hcdm
    UNION ALL
    SELECT 'EDLDB.rxp_notification_service.upload_entity',
           associated_resource_type::VARCHAR, COUNT(*),
           COALESCE(COUNT_IF(has_nonzero_rx_id), 0), FALSE
    FROM notification
    GROUP BY associated_resource_type
    UNION ALL
    SELECT 'EDLDB.rxp_notification_service.upload_entity', NULL, COUNT(*),
           COALESCE(COUNT_IF(has_nonzero_rx_id), 0), TRUE
    FROM notification
)
SELECT source_table, associated_resource_type, entry_count, entries_with_nonzero_rx_id, is_total,
       TO_CHAR(CONVERT_TIMEZONE('UTC', CURRENT_TIMESTAMP()),
               'YYYY-MM-DD HH24:MI:SS') AS extracted_at_utc
FROM counts
ORDER BY is_total, associated_resource_type NULLS LAST, source_table
```

Independent prescription-ID count check

```
-- Independent FLATTEN-based numerator check. Distinct-upload counts also
-- establish whether upload_id can identify a physical source row here.
WITH RAW AS (
    SELECT 'EDLDB.BT_HCA_HCDM.rxp_upload_entity' AS source_table,
           associated_resource_type, upload_id,
           COALESCE(rx_id, '') AS rx_id_list
    FROM EDLDB.BT_HCA_HCDM.rxp_upload_entity
    UNION ALL
    SELECT 'EDLDB.rxp_notification_service.upload_entity',
           associated_resource_type, upload_id,
           COALESCE(TO_JSON(associated_resource_ids), '')
    FROM EDLDB.rxp_notification_service.upload_entity
), expanded AS (
    SELECT r.*, TRIM(f.value::VARCHAR) AS rx_token
    FROM RAW r,
         LATERAL FLATTEN(INPUT => SPLIT(TRANSLATE(r.rx_id_list, '{}[]"', ''), ','), OUTER => TRUE) f
)
SELECT source_table, associated_resource_type,
       COUNT(DISTINCT upload_id) AS distinct_uploads,
       COUNT(DISTINCT CASE
           WHEN LOWER(rx_token) NOT IN ('', 'null')
            AND COALESCE(TRY_TO_DOUBLE(rx_token), 1) <> 0
           THEN upload_id END) AS entries_with_nonzero_rx_id
FROM expanded
GROUP BY source_table, associated_resource_type
```

Table 15 shows the minimum and maximum creation timestamps of the entries currently available in each source, by resource type and overall. All timestamps are shown in UTC.

Table 15: Creation-time coverage of current upload-source entries (UTC).

	EDLDB.BT_HCA_HCDM. rxp_upload_entity		rxp_notification_service. upload_entity	
Resource type	Minimum created	Maximum created	Minimum created	Maximum created
DIET_PRESCRIPTION	2026-07-30 19:51:39	2026-11-02 04:32:31	2026-07-29 17:23:29	2026-11-02 15:54:36
DRUG_PRESCRIPTION	2026-10-01 14:46:41	2026-11-01 19:32:35	2026-10-01 14:46:41	2026-11-01 19:32:35
[SQL NULL]	2026-11-02 04:27:45	2026-11-02 04:27:45	2026-07-30 19:49:25	2026-11-02 15:54:48
Overall	2026-07-30 19:51:39	2026-11-02 04:32:31	2026-07-29 17:23:29	2026-11-02 15:54:48

Notes: No date, order-ID or prescription-ID filters are applied. HCDM uses `CREATED_TS_UTC`; notification-service `CREATED_AT` is converted to UTC before aggregation. Minimum and maximum ignore missing timestamps. Displayed values are rounded to the nearest second (half seconds round upward); the retained extract preserves full precision. Missing creation timestamps: 0 in HCDM and 0 in the notification-service source. The ranges describe available entries, without establishing continuous coverage or the full history of either system. Extraction: 2026-11-02 16:09:54 UTC.

SQL for Table 15

This query calculates creation-time extrema by resource type and overall. Notification-service timestamps are converted to UTC before aggregation; missing timestamps do not contribute to the minimum or maximum.

Minimum and maximum source creation timestamps in UTC

```sql
-- Creation-time coverage of all current source entries, without date or ID filters.
WITH entries AS (
    SELECT 'EDLDB.BT_HCA_HCDM.rxp_upload_entity' AS source_table,
           associated_resource_type,
           created_ts_utc AS created_utc_ts
    FROM EDLDB.BT_HCA_HCDM.rxp_upload_entity
    UNION ALL
    SELECT 'EDLDB.rxp_notification_service.upload_entity',
           associated_resource_type,
           CONVERT_TIMEZONE('UTC', created_at)::TIMESTAMP_NTZ
    FROM EDLDB.rxp_notification_service.upload_entity
)
SELECT source_table, associated_resource_type,
       GROUPING(associated_resource_type) = 1 AS is_overall,
       COUNT(*) AS entry_count,
       COUNT(created_utc_ts) AS timestamp_count,
       TO_CHAR(MIN(created_utc_ts), 'YYYY-MM-DD HH24:MI:SS.FF9') AS min_created_utc_ts,
       TO_CHAR(MAX(created_utc_ts), 'YYYY-MM-DD HH24:MI:SS.FF9') AS max_created_utc_ts,
       TO_CHAR(CONVERT_TIMEZONE('UTC', CURRENT_TIMESTAMP()),
               'YYYY-MM-DD HH24:MI:SS') AS extracted_at_utc
FROM entries
GROUP BY GROUPING SETS (
    (source_table, associated_resource_type),
    (source_table)
)
ORDER BY is_overall, associated_resource_type NULLS LAST, source_table
```

2.6.1.1 What's the distribution of all the vet diet prescriptions in the prescription upload tables by APPROVAL_CHANNEL ?

Table 16 joins all distinct prescription IDs in each upload source to EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS on rx_id, then retains matched prescriptions with IS_VET_DIET_FLAG = TRUE. Approval channels are taken from the prescription table. Each prescription counts once per source, even when it appears in multiple uploads. No date, CMPU, approval-channel, status or upload-resource-type filters are applied.

Table 16: Approval channels of distinct vet-diet prescriptions found in the upload sources.

Approval channel	EDLDB.BT_HCA_HCDM. rxp_upload_entity Prescriptions (%)	rxp_notification_service. upload_entity Prescriptions (%)	Difference Notification minus HCDM
CUSTOMER_UPLOAD	153,015 (83.96%)	153,083 (83.95%)	+68
PH	20,739 (11.38%)	20,760 (11.38%)	+21
FAX	2,801 (1.54%)	2,809 (1.54%)	+8
CLINIC_EMAIL	2,146 (1.18%)	2,148 (1.18%)	+2
VERBAL	1,784 (0.98%)	1,786 (0.98%)	+2
CUSTOMER_EMAIL	1,389 (0.76%)	1,389 (0.76%)	0
PARTNER	330 (0.18%)	330 (0.18%)	0
RHAPSODY	32 (0.02%)	32 (0.02%)	0
SNAIL_MAIL	9 (0.00%)	9 (0.00%)	0
RE_AUTH	1 (0.00%)	1 (0.00%)	0
Total	182,246 (100.00%)	182,347 (100.00%)	+101

Notes: Prescription IDs are expanded from HCDM RX_ID and notification-service ASSOCIATED_RESOURCE_IDS; missing, blank and zero tokens are excluded before joining. Vet-diet classification comes from the prescription registry, not the upload resource type. Percentages use the matched vet-diet total in each source. The difference is the notification-service count minus the HCDM count. The sources contain 232,111 and 233,285 distinct populated prescription IDs, respectively. Excluded non-vet-diet matches: 49,865 and 50,261; unmatched IDs: 0 and 677; matched IDs with unknown vet-diet flag: 0 and 0. Unmatched IDs cannot be assigned a vet-diet category or approval channel from this registry. NULL approval channels remain in the distribution when present. Both channel counts and join-coverage counts reconcile to independent Snowflake aggregates. Percentages may not sum to 100% because of rounding. Extraction: 2026-11-02 16:23:12 UTC.

SQL for Table 16

The first query links distinct uploaded prescription IDs to the registry. The table renderer retains matched vet-diet prescriptions and counts their approval channels. The second query independently verifies channel counts and the unmatched, non-vet-diet and unknown-category populations.

All upload prescription IDs joined to the prescription registry

```sql
-- All normalized source prescription IDs, joined to the prescription registry.
-- No date, CMPU, approval-channel, approval-status or upload-resource-type filters.
WITH upload_ids AS (
    SELECT DISTINCT 'EDLDB.BT_HCA_HCDM.rxp_upload_entity' AS source_table,
           TRIM(f.value::VARCHAR) AS rx_id
    FROM EDLDB.BT_HCA_HCDM.RXP_UPLOAD_ENTITY u,
         LATERAL FLATTEN(INPUT => SPLIT(TRANSLATE(COALESCE(u.rx_id, ''), '{}[]"', ''), ',')) f
    WHERE LOWER(TRIM(f.value::VARCHAR)) NOT IN ('', 'null')
      AND COALESCE(TRY_TO_DOUBLE(TRIM(f.value::VARCHAR)), 1) <> 0
    UNION
    SELECT 'EDLDB.rxp_notification_service.upload_entity',
           TRIM(f.value::VARCHAR)
    FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY u,
         LATERAL FLATTEN(INPUT => SPLIT(
             TRANSLATE(COALESCE(TO_JSON(u.associated_resource_ids), ''), '{}[]"', ''), ','
         )) f
    WHERE LOWER(TRIM(f.value::VARCHAR)) NOT IN ('', 'null')
      AND COALESCE(TRY_TO_DOUBLE(TRIM(f.value::VARCHAR)), 1) <> 0
)
SELECT TO_CHAR(CONVERT_TIMEZONE('UTC', CURRENT_TIMESTAMP()),
               'YYYY-MM-DD HH24:MI:SS') AS extracted_at_utc,
       u.source_table, u.rx_id,
       p.rx_id AS matched_rx_id,
       p.is_vet_diet_flag, p.approval_channel
FROM upload_ids u
LEFT JOIN EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS p ON u.rx_id = TRIM(p.rx_id)
```

DRAFT

Independent approval-channel and join-coverage aggregation

```sql
-- Independent warehouse counts by registry match/category and approval channel.
WITH upload_ids AS (
    SELECT DISTINCT 'EDLDB.BT_HCA_HCDM.rxp_upload_entity' AS source_table,
           TRIM(f.value::VARCHAR) AS rx_id
    FROM EDLDB.BT_HCA_HCDM.RXP_UPLOAD_ENTITY u,
         LATERAL FLATTEN(INPUT => SPLIT(TRANSLATE(COALESCE(u.rx_id, ''), '{}[]"', ''), ',')) f
    WHERE LOWER(TRIM(f.value::VARCHAR)) NOT IN ('', 'null')
      AND COALESCE(TRY_TO_DOUBLE(TRIM(f.value::VARCHAR)), 1) <> 0
    UNION
    SELECT 'EDLDB.rxp_notification_service.upload_entity',
           TRIM(f.value::VARCHAR)
    FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY u,
         LATERAL FLATTEN(INPUT => SPLIT(
             TRANSLATE(COALESCE(TO_JSON(u.associated_resource_ids), ''), '{}[]"', ''), ','
         )) f
    WHERE LOWER(TRIM(f.value::VARCHAR)) NOT IN ('', 'null')
      AND COALESCE(TRY_TO_DOUBLE(TRIM(f.value::VARCHAR)), 1) <> 0
)
SELECT u.source_table,
       CASE WHEN p.rx_id IS NULL THEN 'Unmatched RX_ID'
            WHEN p.is_vet_diet_flag = TRUE THEN 'Vet diet'
            WHEN p.is_vet_diet_flag = FALSE THEN 'Non-vet diet'
            ELSE 'Unknown vet-diet flag' END AS match_category,
       CASE WHEN p.is_vet_diet_flag = TRUE THEN p.approval_channel END AS approval_channel,
       COUNT(*) AS joined_rows,
       COUNT(DISTINCT u.rx_id) AS prescriptions
FROM upload_ids u
LEFT JOIN EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS p ON u.rx_id = TRIM(p.rx_id)
GROUP BY u.source_table, match_category,
         CASE WHEN p.is_vet_diet_flag = TRUE THEN p.approval_channel END
```

DRAFT

2.6.1.2 What is the coverage of vet diet prescriptions with APPROVAL_CHANNEL = CUSTOMER_UPLOAD, by PRESCRIPTION_SOURCE and RX_STATUS?

Table 17 starts from all vet-diet prescriptions in EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS with IS_VET_DIET_FLAG = TRUE and APPROVAL_CHANNEL = CUSTOMER_UPLOAD, created on or after July 30, 2026, 00:00 UTC, and shows upload-source coverage by PRESCRIPTION_SOURCE and current RX_STATUS. All statuses and prescription sources are included; there is no upper creation-date bound. Each qualifying prescription counts once.

Table 17: Upload-source coverage by prescription source and current status for vet-diet customer-upload prescriptions created on or after July 30, 2026.

Prescription source	RX_STATUS	Eligible	HCDM (%)	Notification (%)	Both (%)	HCDM only	Notification only	Neither (%)
CMPU	SYSTEM_CLOSED	109,508	9,483 (8.66%)	9,483 (8.66%)	9,483 (8.66%)	0	0	100,025 (91.34%)
CMPU	APPROVED	50,009	31,660 (63.31%)	31,660 (63.31%)	31,660 (63.31%)	0	0	18,349 (36.69%)
CMPU	VET_REVIEW	8,549	393 (4.60%)	496 (5.80%)	393 (4.60%)	0	103	8,053 (94.20%)
CMPU	CLOSED	2,533	1,244 (49.11%)	1,244 (49.11%)	1,244 (49.11%)	0	0	1,289 (50.89%)
CMPU	DENIED	1	0 (0.00%)	0 (0.00%)	0 (0.00%)	0	0	1 (100.00%)
CMPU	All statuses	170,600	42,780 (25.08%)	42,883 (25.14%)	42,780 (25.08%)	0	103	127,717 (74.86%)
CUSTOMER_UPLOAD	APPROVED	98,296	56,423 (57.40%)	56,424 (57.40%)	56,423 (57.40%)	0	1	41,872 (42.60%)
CUSTOMER_UPLOAD	SYSTEM_CLOSED	13,241	13,164 (99.42%)	13,164 (99.42%)	13,164 (99.42%)	0	0	77 (0.58%)
CUSTOMER_UPLOAD	CLOSED	4,157	1,829 (44.00%)	1,829 (44.00%)	1,829 (44.00%)	0	0	2,328 (56.00%)
CUSTOMER_UPLOAD	VET_REVIEW	1,165	923 (79.23%)	934 (80.17%)	923 (79.23%)	0	11	231 (19.83%)
CUSTOMER_UPLOAD	All statuses	116,859	72,339 (61.90%)	72,351 (61.91%)	72,339 (61.90%)	0	12	44,508 (38.09%)
PRACTICE_HUB	APPROVED	33,901	26,792 (79.03%)	26,792 (79.03%)	26,792 (79.03%)	0	0	7,109 (20.97%)
PRACTICE_HUB	CLOSED	1,922	1,314 (68.37%)	1,314 (68.37%)	1,314 (68.37%)	0	0	608 (31.63%)
PRACTICE_HUB	SYSTEM_CLOSED	770	39 (5.06%)	39 (5.06%)	39 (5.06%)	0	0	731 (94.94%)
PRACTICE_HUB	VET_REVIEW	2	2 (100.00%)	2 (100.00%)	2 (100.00%)	0	0	0 (0.00%)
PRACTICE_HUB	All statuses	36,595	28,147 (76.91%)	28,147 (76.91%)	28,147 (76.91%)	0	0	8,448 (23.09%)
FAX	APPROVED	5,717	4,295 (75.13%)	4,295 (75.13%)	4,295 (75.13%)	0	0	1,422 (24.87%)
FAX	CLOSED	434	292 (67.28%)	292 (67.28%)	292 (67.28%)	0	0	142 (32.72%)
FAX	SYSTEM_CLOSED	378	35 (9.26%)	35 (9.26%)	35 (9.26%)	0	0	343 (90.74%)
FAX	DENIED	4	0 (0.00%)	0 (0.00%)	0 (0.00%)	0	0	4 (100.00%)
FAX	VET_REVIEW	3	0 (0.00%)	0 (0.00%)	0 (0.00%)	0	0	3 (100.00%)
FAX	All statuses	6,536	4,622 (70.72%)	4,622 (70.72%)	4,622 (70.72%)	0	0	1,914 (29.28%)
CLINIC_EMAIL	APPROVED	5,695	4,282 (75.19%)	4,282 (75.19%)	4,282 (75.19%)	0	0	1,413 (24.81%)
CLINIC_EMAIL	CLOSED	467	302 (64.67%)	302 (64.67%)	302 (64.67%)	0	0	165 (35.33%)
CLINIC_EMAIL	SYSTEM_CLOSED	281	19 (6.76%)	19 (6.76%)	19 (6.76%)	0	0	262 (93.24%)
CLINIC_EMAIL	DENIED	1	0 (0.00%)	0 (0.00%)	0 (0.00%)	0	0	1 (100.00%)
CLINIC_EMAIL	All statuses	6,444	4,603 (71.43%)	4,603 (71.43%)	4,603 (71.43%)	0	0	1,841 (28.57%)
PARTNER_API	APPROVED	421	336 (79.81%)	336 (79.81%)	336 (79.81%)	0	0	85 (20.19%)
PARTNER_API	CLOSED	19	11 (57.89%)	11 (57.89%)	11 (57.89%)	0	0	8 (42.11%)
PARTNER_API	SYSTEM_CLOSED	10	0 (0.00%)	0 (0.00%)	0 (0.00%)	0	0	10 (100.00%)
PARTNER_API	All statuses	450	347 (77.11%)	347 (77.11%)	347 (77.11%)	0	0	103 (22.89%)
DIGITAL	SYSTEM_CLOSED	141	31 (21.99%)	31 (21.99%)	31 (21.99%)	0	0	110 (78.01%)
DIGITAL	APPROVED	128	96 (75.00%)	96 (75.00%)	96 (75.00%)	0	0	32 (25.00%)
DIGITAL	VET_REVIEW	6	0 (0.00%)	0 (0.00%)	0 (0.00%)	0	0	6 (100.00%)
DIGITAL	CLOSED	3	2 (66.67%)	2 (66.67%)	2 (66.67%)	0	0	1 (33.33%)
DIGITAL	All statuses	278	129 (46.40%)	129 (46.40%)	129 (46.40%)	0	0	149 (53.60%)
CVC	APPROVED	50	41 (82.00%)	41 (82.00%)	41 (82.00%)	0	0	9 (18.00%)
CVC	CLOSED	1	0 (0.00%)	0 (0.00%)	0 (0.00%)	0	0	1 (100.00%)
CVC	All statuses	51	41 (80.39%)	41 (80.39%)	41 (80.39%)	0	0	10 (19.61%)
RHAPSODY	APPROVED	6	5 (83.33%)	5 (83.33%)	5 (83.33%)	0	0	1 (16.67%)
RHAPSODY	CLOSED	1	1 (100.00%)	1 (100.00%)	1 (100.00%)	0	0	0 (0.00%)
RHAPSODY	All statuses	7	6 (85.71%)	6 (85.71%)	6 (85.71%)	0	0	1 (14.29%)
All sources	All statuses	337,820	153,014 (45.29%)	153,129 (45.33%)	153,014 (45.29%)	0	115	184,691 (54.67%)

DRAFT

Notes: Coverage requires at least one current upload-source entry with the same `rx_id`. HCDM `RX_ID` and notification-service `ASSOCIATED_RESOURCE_IDS` are split into individual trimmed IDs; NULL, blank and zero tokens are excluded. Each upload source is reduced to a distinct set of prescription IDs before joining, so multiple uploads do not increase the prescription count. The join uses `rx_id` only, without customer, order, upload-date or associated-resource-type restrictions. HCDM and Notification count any match in that source and overlap. Within each prescription-source/status group, Both, HCDM only, Notification only and Neither partition the eligible prescriptions. Percentages use the eligible count on that row. All statuses rows are prescription-source subtotals; All sources / All statuses is the overall total. Do not add subtotals to their detail rows. The two only columns show counts. SQL NULL source or status is retained when present. Percentages may not sum to 100% after rounding. No current match does not establish that an upload never existed. The row-level joins and independent Snowflake aggregates agree for every source/status group, source subtotal, status subtotal and overall. Extraction: 2026-11-02 17:49:55 UTC.

SQL for Table 17

The first query returns one record per qualifying prescription, joined to distinct upload-source prescription IDs on prescription ID only. The second query independently counts upload-source membership and overlap by prescription source and current status, including source subtotals, status subtotals and overall.

Vet-diet customer-upload prescription joins by prescription source and status

```sql
-- One record per qualifying vet-diet prescription, retaining every current RX_STATUS.
-- July 30, 2026 is inclusive at 00:00 UTC; there is no upper creation-date bound.
WITH prescriptions AS (
    SELECT TRIM(rx_id) AS rx_id, rx_creation_datetime, rx_status, prescription_source,
           COUNT(*) OVER (PARTITION BY TRIM(rx_id)) AS prescription_source_rows
    FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS
    WHERE approval_channel = 'CUSTOMER_UPLOAD'
      AND is_vet_diet_flag = TRUE
      AND rx_creation_datetime >= '2026-07-30 00:00:00'::TIMESTAMP_NTZ
), hcdm_ids AS (
    SELECT DISTINCT TRIM(f.value::VARCHAR) AS rx_id
    FROM EDLDB.BT_HCA_HCDM.RXP_UPLOAD_ENTITY u,
         LATERAL FLATTEN(INPUT => SPLIT(TRANSLATE(COALESCE(u.rx_id, ''), '{}[]"', ''), ',')) f
    WHERE LOWER(TRIM(f.value::VARCHAR)) NOT IN ('', 'null')
      AND COALESCE(TRY_TO_DOUBLE(TRIM(f.value::VARCHAR)), 1) <> 0
), notification_ids AS (
    SELECT DISTINCT TRIM(f.value::VARCHAR) AS rx_id
    FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY u,
         LATERAL FLATTEN(INPUT => SPLIT(
             TRANSLATE(COALESCE(TO_JSON(u.associated_resource_ids), ''), '{}[]"', ''), ','
         )) f
    WHERE LOWER(TRIM(f.value::VARCHAR)) NOT IN ('', 'null')
      AND COALESCE(TRY_TO_DOUBLE(TRIM(f.value::VARCHAR)), 1) <> 0
)
SELECT TO_CHAR(CONVERT_TIMEZONE('UTC', CURRENT_TIMESTAMP()),
               'YYYY-MM-DD HH24:MI:SS') AS extracted_at_utc,
       p.rx_id, p.prescription_source, p.rx_status,
       p.rx_creation_datetime::VARCHAR AS rx_creation_datetime,
       p.prescription_source_rows,
       h.rx_id IS NOT NULL AS has_hcdm_upload,
       n.rx_id IS NOT NULL AS has_notification_upload
FROM prescriptions p
LEFT JOIN hcdm_ids h ON p.rx_id = h.rx_id
LEFT JOIN notification_ids n ON p.rx_id = n.rx_id
```

Independent prescription-ID membership aggregation by source and status

```sql
-- Independent vet-diet membership aggregation by status; rx_id-only upload joins.
WITH prescriptions AS (
    SELECT TRIM(rx_id) AS rx_id, rx_status, prescription_source
    FROM EDLDB.BT_HCA_HCDM.RXP_PRESCRIPTIONS
    WHERE approval_channel = 'CUSTOMER_UPLOAD'
      AND is_vet_diet_flag = TRUE
      AND rx_creation_datetime >= '2026-07-30 00:00:00'::TIMESTAMP_NTZ
), hcdm_ids AS (
    SELECT DISTINCT TRIM(f.value::VARCHAR) AS rx_id
    FROM EDLDB.BT_HCA_HCDM.RXP_UPLOAD_ENTITY u,
         LATERAL FLATTEN(INPUT => SPLIT(TRANSLATE(COALESCE(u.rx_id, ''), '{}[]"', ''), ',')) f
    WHERE LOWER(TRIM(f.value::VARCHAR)) NOT IN ('', 'null')
      AND COALESCE(TRY_TO_DOUBLE(TRIM(f.value::VARCHAR)), 1) <> 0
), notification_ids AS (
```

```
    SELECT DISTINCT TRIM(f.value::VARCHAR) AS rx_id
    FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY u,
         LATERAL FLATTEN(INPUT => SPLIT(
             TRANSLATE(COALESCE(TO_JSON(u.associated_resource_ids), ''), '{}[]"', ''), ','
         )) f
    WHERE LOWER(TRIM(f.value::VARCHAR)) NOT IN ('', 'null')
      AND COALESCE(TRY_TO_DOUBLE(TRIM(f.value::VARCHAR)), 1) <> 0
), membership AS (
    SELECT p.rx_id, p.rx_status, p.prescription_source,
           h.rx_id IS NOT NULL AS in_hcdm,
           n.rx_id IS NOT NULL AS in_notification
    FROM prescriptions p
    LEFT JOIN hcdm_ids h ON p.rx_id = h.rx_id
    LEFT JOIN notification_ids n ON p.rx_id = n.rx_id
)
SELECT prescription_source, rx_status,
       GROUPING(prescription_source) = 1 AS is_source_rollup,
       GROUPING(rx_status) = 1 AS is_status_rollup,
       GROUPING(prescription_source) = 1 AND GROUPING(rx_status) = 1 AS is_overall,
       COUNT(*) AS source_rows,
       COUNT(DISTINCT rx_id) AS eligible_prescriptions,
       COUNT_IF(rx_id IS NULL OR rx_id = '') AS missing_rx_ids,
       COUNT(DISTINCT CASE WHEN in_hcdm THEN rx_id END) AS any_hcdm,
       COUNT(DISTINCT CASE WHEN in_notification THEN rx_id END) AS any_notification,
       COUNT(DISTINCT CASE WHEN in_hcdm AND in_notification THEN rx_id END) AS both_sources,
       COUNT(DISTINCT CASE WHEN in_hcdm AND NOT in_notification THEN rx_id END) AS hcdm_only,
       COUNT(DISTINCT CASE WHEN NOT in_hcdm AND in_notification THEN rx_id END) AS notification_only,
       COUNT(DISTINCT CASE WHEN NOT in_hcdm AND NOT in_notification THEN rx_id END) AS neither_source
FROM membership
GROUP BY GROUPING SETS (
    (prescription_source, rx_status),
    (prescription_source),
    (rx_status),
    ()
)
```

DRAFT

2.6.1.3 What is the distribution of order cancellation reasons in the prescription upload tables?

This comparison includes entries with `associated_resource_type = DIET_PRESCRIPTION` and a non-null associated order ID in each source. HCDM uses `ORDER_ID`; the notification-service source uses `context:orderId`. Cancellation reasons are joined from `EDLDB.ECOM.ORDERS`. No date, CMPU, study-cohort, order-item, upload-type, or replication filters are applied. In HCDM, 137,431 of 176,401 diet-upload entries have an order ID, representing 115,929 distinct orders. In the notification-service source, 140,104 of 179,177 entries have an order ID, representing 116,223 distinct orders. Table 18 compares the cancellation reasons side by side. Each order counts once within each source regardless of the number of uploads.

Table 18: Order cancellation reasons for diet-prescription uploads with associated order IDs.

Order cancellation reason	EDLDB.BT_HCA_HCDM. rxp_upload_entity Orders (%)	rxp_notification_service. upload_entity Orders (%)	Difference Notification minus HCDM
No recorded reason (SQL NULL)	102,797 (88.67%)	102,937 (88.57%)	+140
Customer Initiated Cancellation	4,832 (4.17%)	4,833 (4.16%)	+1
Customer Request	4,320 (3.73%)	4,320 (3.72%)	0
Aged Order	1,938 (1.67%)	1,947 (1.68%)	+9
Pharmacy Rejection	1,078 (0.93%)	1,078 (0.93%)	0
Prescription Issue	555 (0.48%)	556 (0.48%)	+1
Other	267 (0.23%)	267 (0.23%)	0
No matching ECOM order	0 (0.00%)	143 (0.12%)	+143
Duplicate Order	52 (0.04%)	52 (0.04%)	0
No Response	47 (0.04%)	47 (0.04%)	0
Payment Declined	26 (0.02%)	26 (0.02%)	0
Fraud	14 (0.01%)	14 (0.01%)	0
Payment Issue	2 (0.00%)	2 (0.00%)	0
Reseller	1 (0.00%)	1 (0.00%)	0
Total	115,929 (100.00%)	116,223 (100.00%)	+294

Notes: Percentages use each source's distinct-order total, and the difference is the notification-service order count minus the HCDM order count. Totals include unmatched IDs; the two source populations are not restricted to their intersection. 115,929 order IDs occur in both sources, 0 occur only in HCDM and 294 occur only in the notification-service source. Unmatched order IDs: 0 in HCDM and 143 in the notification-service source. A NULL reason does not imply that an order was not canceled. Of 13,266 HCDM-linked orders and 13,277 notification-linked orders with status `Canceled`, 134 and 134, respectively, have a NULL reason. Percentages may not sum to 100% because of rounding. Upload extraction: 2026-11-02 15:45:52.376 Z; order extraction: 2026-11-02 15:45:54.406 Z (UTC).

SQL for Table 18

The first two queries extract the source records used by the table renderer. The final query independently aggregates the counts in Snowflake. Orders count once per source; the table renderer also calculates percentages and, for the source comparison, the count difference.

Diet uploads and their associated order IDs

```sql
-- All DIET_PRESCRIPTION physical source rows; keep missing order IDs for coverage.
SELECT CURRENT_TIMESTAMP()::VARCHAR AS extracted_at_session_time,
       'EDLDB.BT_HCA_HCDM.rxp_upload_entity' AS source_table,
       upload_id, order_id AS order_id_text
FROM EDLDB.BT_HCA_HCDM.RXP_UPLOAD_ENTITY
WHERE associated_resource_type = 'DIET_PRESCRIPTION'
UNION ALL
SELECT CURRENT_TIMESTAMP()::VARCHAR,
       'EDLDB.rxp_notification_service.upload_entity',
       upload_id,
       IFF(IS_NULL_VALUE(context:orderId), NULL, context:orderId::VARCHAR)
FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY
WHERE associated_resource_type = 'DIET_PRESCRIPTION'
```

ECOM order records for linked diet uploads

```sql
-- Current ECOM order records for numeric order IDs associated with diet uploads.
-- No date, order status, order-item, replication, or study-cohort filters.
WITH ids AS (
    SELECT TRY_TO_NUMBER(TRIM(order_id)) AS order_id
    FROM EDLDB.BT_HCA_HCDM.RXP_UPLOAD_ENTITY
    WHERE associated_resource_type = 'DIET_PRESCRIPTION'
      AND REGEXP_LIKE(TRIM(order_id), '^[0-9]+$')
```

DRAFT

```
    UNION
    SELECT TRY_TO_NUMBER(TRIM(context:orderId::VARCHAR))
    FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY
    WHERE associated_resource_type = 'DIET_PRESCRIPTION'
      AND REGEXP_LIKE(TRIM(context:orderId::VARCHAR), '^[0-9]+$')
)
SELECT CURRENT_TIMESTAMP()::VARCHAR AS extracted_at_session_time,
       o.order_id, o.order_cancel_reason, o.order_status_description
FROM EDLDB.ECOM.ORDERS o
JOIN ids i ON o.order_id = i.order_id
```

Independent source-by-reason aggregation

```
-- Independent warehouse aggregation, preserving distinct-order and upload grains.
WITH RAW AS (
    SELECT 'EDLDB.BT_HCA_HCDM.rxp_upload_entity' AS source_table,
           order_id AS order_id_text
    FROM EDLDB.BT_HCA_HCDM.RXP_UPLOAD_ENTITY
    WHERE associated_resource_type = 'DIET_PRESCRIPTION'
    UNION ALL
    SELECT 'EDLDB.rxp_notification_service.upload_entity',
           IFF(IS_NULL_VALUE(context:orderId), NULL, context:orderId::VARCHAR)
    FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY
    WHERE associated_resource_type = 'DIET_PRESCRIPTION'
), coverage AS (
    SELECT source_table, COUNT(*) AS diet_uploads,
           COUNT_IF(order_id_text IS NOT NULL) AS uploads_with_order_id
    FROM RAW GROUP BY source_table
), ids AS (
    SELECT source_table, TRY_TO_NUMBER(TRIM(order_id_text)) AS order_id,
           COUNT(*) AS upload_entries
    FROM RAW
    WHERE REGEXP_LIKE(TRIM(order_id_text), '^[0-9]+$')
    GROUP BY source_table, TRY_TO_NUMBER(TRIM(order_id_text))
)
SELECT i.source_table, c.diet_uploads, c.uploads_with_order_id,
       o.order_id IS NOT NULL AS matched,
       o.order_cancel_reason,
       COUNT(*) AS distinct_orders, SUM(i.upload_entries) AS upload_entries,
       COUNT_IF(o.order_status_description = 'Canceled') AS canceled_orders,
       COUNT_IF(o.order_status_description = 'Canceled' AND o.order_cancel_reason IS NULL) AS canceled_with_null_reason
FROM ids i
LEFT JOIN EDLDB.ECOM.ORDERS o ON o.order_id = i.order_id
JOIN coverage c ON i.source_table = c.source_table
GROUP BY i.source_table, c.diet_uploads, c.uploads_with_order_id,
         o.order_id IS NOT NULL, o.order_cancel_reason
```

2.6.1.4 What are the order cancellation reasons for null associated resource type entries in the rxp_notification_service.upload_entity table?

Of the 40,451 notification-service upload entries with `associated_resource_type IS NULL`, 1,998 (4.94%) have both a non-null `context:orderId` and `context:orderItemId`; 38,453 have neither. These entries reference 1,726 distinct orders, of which 1,724 match `EDLDB.ECOM.ORDERS` and 2 are unmatched. Table 19 shows the cancellation-reason distribution across these distinct order IDs.

Table 19: Cancellation reasons for orders referenced by uploads with no associated resource type.

Order cancellation reason	Distinct orders	Share (%)
No recorded reason (SQL NULL)	1,587	91.95
Aged Order	67	3.88
Customer Initiated Cancellation	43	2.49
Customer Request	15	0.87
Prescription Issue	7	0.41
Pharmacy Rejection	3	0.17
No matching ECOM order	2	0.12
Fraud	1	0.06
Other	1	0.06
Total	1,726	100.00

Notes: All available NULL-type uploads are included, without date or other filters. Missing JSON keys and JSON null count as missing identifiers. Each order counts once regardless of upload count; percentages use all 1,726 distinct order IDs, including unmatched IDs. A NULL cancellation reason does not imply that the order was not canceled. Of 139 matched orders with status `Canceled`, 2 have a NULL reason and 67 (48.20%) have reason `Aged Order`. Percentages may not sum to 100% because of rounding. Upload extraction: 2026-11-02 15:35:14.335 Z; order extraction: 2026-11-02 15:35:15.577 Z (UTC).

SQL for Table 19

The first two queries extract the source records used by the table renderer. The final query independently aggregates the counts in Snowflake. Orders count once per source; the table renderer also calculates percentages and, for the source comparison, the count difference.

NULL-type uploads and their context identifiers

```sql
-- All available physical uploads with SQL NULL associated_resource_type.
-- JSON null is missing, not the literal string 'null'; preserve blank strings.
SELECT CURRENT_TIMESTAMP()::VARCHAR AS extracted_at_session_time,
       upload_id,
       TYPEOF(context) AS context_type,
       TYPEOF(context:orderId) AS order_id_value_type,
       TYPEOF(context:orderItemId) AS order_item_id_value_type,
       IFF(IS_NULL_VALUE(context:orderId), NULL, context:orderId::VARCHAR) AS context_order_id,
       IFF(IS_NULL_VALUE(context:orderItemId), NULL, context:orderItemId::VARCHAR) AS context_order_item_id,
       replication_deleted_flag,
       created_at::VARCHAR AS upload_created_at
FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY
WHERE associated_resource_type IS NULL
```

ECOM order records linked to NULL-type uploads

```sql
-- Order-level source extract for every numeric CONTEXT orderId in the NULL-type population.
-- No date, cancellation-status, upload-type, item-ID, or deleted-record filters.
WITH ids AS (
    SELECT DISTINCT
           TRY_TO_NUMBER(TRIM(context:orderId::VARCHAR)) AS order_id
    FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY
    WHERE associated_resource_type IS NULL
      AND REGEXP_LIKE(TRIM(context:orderId::VARCHAR), '^[0-9]+$')
)
SELECT CURRENT_TIMESTAMP()::VARCHAR AS extracted_at_session_time,
       o.order_id,
       o.order_cancel_reason,
       o.order_status,
       o.order_status_description,
       o.order_cancel_dttm::VARCHAR AS order_cancel_dttm
FROM EDLDB.ECOM.ORDERS o
JOIN ids i ON o.order_id = i.order_id
```

DRAFT

Independent identifier and reason aggregation

```sql
-- Independent warehouse aggregation of the identifier and order/reason counts.
WITH uploads AS (
    SELECT upload_id,
           IFF(IS_NULL_VALUE(context:orderId), NULL, context:orderId::VARCHAR) AS order_id_text,
           IFF(IS_NULL_VALUE(context:orderItemId), NULL, context:orderItemId::VARCHAR) AS item_id_text
    FROM EDLDB.RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY
    WHERE associated_resource_type IS NULL
), ids AS (
    SELECT upload_id, TRY_TO_NUMBER(TRIM(order_id_text)) AS order_id
    FROM uploads
    WHERE REGEXP_LIKE(TRIM(order_id_text), '^[0-9]+$')
)
SELECT (SELECT COUNT(*) FROM uploads) AS null_type_uploads,
       (SELECT COUNT_IF(order_id_text IS NOT NULL) FROM uploads) AS order_id_present,
       (SELECT COUNT_IF(item_id_text IS NOT NULL) FROM uploads) AS item_id_present,
       (SELECT COUNT_IF(order_id_text IS NOT NULL AND item_id_text IS NOT NULL) FROM uploads) AS both_ids_present,
       o.order_status_description,
       o.order_cancel_reason,
       o.order_id IS NOT NULL AS matched,
       COUNT(*) AS upload_entries,
       COUNT(DISTINCT i.order_id) AS distinct_orders
FROM ids i
LEFT JOIN EDLDB.ECOM.ORDERS o ON o.order_id = i.order_id
GROUP BY o.order_status_description, o.order_cancel_reason, o.order_id IS NOT NULL
```

DRAFT

2.6.2 How are data sources connected?

The diagrams describe the analytical relationships used throughout this report, including the executive summary's underlying tables and figures. They do not assert database-enforced foreign keys or upstream replication lineage. All physical sources resolve in `EDLDB`; `HCDM` abbreviates `BT_HCA_HCDM`. Blue solid boxes are physical tables or views; amber dashed boxes are derived datasets. Arrows carry source evidence into a result; edge labels state join fields and direction, rather than implying cardinality. The two upload sources are compared independently, without a record-to-record join between them. The notification source is used in verification, while the later study figures use HCDM uploads.

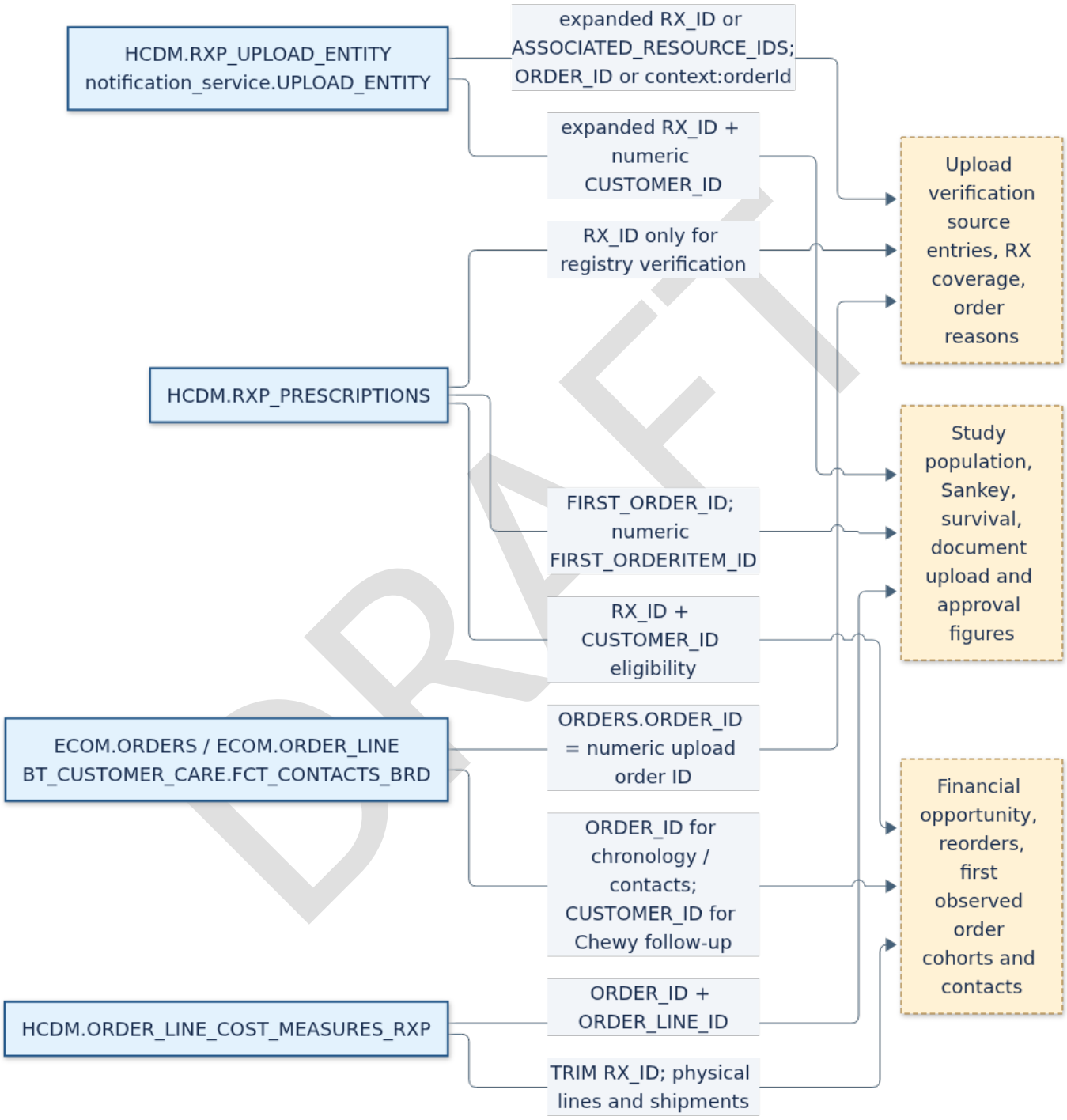


Figure 7: Report-wide source connections; details and matching rules follow.

DRAFT

Source grains and counting units.

Source (within EDLDB)	Grain used by this report
HCDM.RXP_UPLOAD_ENTITY	Physical upload row; entity analyses validate UPLOAD_ID and count it once, despite multiple expanded prescription IDs.
RXP_NOTIFICATION_SERVICE.UPLOAD_ENTITY	Physical upload row; prescription IDs may be a list. Context order and item IDs are separate fields.
HCDM.RXP_PRESCRIPTIONS	Prescription RX_ID; analyses check uniqueness or consistency of repeated registry values.
HCDM.ORDER_LINE_COST_MEASURES_RXP	Shipment × physical line: ORDER_ID, ORDER_LINE_ID, SHIPMENT_KEY. A line can have multiple shipments.
ECOM.ORDERS	Order header ORDER_ID; linked-order verification checks uniqueness, chronology analyses check consistent repeated values.
ECOM.ORDER_LINE	Physical line ORDER_ID, ORDER_LINE_ID; NET_SALES is already the full line amount.
BT_CUSTOMER_CARE.FCT_CONTACTS_BRD	Contact leg/session evidence, identified by order, SESSION_ID and LEG_ID for conflict checks; published counts use distinct customers, orders or relative days.

Duplicates and aggregation. Source-entry counts and timestamp extrema use all physical upload rows without deduplication. Prescription verification expands IDs and reduces them to distinct source/prescription sets, preventing upload multiplicity from multiplying registry counts. Coverage preserves eligible prescriptions with left joins; the channel distribution excludes unmatched and non-vet-diet IDs from its denominator. Order-reason comparisons count each normalized order once per source, retaining unmatched IDs separately from matched orders whose reason is SQL NULL. `context:orderItemId` is audited for presence in the NULL-resource analysis; it is not a join key for those order-level results.

For first-order figures, shipment rows collapse to one (ORDER_ID, ORDER_LINE_ID) before prescription association. Attribute-variant checks prevent conflicting shipment evidence from being silently resolved by SQL MIN/MAX. Multiple prescription associations are retained for audit, then distinct prescriptions, orders or lines are counted according to each output's grain. Missing or invalid first-order keys are audited outside the matched-line denominator. The decision extract uses an inner join because it describes matched first-order populations only.

Financial calculations instead retain one amount per distinct (ORDER_ID, ORDER_LINE_ID, SHIPMENT_KEY) and sum shipments to lines; identical amounts on different shipments remain separate contributions. Repeated associations cannot multiply sales. The baseline includes only shipment or cancellation records in its reporting window. Chewy-wide follow-up uses ECOM.ORDER_LINE.NET_SALES once per line, without summing it as shipment data. Consistent duplicates may collapse; conflicting key values fail the relevant generator's validation. Totals count distinct customers and orders again, rather than adding overlapping category counts.

DRAFT

Upload verification joins. The channel and coverage tables match normalized prescription ID only, with no customer, upload-date or order restriction. The coverage cohort applies its registry creation cutoff and vet-diet/customer-upload filters; the channel comparison covers all uploaded vet-diet prescriptions. Numeric order IDs require trimmed digits before conversion. Source-order extracts are merged in Python with validated ECOM order headers using a left join.

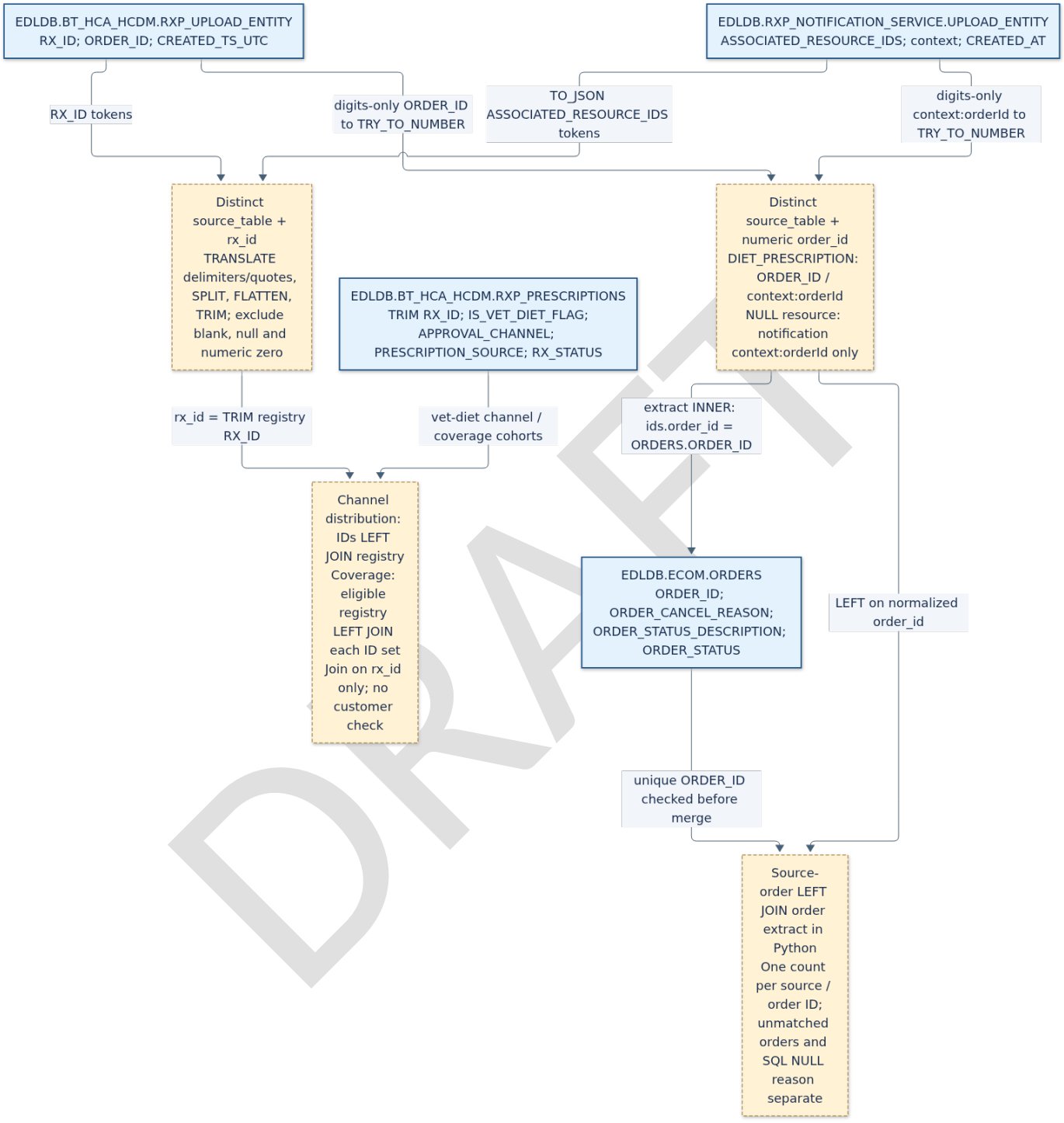


Figure 8: Upload-source prescription expansion, registry matching and context-order matching.

DRAFT

Prescription, first-order and upload evidence. The study population uses CMPU vet-diet prescriptions created August 15 through October 14, 2026, UTC; focused figures also require `APPROVAL_CHANNEL = CUSTOMER_UPLOAD`. Both first-order keys are required: `FIRST_ORDER_ID = ORDER_ID` and the digits-only numeric conversion of `FIRST_ORDERITEM_ID = ORDER_LINE_ID`. Upload matching additionally requires registry `CUSTOMER_ID` to equal the numeric upload customer ID.

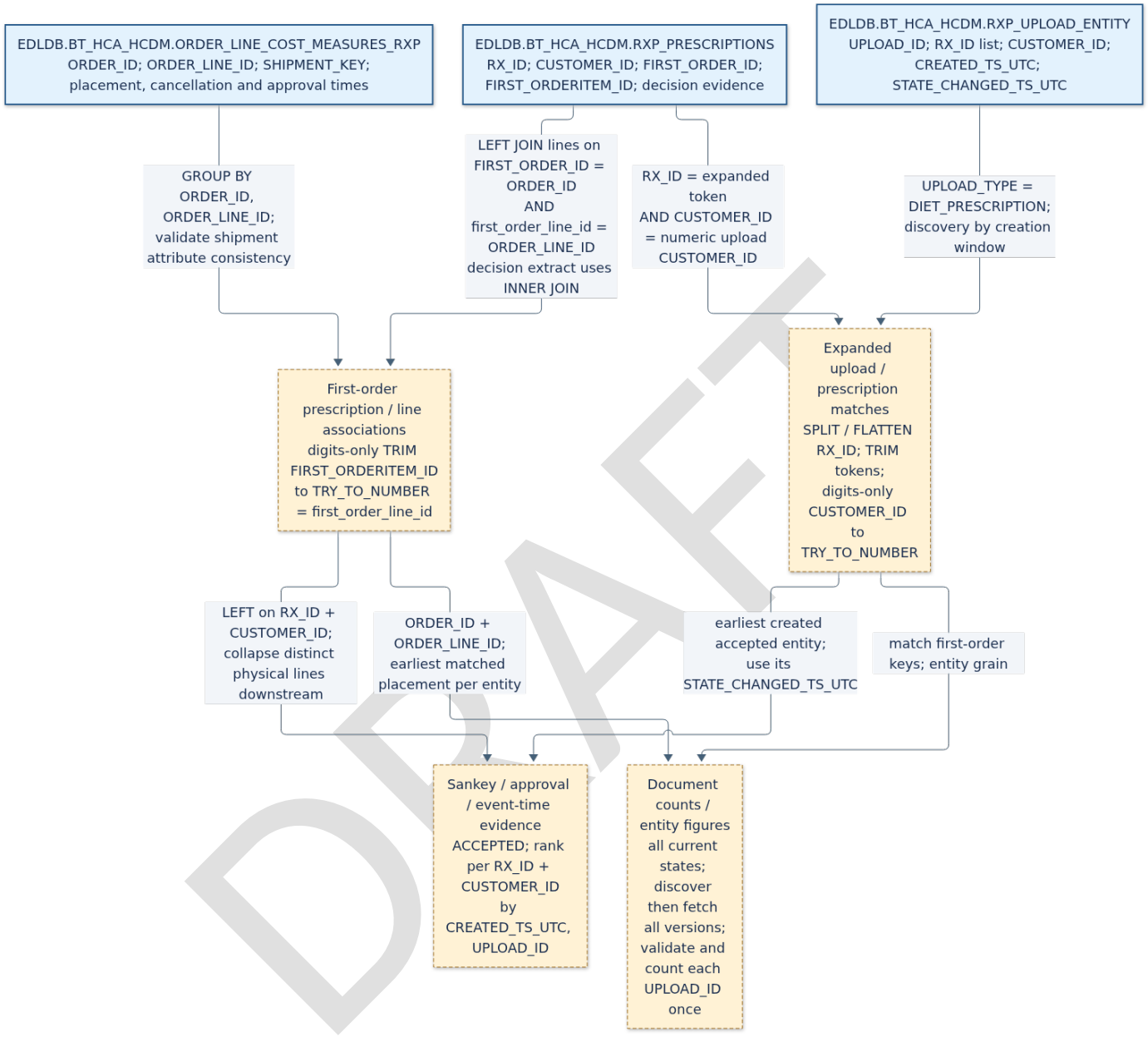


Figure 9: First-order line matching and two distinct ways of selecting upload evidence.

DRAFT

Selection and observation windows. The Sankey, approval and survival pipelines select the earliest-created currently `ACCEPTED` upload per prescription/customer, breaking ties by `UPLOAD_ID`; the measured upload time is that entity's `STATE_CHANGED_TS_UTC`, not its creation time. Survival selects the earliest-created associated entity again at line grain. Approval comparisons use the line's `MAX_APPROVED_DATETIME`; survival uses `MIN_APPROVED_DENIED_DATETIME` for its 96% interval estimate when actual upload time is unavailable, then compares the candidate with cancellation. These estimates are derived events, not extra source tables.

Entity-state figures and document counts retain all current states, discover entities by creation time, fetch all versions of discovered IDs to expose conflicts, and count each entity once. The order-relative entity figure uses its earliest populated matched first-order placement. Upload discovery starts August 1; the Sankey/approval/entity figures use an exclusive October 26 bound, while survival uses an exclusive October 27 bound. Unknown creation times are retained for entity auditing. Current state changes and prescription decisions can occur after the discovery or aged-cancellation cutoff; these are snapshots, not historical event streams.

The study's first-versus-all-order comparison additionally matches `TRIM(line.RX_ID) = prescription.RX_ID` and customer ID for all linked orders, without an order-placement window. Its frozen physical keys are passed to `prescription_cohort_order_sales.sql` for shipment financial enrichment on both order and line ID. This differs from joining only `FIRST_ORDER_ID`. Decision evidence comes from the registry plus the matched line's approval/denial fields; no separate decision table is joined.

Financial, reorder and contact rules. The financial baseline's SQL left-joins the registry on trimmed line `RX_ID`; Python then requires nonmissing, equal numeric customer IDs and the CMPU/vet-diet/customer-upload flags. Aged-anchor queries apply these same customer and prescription checks directly in SQL. The baseline's shipped-or-cancelled window and the anchors' cancellation window are May 1 inclusive to August 1 exclusive, 2026; they do not impose the study's prescription-creation window.

First observed vet-diet and first customer-upload vet-diet cohorts use RxP line history, then ECOM order headers by order and customer ID. Chronology ranks by placement then order ID; earliest-time ties, missing placements and missing headers are audited and excluded from unambiguous first-order cohorts. Vet-diet history uses line vet-diet flags without requiring a registry match; customer-upload history also matches registry prescription/customer IDs and restricts that channel. Both histories span all prescription sources and dates. This is observed history, not a claim of complete lifetime coverage.

DRAFT

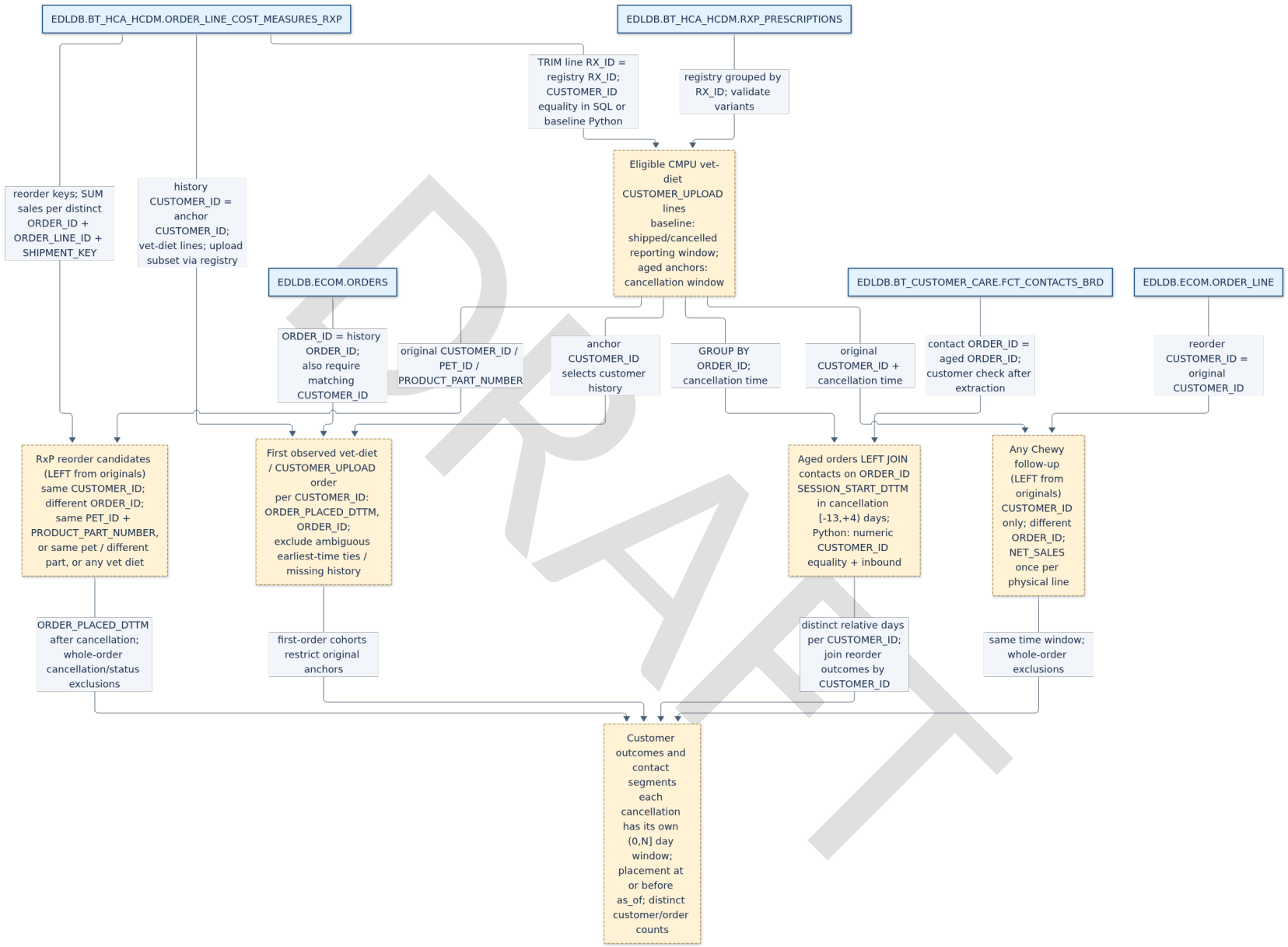


Figure 10: Financial eligibility, order chronology, reorder matching and contact matching.

RxP follow-up matches exact customer, non-null pet and nonblank product part number for the same-product endpoint. Its other endpoints use the same pet with a different part, or any vet-diet line for that customer. Chewy-wide follow-up matches customer only in `ECOM.ORDER_LINE`. All endpoints require a different order placed strictly after its matched cancellation and through the specified follow-up day inclusive, bounded by the October 28, 2026, 00:00 UTC observation cutoff. Windows of 7–70 days overlap; each cancellation is its own anchor. Whole candidate orders with cancellation evidence or unknown line status are excluded. Orders and sales are deduplicated across repeated anchor matches; a returning customer counts once within each endpoint/window.

Contacts join the original aged order on `ORDER_ID` in SQL; Python normalizes identifiers and additionally requires equal nonmissing customer IDs, inbound direction and a populated timestamp. Contact legs are checked for conflicts and exact duplicate rows collapse. Relative days are elapsed UTC 24-hour bins, from $[-13, +4)$ days around cancellation, rather than calendar dates. The report includes handled, unhandled and unknown-handled inbound attempts. Frequency counts distinct relative days across a customer's aged orders. The contact-frequency and -7 through $+3$ contact-window comparisons join customer-level segments to verified 14-day same-pet/part outcomes by `CUSTOMER_ID`, retaining customers without a reorder in the denominator.

Table 6 uses the broader financial shipment/cancellation cohort and the same contact order/customer joins, grouping distinct contacting customers by current reason. No creation-relative day bounds apply; contacts are capped at the latest observed shipment/cancellation event per order and reason. Its SQL and generator are `src/sql/opportunity_customer_contacts_by_cancel_reason.sql` and `src/python/create_opportunity_contacts_by_cancel_reason.py`.

Figure 11 isolates the contact-matching pipeline, including the customer checks applied after the SQL extraction.

DRAFT

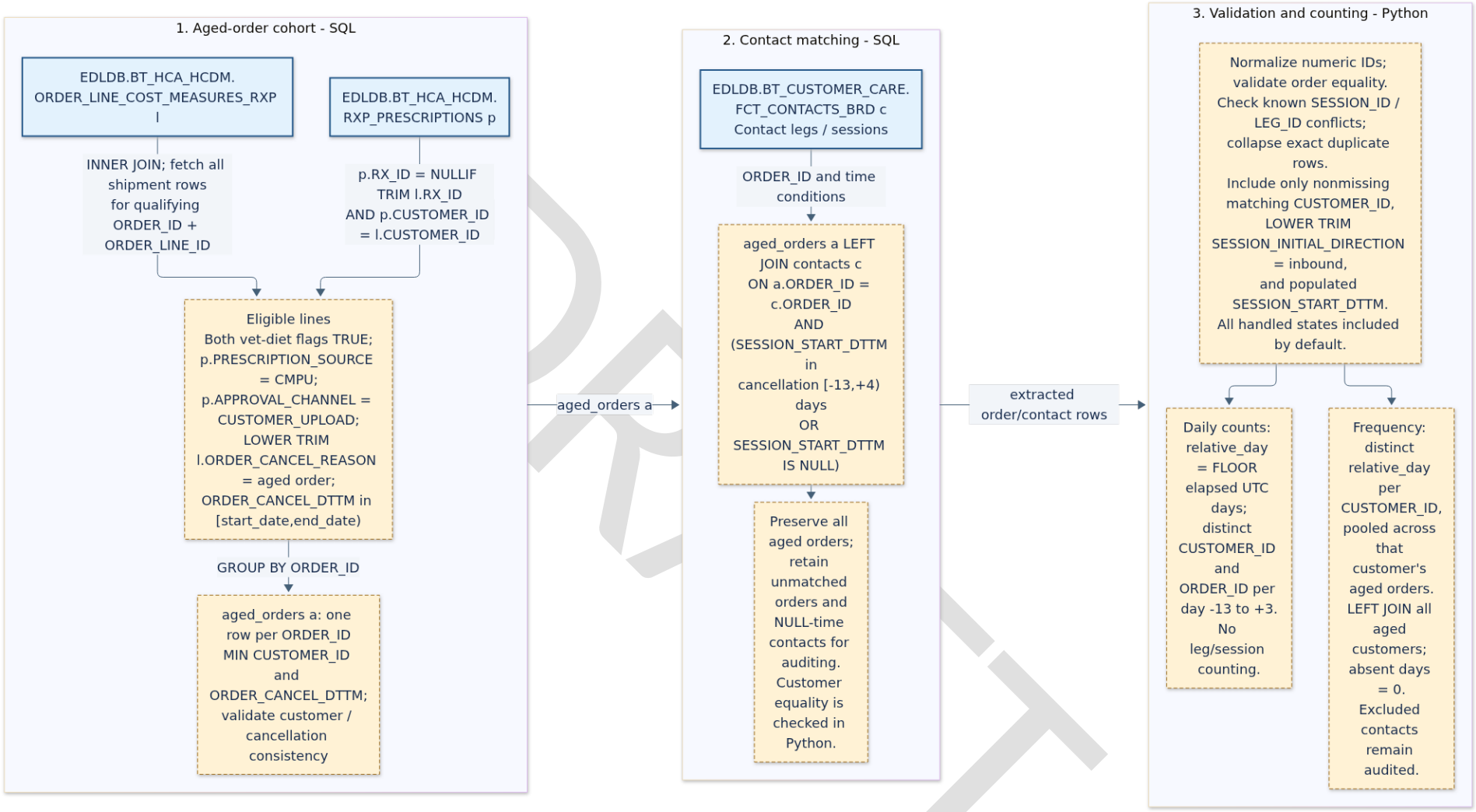


Figure 11: Contact matching: aged-order eligibility, SQL order/time join, Python customer checks and distinct counting.

Reproduction and source trace. Run `make data-source-diagrams` to generate the editable Mermaid sources and transparent PNGs; `make report` also rebuilds missing or outdated diagrams. The tracked definitions are in `src/python/create_data_source_diagrams.py`; the Markdown grain table is `outputs/tables/data-source-connections/source-grains.md`. The following implementation files define the relationships above; independently aggregated verification SQL checks the same populations without adding sources. Unused exploratory queries are excluded from this map.

- Verification: `src/sql/upload_data_verification.sql`, `upload_source_time_coverage.sql`, `upload_vet_diet_approval_channels.sql`, `cmpu_approved_prescription_upload_coverage.sql`, `diet_upload_source_orders.sql`, `diet_upload_linked_orders.sql`, `null_resource_upload_context.sql` and `null_resource_upload_orders.sql` (all under `src/sql/`); their corresponding `src/python/analyze_*.py` modules validate and merge the retained extracts.
- Study figures: `src/sql/cmpu_first_order_line_status.sql`, `cmpu_first_order_line_event_times.sql`, `cmpu_first_order_line_approval_distribution.sql`, `cmpu_upload_to_max_approval_extract.sql`, `cmpu_upload_entity_state_change_distributions.sql`, `cmpu_upload_entity_order_creation_distributions.sql`, `cmpu_study_population_uploaded_documents.sql` and `cmpu_study_population_prescription_decisions.sql`; the corresponding `src/python/create_cmpu_*.py` modules enforce line/entity grains. The frozen all-order membership query is retained under `data/opportunity/prescription-cohort-orders/`; financial enrichment uses `src/sql/prescription_cohort_order_sales.sql`.
- Financial and contacts: `src/sql/opportunity_order_cancellation_summary.sql`, `opportunity_aged_reorders.sql`, `opportunity_new_upload_customer_follow_up.sql`, `opportunity_new_vet_diet_customer_follow_up.sql`, and the three `opportunity_*chewy_follow_up.sql` templates; `opportunity_aged_first_vet_diet_order_customers.sql`, `opportunity_aged_first_customer_upload_vet_diet_order_customers.sql` and `opportunity_aged_customer_contacts.sql` define history and contact extracts. The corresponding opportunity Python generators aggregate outcomes; `src/python/create_opportunity_contact_frequency_reorders.py` combines contact segments and reorder outcomes. Retained rendered queries, source fingerprints and local verification SQL are under `data/opportunity/`.

DRAFT

3 References

[1] *RXP CMPU Vet Diet Cancellations Analysis*. GitHub repository. https://github.com/myunt1-chwy/rxp-cmpu-vet-diet-cancellations-analysis.

DRAFT